Communication and Tracking System for a Stratospheric Balloon

Module : ITRI671

Denzel Verster



orcid.org/0009-0002-8074-6007

Student number: 45036349

Degree and Year: *BSc Honours* in Computer Science and Information Technology 2026

Supervisor: Prof. Gunther Drevin
Co-supervisor: Mr. J. Heystek Grobler
Co-supervisor: Dr. Rudi Serfontein
Co-supervisor: Dr. Monica Barnard
Co-supervisor: Prof. Du Toit Strauss
Co-supervisor: Mr. Nathan Morgenrood
Co-supervisor: Mr. Bernard Swanepoel

Campus: Potchefstroom

Date: 20 October 2026

Phase: Final Report

Abstract

Stratospheric research balloons need reliable tracking and telemetry, yet they must stay within strict power, weight and bandwidth limits, and cellular networks are not designed to serve aerial users. This study asked how a reliable and power-efficient communication and tracking system can be built for a stratospheric balloon that operates at high altitudes. Following a design science research methodology within the pragmatic paradigm, a 433 MHz UHF communication and tracking system was designed, built and tested in three development cycles. The airborne unit reads GPS fixes on a Raspberry Pi Zero and transmits them through a Radiometrix NTX2 transmitter in a custom frame structure, with six copies of every frame. The ground station decodes the signal from an NRX2 receiver with an Arduino Uno R3 and logs it on a computer.

Stationary GPS tests over 22 sessions gave a mean positional deviation of 5.623 m and a maximum of 51.475 m. In six communication link tests at 5 to 20 m, at least one copy of every group was received, and 98.8% of copies arrived. In a balloon flight, all 18 packets sent after liftoff were received, up to an altitude of 5.4 km and a distance of 7.2 km, although only 85.2% of individual copies arrived. The power bank then stopped supplying the payload after about 17.5 minutes, and the payload was recovered with an independent GPS tracker and not with the developed system. The study delivered a tested prototype of both ends of the link. Reliability at stratospheric altitude, power consumption and independent recovery were not demonstrated, which requires a repeat flight with a tested power supply and a backup tracker.

Contents

Abstract **i**

List of figures **vi**

List of tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Introduction 1

1.2 Concepts central to the study 2

1.2.1 Telemetry systems in high-altitude platforms 2

1.2.2 UHF-based telemetry communication systems 2

1.2.3 GPS-based tracking systems 3

1.2.4 Overview of related literature 3

1.3 Problem description and objectives 4

1.3.1 Problem description 4

1.3.2 Research question 5

1.3.3 Aims and objectives 5

1.4 Research methodology 6

1.4.1 Research paradigm and methodology 6

1.4.2 Data collection 7

1.4.3 Research methods 7

1.4.4 Data analysis 8

1.4.5 The artefact project . . . 8
1.4.6 Project management and project plan . . . 9
1.4.7 Development platform, resources, and environment . . . 11
1.5 Ethical considerations . . . 11
1.5.1 Information leakage . . . 12
1.5.2 Spectrum and regulatory compliance . . . 12
1.5.3 Flight safety and payload recovery . . . 12
1.6 Provisional chapter division . . . 12
1.7 Conclusion . . . 13

2 Research methodology **14**

2.1 Introduction . . . 14
2.2 Research paradigms . . . 14
2.2.1 Positivism . . . 14
2.2.2 Interpretivism . . . 15
2.2.3 Critical theory . . . 15
2.2.4 Pragmatism . . . 15
2.3 Research methodology . . . 16
2.3.1 Design science research (DSR) . . . 16
2.3.2 Applicability of design science research . . . 17
2.3.3 Research rigour and validity . . . 18
2.4 Research methods . . . 18
2.4.1 Applied research methods: . . . 18
2.4.2 Research procedure . . . 19
2.5 Research design . . . 19
2.6 Conclusion . . . 20

3 Literature review **21**

3.1 Introduction . . . 21

3.2 Related literature 21
3.2.1 Telemetry systems in high-altitude platforms 22
3.2.2 UHF-based telemetry communication systems 23
3.2.3 GPS-based tracking systems 24
3.2.4 Additional considerations for high-altitude tracking and communication systems 25
3.3 How the literature guides this study 28
3.4 Conclusion 29

4 Empirical study **30**

4.1 Introduction 30
4.2 Artefact description 31
4.2.1 Purpose and scope 31
4.2.2 System architecture 31
4.2.3 System requirements 32
4.3 Description of the development process 33
4.3.1 Problem identification and objective definition 33
4.3.2 Design and development cycle 1 33
4.3.3 Design and development cycle 2 36
4.3.4 Ground station design and testing 40
4.4 Findings 43
4.4.1 GPS-based testing results 43
4.4.2 Communication link testing results 47
4.4.3 Flight results 49
4.4.4 Summary of findings 52
4.5 Conclusion 53

5 Conclusion **54**

5.1 Introduction 54

5.2 Overview of the research 54
5.3 Contributions of the research 56
5.4 Limitations and future research 56
5.4.1 Limitations 56
5.4.2 Future research 57
5.5 Evaluation of the study 57
5.5.1 Reliability 57
5.5.2 Validity 58
5.6 Reflection on the study 58
5.7 Conclusion of the study 59

Bibliography **59**

A Code of Conduct **63**

B Student Study Leader Agreement **65**

C Declaration of Integrity **67**

D Ethics Form **69**

E Programs **71**

E.1 Ground-based GPS acquisition and logging program 71
E.2 GPS data analysis program 74
E.3 GPS-NTX2 transmission program 78
E.4 Arduino-NRX2 receiver program 83
E.5 Receiver logging program 88
E.6 Flight Analysis program 91

List of Figures

1.1 Design Science Research (DSR) process model adopted from Peffers et al. [25] . . 6

1.2 Project Gantt Chart 9

2.1 Overview of the design science research (DSR) lifecycle 17

2.2 Research design followed in the study. 20

3.1 High-altitude balloon payload architecture proposed by Wiyagi et al. [37]. 27

4.1 UART wiring of the Raspberry Pi Zero and Uputronics GPS 34

4.2 Successful GPS data acquisition 35

4.3 Raspberry Pi Zero to NTX2 transmitter wiring 36

4.4 Completed NTX2 transmitter artefact 37

4.5 Airborne quarter-wave ground-plane antenna 37

4.6 Transmission program terminal output 40

4.7 NRX2 to Arduino Uno R3 wiring 40

4.8 Ground station antenna 41

4.9 Arduino Serial Monitor output 43

4.10 GPS log analysis terminal output 45

4.11 Receiver logging program terminal output 48

4.12 Altitude and number of satellites used against time relative to liftoff. 50

4.13 RSSI and copies decoded against distance to the ground 51

List of Tables

4.1 GPS positional consistency summary 44

4.2 UHF link performance 47

4.3 Airborne GPS data summary 49

4.4 Airborne Telemetry link performance 51

5.1 Outcome of the study objectives 55

List of Abbreviations

ADR	Adaptive Data Rate
AF	Audio Frequency
CRC	Cyclic Redundancy Check
DC	Direct Current
DSR	Design Science Research
FM	Frequency Modulation
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
ICASA	Independent Communications Authority of South Africa
ISM	Industrial, Scientific and Medical Band
JSON	JavaScript Object Notation
LEO	Low Earth orbit
LoRaWAN	Long Range Wide Area Network
LPWAN	Low-Power Wide-Area Network
NBFM	Narrowband Frequency Modulation
RSSI	Received Signal Strength Indicator
RXD	Receive Data
TXD	Transmit Data
UART	Universal Asynchronous Receiver-Transmitter
UHF	Ultra High Frequency
UKHAS	UK High Altitude Society
USB	Universal Serial Bus
WAN	Wide-Area Network

Chapter 1

Introduction

1.1 Introduction

Reliable communication at extreme altitudes remains a significant challenge in modern remote sensing and aerospace systems [16, 17]. Stratospheric research platforms are gaining more attention as alternative platforms for Earth observation and remote sensing applications [22].

Among these platforms, stratospheric research balloons that operate in the upper atmosphere provide an effective platform for the collection of atmospheric data and experimental research [35]. These types of systems require long-range communication and tracking capabilities to enable telemetry transmission to a ground station and the recovery of the stratospheric balloon [3, 27, 35]. The challenge of establishing communication and accurate position tracking of a stratospheric balloon is maintaining long-range communication under strict power, weight, and budget constraints [3, 27]. Low-power UHF communication systems operating in the 433 MHz band are commonly used for long-range telemetry applications due to their energy efficiency [26, 27]. Their suitability for balloon-based platforms has also been demonstrated in prior stratospheric deployments [3].

This study focuses on the design of a tracking and communication system that allows for essential telemetry data, like location data, to be efficiently transmitted to a ground station. This chapter covers the fundamental concepts that are relevant to this study, which is followed by a problem description and research objectives. The research methodology is outlined with ethical considerations as well. Finally, a provisional chapter division structure is presented to guide the remainder of this research project.

The chapter is organised as follows. The key concepts underlying the study, namely telemetry in high-altitude platforms, UHF communication and GPS tracking, are introduced in Section 1.2, followed by an overview of related literature. Section 1.3 presents the problem, research question, aims and objectives. Section 1.4 describes the research methodology, artefact, project plan, and development platform. Ethical considerations, including information leakage, spectrum compliance, flight safety and payload recovery, are addressed in Section 1.5. The provisional chapter division is presented in Section 1.6, followed by the conclusion of the chapter in Section 1.7.

1.2 Concepts central to the study

This section introduces the key concepts that underpin the communication and tracking system investigated in this study. The concepts provide the foundation for understanding the role of telemetry, UHF-based communication, and GPS-based tracking in stratospheric balloon applications.

1.2.1 Telemetry systems in high-altitude platforms

Telemetry systems are mainly used to collect, process, and transmit data gathered from remote platforms to a receiving station [27]. In the context of stratospheric balloon systems, telemetry plays a crucial role in enabling real-time monitoring during flight. The transmitted data typically includes variables such as GPS location, altitude, temperature, and system status [27, 3]. This information is essential not only for tracking the stratospheric balloon but also to increase the chances of payload recovery.

A telemetry system generally consists of three main components: data collection, data processing, and data transmission [27, 35]. Data collection involves the gathering of data from onboard sensors, such as GPS location from a GPS module. This data is then processed and formatted into a structure that is suitable for transmission. Finally, the data is transmitted to a ground station using a suitable communication medium [37].

High altitude telemetry systems have limited bandwidth and power usage constraints [14, 21, 27]. Limited transmission rates require careful selection and prioritisation of data to ensure that only essential information is communicated. The power limitations also require efficient transmission strategies, such as reducing transmission frequency and optimising packet size. These constraints emphasise the importance of designing telemetry systems that balance data accuracy, transmission reliability, and energy efficiency, particularly in applications such as stratospheric balloon systems where resources are limited.

1.2.2 UHF-based telemetry communication systems

Ultra High Frequency (UHF) communication systems operating in the Industrial, Scientific and Medical (ISM) bands, such as 433 MHz, are widely used for long-range, low-power wireless communication [21, 26]. These systems are suitable for stratospheric balloon applications due to their reliability in achieving long-distance line-of-sight communication with relatively low power consumption [3, 26, 35]. Unlike higher frequency communication technologies, UHF signals experience less attenuation over long distances and are more tolerant to environmental factors, making them effective for high-altitude applications where infrastructure-based communication, such as cellular networks, is unavailable [11, 17, 26].

In the context of this study, UHF communication provides a suitable and practical solution for maintaining telemetry transmission between the stratospheric balloon and the ground station. Its low power requirements and long range capabilities make it well-suited to the constraints of high-altitude systems, where energy efficiency and communication reliability are critical.

1.2.3 GPS-based tracking systems

Global Positioning System (GPS) technology enables position tracking by receiving signals from multiple satellites and computing receiver location using signal timing [6]. GPS-based tracking systems are essential for monitoring the geographical location and trajectory of a stratospheric balloon during flight, allowing operators to track movement in real time and support payload recovery operations [6, 14, 27].

A GPS module in a system typically receives continuous satellite signals and outputs positioning data in standard formats [6]. This data is then processed and integrated into the telemetry system for transmission to a ground station. The integration of GPS data with telemetry communication enables continuous situational awareness, which is critical for the successful operation of high-altitude balloon systems [6, 14].

Although GPS provides reliable global positioning data, its effectiveness in high-altitude applications depends on several factors, such as signal availability, antenna performance, and system integration [6, 14, 36]. Additionally, the transmission of GPS data must be carefully managed to balance tracking accuracy with power consumption and communication constraints, particularly when operating over long distances with limited bandwidth.

In the context of this study, GPS data should also be available to the onboard stratospheric balloon system to enable capabilities like geofencing, which will limit the available travel area of the stratospheric balloon. Furthermore, redundancy mechanisms for tracking are important to improve the likelihood of payload recovery in the event of communication failure. These considerations highlight the importance of efficient integration between GPS tracking and the communication subsystem [14].

1.2.4 Overview of related literature

Existing literature indicates that stratospheric balloon platforms are increasingly used as cost-effective alternatives to satellites for atmospheric sensing, experimentation, and remote data collection [22, 35]. These systems depend heavily on reliable telemetry and tracking subsystems to maintain communication with a ground station and support payload recovery.

Previous studies have demonstrated the use of UHF and sub-GHz communication technologies for long-range, low-power telemetry applications [3, 26]. These technologies are generally preferred over higher-frequency alternatives due to favourable propagation characteristics and lower power requirements. However, many existing studies focus on communication range or link feasibility rather than complete system integration.

GPS-based tracking systems are widely used for remote platform positioning and recovery operations [6, 14]. While GPS location acquisition is well established, effective transmission of positioning data remains dependent on the reliability of the telemetry subsystem.

Research also suggests that conventional cellular communication systems are not ideal for high-altitude platforms because network coverage is optimised for terrestrial users rather than aerial systems [4]. This limits the practicality of cellular-only solutions for stratospheric balloon missions.

Although prior studies provide valuable insight into communication technologies, telemetry architectures, and tracking systems, less attention has been given to integrated low-power solutions specifically designed for stratospheric balloon applications. A need therefore exists for a system that combines efficient telemetry transmission, reliable tracking, and practical recovery support under high-altitude operating constraints.

This study contributes by designing, implementing, and evaluating an integrated low-power communication and tracking system for a stratospheric balloon. The proposed artefact combines GPS-based positioning, UHF telemetry transmission, and recovery-oriented system features within a single platform intended for operation under realistic power, range, and environmental constraints.

A more comprehensive review of related literature is presented later in Chapter 3.

1.3 Problem description and objectives

This section describes the problem addressed by the study and establishes the research question, aim, and objectives that guide the development and evaluation of the proposed communication and tracking system.

1.3.1 Problem description

Stratospheric research balloon systems require reliable and efficient tracking and communication with a ground station for essential telemetry and payload recovery. However, operating a stratospheric balloon at altitudes of between twenty and thirty kilometres introduces several technical challenges, including low power constraints, long communication distances, and the absence of communication infrastructure like cellular networks [3, 4, 11, 17]. These challenges make it difficult to maintain continuous tracking and telemetry transmission [21].

Existing communication and tracking solutions, such as cellular-based systems, are primarily designed for terrestrial coverage and user distribution on the ground. Cellular networks are primarily designed for ground coverage rather than high-altitude aerial platforms, resulting in limited applicability for stratospheric balloon communication systems [4]. In addition, studies on aerial cellular coverage indicate that signal geometry, antenna orientation, and line-of-sight constraints significantly reduce reliability at high altitudes, making cellular communication unsuitable for sustained telemetry at stratospheric operating heights [4].

Additionally, the limited data rates of narrowband UHF communication systems require careful optimisation of telemetry payload size and transmission intervals [21, 26].

Although GPS will provide for global positioning capability, the reliable and effective transmission of this data to ground stations remains a challenge [6, 14]. Redundancies for tracking and payload recovery will also have to be implemented [3, 14].

This clearly indicates the need for a system-level design that integrates GPS tracking, efficient telemetry management, and long-range communication in a manner that ensures efficient position tracking and energy efficiency under stratospheric conditions.

1.3.2 Research question

How can a reliable and power-efficient communication and tracking system be built for a stratospheric balloon system that can operate at high altitudes?

1.3.3 Aims and objectives

This study aims to achieve the following:

- To design a telemetry system capable of collecting and transmitting essential flight data to a ground station
- To develop a GPS-based tracking system for real-time global position monitoring
- Implement a long-range, low-power communication system suitable for high-altitude environments
- Optimise telemetry data transmission for optimal efficiency under bandwidth and power constraints
- Establish methods of redundancies for communication and payload recovery
- To enable reliable tracking and recovery of the stratospheric balloon payload
- To evaluate the performance of the system through GPS-based and communication-link testing

These aims and objectives guide the design, implementation and evaluation of the proposed communication and tracking system. By addressing both the technical challenges of high-altitude communication and the practical requirements of reliable tracking and payload recovery, this study aims to develop a reliable and efficient solution that meets the constraints of stratospheric balloon applications. Achieving these objectives is expected to address the research question and assess the feasibility of a low-power, long-range communication and tracking system at high altitudes. The expected contribution of this study is a validated prototype and design framework for reliable low-power balloon telemetry systems.

1.4 Research methodology

The research methodology section covers the research methodology and paradigm selected for the development of the telemetry system. Data collection and research methods are discussed further followed by the data analysis that will be conducted, the final artefact and then the project management and project plan. Finally, the development platform, resources and environment is also discussed.

1.4.1 Research paradigm and methodology

This study is grounded in the Pragmatic paradigm and will follow a Design Science Research (DSR) methodology. Pragmatism assumes that the value of knowledge lies in its practical applicability and usefulness in solving real-world problems [7]. This is suitable for the current study because the performance of the tracking and communication system is influenced by real technical and environmental factors such as radio propagation, atmospheric conditions, hardware limitations, and power constraints.

DSR is an approach to problem-solving that enhances knowledge through the creation of artefacts. DSR contributes to scientific and technological understanding by developing solutions that address practical problems [24, 25].

The Design Science Research (DSR) process follows an iterative methodology consisting of problem identification, artefact design and development, demonstration, and evaluation cycles. The process model guides the structured development and assessment of the proposed system artefact. Figure 1.1 illustrates the DSR process model used in this study.

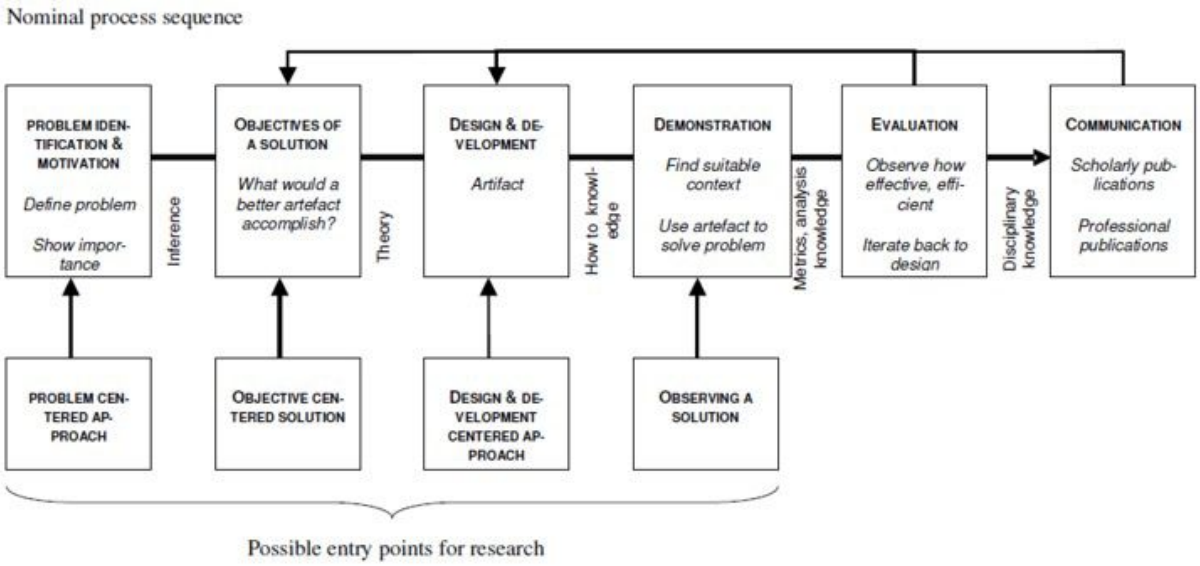


Figure 1.1: Design Science Research (DSR) process model adopted from Peffers et al. [25]

In this study, the artefact produced through the DSR process is a communication and tracking system for stratospheric balloon applications. This research methodology is applicable as this study aims to design and evaluate a communication and tracking system for a stratospheric balloon. Multiple stratospheric balloon launches will take place to test and evaluate the functioning, effectiveness, and efficiency of a communication and tracking system.

A quantitative research approach is selected, as the performance of the system will be evaluated using measurable metrics such as communication range, signal strength, packet transmission success rate, and GPS accuracy. This allows for objective assessment and comparison against defined performance criteria.

1.4.2 Data collection

Data will be collected during the testing and evaluation of the communication and tracking system through a combination of GPS-based testing and communication-link testing.

The primary data collected will include:

- GPS coordinates (latitude, longitude, altitude)
- Signal strength indicators (RSSI)
- Packet transmission success and failure rates
- Transmission intervals and timing data
- System status data (e.g. power levels and system uptime)

Data will be collected using an onboard telemetry system consisting of a GPS module, single-board computer, and UHF transmitter. The transmitted telemetry data will be received by a ground station equipped with a UHF receiver and logged using data logging software.

Additional data may be collected through controlled ground testing to evaluate communication range and reliability under line-of-sight conditions [11]. All collected data will be stored in structured log files for further analysis.

1.4.3 Research methods

The research methods applied in this study are aimed at evaluating the performance, reliability, and effectiveness of the proposed communication and tracking system under both controlled and real-world conditions. A combination of experimental, field-based and iterative development approaches will be used to ensure a comprehensive assessment of the system.

- Experimental testing - Measuring communication performance under controlled conditions
- Field testing - High-altitude balloon launches [31]
- Iterative prototyping - Improvements made based on results from each testing phase

These methods support the DSR approach by enabling the systematic design, testing, and improvement of the system. Experimental testing will provide for controlled performance evaluation, while field testing validates real-world applicability [25]. Iterative prototyping ensures continuous improvement of the system based on observed results, contributing to a robust and reliable final design.

1.4.4 Data analysis

The collected data will be analysed to evaluate the performance, reliability, and efficiency of the communication and tracking system.

Key performance metrics will include:

- Maximum communication range
- Packet reception success rate
- Signal strength
- Transmission interval accuracy
- GPS accuracy and consistency

Data analysis will be performed using software tools capable of processing and visualising telemetry data. The collected data will be compared against expected performance benchmarks to determine whether the system meets the defined objectives.

Graphs and statistical summaries will be used to identify trends, such as signal degradation over distance and reliability under different conditions. The results of this analysis will be used to evaluate system performance and identify areas for improvement. The system will be evaluated against criteria such as reliable communication range, acceptable packet success rate, and consistent GPS accuracy during flight.

1.4.5 The artefact project

The artefact of this mini-dissertation is a communication and tracking system designed specifically for stratospheric balloon applications. The system integrates GPS-based tracking with a UHF telemetry communication link to enable reliable transmission of essential data to a ground station.

The system consists of the following components:

- A GPS module for acquiring real-time positional data
- A single-board computer for processing and formatting telemetry data
- A UHF transmitter and receiver pair for long-range communication

- An Arduino Uno R3 for decoding transmissions

The purpose of the system is to provide reliable position tracking and telemetry transmission under high-altitude operation, limited power, bandwidth, and environmental constraints.

The development of the system will follow a structured lifecycle consisting of design, implementation, testing, and evaluation phases. Iterative improvements will be made based on testing results to optimise system performance and reliability.

1.4.6 Project management and project plan

This project will be managed using an iterative approach aligned with the Design Science Research (DSR) methodology. The project will follow the core DSR phases, adapted for this study as follows:

- Problem identification and requirement analysis
- Definition of system objectives and specifications
- Design and development of the communication and tracking system
- Demonstration through GPS-based and communication-link testing
- Evaluation of system performance using defined metrics
- Refinement and optimisation based on evaluation results

These phases will be executed iteratively, allowing improvements to be made to the system after each testing and evaluation cycle. Milestones will be defined for each phase to track progress and ensure timely completion of the project.

The milestones include completion of system design, successful ground testing, initial flight testing, and final system evaluation. Below, a Gantt chart Figure 1.2 is used to visually represent the project timeline, including task durations, dependencies, and key milestones.

Gantt chart

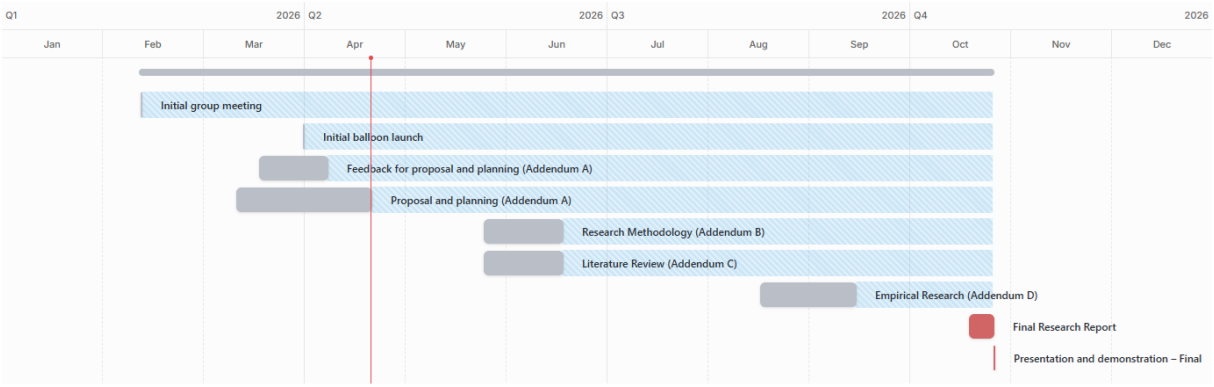


Figure 1.2: Project Gantt Chart

Integration with group project

This study forms part of a broader group project that involves three Honours students, where each student is tasked with developing a subsystem of a stratospheric balloon.

The project is divided among students as follows:

- This study focuses on the development and design of a tracking and communication subsystem, which is responsible for telemetry transmission and position tracking of the stratospheric balloon.
- A second project focuses on the design of the control system and onboard software environment, which manages system processes, payload release mechanisms and geofencing capabilities.
- The third project focuses on a sensor subsystem that is primarily responsible for collecting atmospheric data using a particulate sensor during the flight of the stratospheric balloon.

The communication and tracking subsystem will receive processed data from the control system and sensor subsystem, while integrating GPS data within the communication subsystem, and then package the information into telemetry data and transmit it to a ground station. This establishes a clear dependency where the system relies on accurate data input from other subsystems, while simultaneously enabling system-wide monitoring and recovery capabilities.

Although the overall project is developed collaboratively, this study maintains a clear focus on the communication and tracking subsystem. The design, implementation, and evaluation of this subsystem are conducted independently, ensuring that the research contribution remains distinct and aligned with the objectives of this study.

Scope, limitations, and risk analysis

The scope of this study is limited to the design and evaluation of a communication and tracking system for a stratospheric balloon operating in the UHF frequency band [26, 11]. The system will focus on transmitting essential telemetry data, including GPS location and system status, to a ground station.

Limitations of the study include constraints related to power consumption, hardware capabilities, and environmental factors such as atmospheric conditions. Additionally, the system will be evaluated under controlled test conditions, which may not fully represent all real-world scenarios.

Potential risks include communication signal loss, hardware failure after launch, and challenges in payload recovery. These risks will be mitigated through the implementation of redundancy mechanisms, careful system design, and pre-flight testing procedures.

1.4.7 Development platform, resources, and environment

The development of the communication and tracking system will utilise a combination of hardware and software resources. The hardware platform will include a UPUTRONICS GPS module for location tracking [6], a RADIOMETRIX UHF transmitter and receiver operating in the 433 MHz band for communication [11, 26], and a Raspberry Pi single-board computer for data processing and system control [34].

Software development will be conducted using embedded programming environments suitable for single-board computer-based systems, primarily using Python for system control and data handling on the Raspberry Pi, which is widely used in low-power embedded and prototype telemetry systems [34]. Data analysis and testing will be supported by telemetry logging utilities and data visualisation tools such as Python-based plotting libraries.

The components consist of the following:

- UPUTRONICS GPS module
- RADIOMETRIX UHF transmitter
- RADIOMETRIX UHF receiver
- Raspberry Pi single-board computer
- Arduino Uno R3 microcontroller
- Laptop running a Windows 10 operating system
- Power supply system (battery-based)

The selected components are chosen based on their low power consumption, reliability, and suitability for high-altitude and long-range telemetry applications, as supported by existing LPWAN and stratospheric balloon studies [3, 26, 35]. The development environment enables fast prototyping and testing, enabling iterative improvement of the system design.

1.5 Ethical considerations

This section considers and discusses the main ethical considerations associated with the testing and deployment of the tracking and communication artefact. These considerations include the potential leakage of information through telemetry transmissions, compliance with applicable radio-frequency spectrum regulations, and the safety of the balloon system during flight and payload recovery. Addressing these considerations is important to ensure that the development and testing of the artefact are conducted responsibly and that potential risks to people, property, and the surrounding environment are appropriately considered.

1.5.1 Information leakage

Telemetry systems used in remote sensing and high-altitude platforms may unintentionally expose operational information even when the transmitted data packet is encrypted. This is possible, as shown in [18], because information can still be deduced by an outside observer from transmission behaviour alone, even if the data being transmitted has been encrypted.

To mitigate leaking information, this system will only transmit essential data, such as GPS location, in time intervals of around 1 minute.

1.5.2 Spectrum and regulatory compliance

The system operates in the 433 MHz ISM band, which is subject to regulatory constraints on transmission power and usage conditions [12, 13]. Therefore, the design of the telemetry system considers regulatory limitations associated with low-power telemetry systems to ensure compliant operation.

The use of sub-GHz ISM band communication is consistent with low-power wider area networking principles, where long-range connectivity is achieved under strict energy and spectrum constraints [26].

1.5.3 Flight safety and payload recovery

Stratospheric balloon systems introduce operational risks related to uncontrolled descent, payload separation, and environmental interaction during recovery, as identified in high-altitude platform system studies [8, 35]. These risks must be managed to ensure the safety of people, property, and equipment.

To address these risks, controlled ground testing is performed before launching the stratospheric balloon, and recovery procedures are pre-planned based on expected flight trajectories [16]. Redundant tracking mechanisms are also considered to improve payload location accuracy in the event of communication failure [3, 35].

These measures ensure that system testing and deployment are conducted in a controlled and responsible manner, consistent with experimental aerospace safety practices.

1.6 Provisional chapter division

This mini-dissertation will contain the following chapters:

1. Introduction:

   Chapter 1, the introduction to this research project, describes the context and important concepts surrounding the tracking and communication of a stratospheric research balloon system.

2. Research Methodology:

   The research methodology chapter discusses the research methodology and research paradigm selected to evaluate and improve a tracking and communication system for a stratospheric balloon.

3. Literature Review:

   This chapter discusses the related literature that is referenced throughout this research project. This will include articles related to the communication and tracking of a remote research platform that accounts for communication with a ground station for GPS telemetry.

4. Empirical Research:

   This chapter focuses on the design, development, and testing of the proposed communication and tracking system. A design and development approach will be followed, where the system is implemented, tested, and refined through iterative cycles. The performance of the system will be evaluated based on the collected data and defined performance metrics.

5. Conclusion:

   Finally, the conclusion chapter provides the final synthesis of the research project. The conclusion consists of the findings and results of the developed stratospheric research balloon system.

1.7 Conclusion

Chapter 1 discussed the research topic by outlining the context, challenges, and objectives associated with communication and tracking in stratospheric balloon systems. Key concepts of the system relevant to the study were discussed, along with an overview of related literature to establish the foundation for the research.

The problem was defined, a research question, aims, and measurable objectives were formulated to guide this mini-dissertation. The proposed research methodology, project plan, and system design were also explained, along with considerations related to ethics and project execution.

The next chapter will further elaborate on the DSR methodology, providing a discussion of the research paradigm, methods, and design that will be used to conduct this study.

Chapter 2

Research methodology

2.1 Introduction

This chapter entails the philosophical and methodological approaches adopted for the study. The chapter starts by examining research paradigms relevant to computing research and motivates why the Pragmatic paradigm is selected. Thereafter, the design science research (DSR) methodology is discussed in relation to the development of the proposed artefact. The chapter further explains the research methods, data collection procedures, and data analysis techniques used during the study.

The following content in this chapter consists of the paradigms applicable to this study, followed by an explanation of the selected research methodology and the procedures followed to ensure rigour and validity. This chapter also presents the research methods used in the study and concludes with a detailed discussion of the research design and overall structure of the study.

2.2 Research paradigms

Research paradigms provide the philosophical foundation that guides how knowledge is created, interpreted, and evaluated within a study [15]. In computing and information systems research, several paradigms are typically applied depending on the nature of the research problem and the objectives of the study [25]. This study considers several research paradigms to determine their applicability for the development and evaluation of the proposed communication and tracking system.

2.2.1 Positivism

Positivism is based on the assumption that reality exists independently of human interpretation and can therefore be objectively measured and analysed through observable data [15, 23]. Positivist studies usually make use of quantitative methods, experimentation, and statistical analysis to produce measurable and reliable findings [5, 30].

The positivist paradigm is partially applicable to this study because the proposed system will be evaluated using measurable performance metrics like communication range, signal strength, packet transmission success rate, and GPS accuracy [23, 30]. These measurements enable objective evaluation of the system under controlled and operational conditions [25, 30].

This study also focuses on the iterative development and refinement of a technological artefact intended to solve a practical engineering problem [24, 25]. The study, therefore, extends beyond pure observation and measurement, making the positivist paradigm not fully applicable [25].

2.2.2 Interpretivism

Interpretivism focuses on understanding human experiences, perceptions, and subjective meanings within a social context [5, 15]. Interpretivist studies generally make use of qualitative methods such as interviews, observations, and thematic analysis to understand how individuals interpret their experiences [5, 29].

Although interpretivism is valuable in studies involving human behaviour and social interaction, it is an unsuitable paradigm for this study [5, 30]. This study does not investigate subjective human experiences or perceptions, but rather focuses on the technical design and evaluation of a communication and tracking system based on measurable engineering performance criteria.

2.2.3 Critical theory

Critical theory is a paradigm concerned with identifying and addressing societal inequalities, power structures, and social transformation [5, 9]. The research conducted using this paradigm often focuses on challenging dominant systems and promoting social change [5].

Critical Theory is not applicable to this study because the research does not involve political, social, or ideological critique. Instead, the study focuses on the technical development and evaluation of a telemetry and tracking artefact for high-altitude balloon applications.

2.2.4 Pragmatism

Pragmatism focuses on practical problem solving and the usefulness of knowledge in real-world applications [5, 7]. Pragmatic research emphasises selecting methods and approaches that address the research problem the best, rather than adhering strictly to a single philosophical position [5, 7].

The Pragmatic paradigm is suitable for this study because the research aims to design and evaluate a practical engineering solution to the communication and tracking challenges associated with stratospheric balloon systems. The effectiveness of the proposed system depends on real-world operational factors like communication reliability, atmospheric conditions, signal propagation, hardware constraints, and power efficiency [25, 27, 35].

This study focuses on developing a functional and reliable system capable of operating effectively under high-altitude conditions. The Pragmatic paradigm supports the combination of engineering design, experimentation, iterative improvement, and quantitative evaluation required to achieve the objectives of this study.

Based on the characteristics of the study and the objectives of the research, the Pragmatic paradigm is considered the most appropriate philosophical foundation for this investigation [5, 7].

2.3 Research methodology

Research methodology refers to the systematic approach used to conduct research within the context of a specific research paradigm. The methodology guides how the study is structured, how knowledge is generated, and how the validity and reliability of the findings are ensured [5, 25].

2.3.1 Design science research (DSR)

This study follows a DSR methodology. DSR focuses on the creation, development, and evaluation of artefacts intended to solve identified real-world problems [24, 25].

DSR contributes to scientific and technological knowledge through the development of innovative artefacts and the evaluation of their effectiveness in addressing practical challenges [24, 25]. The methodology is commonly applied in computing and information systems research where technological systems or solutions are developed to address engineering or operational problems [2, 24].

The DSR process generally consists of the phases illustrated in Figure 2.1

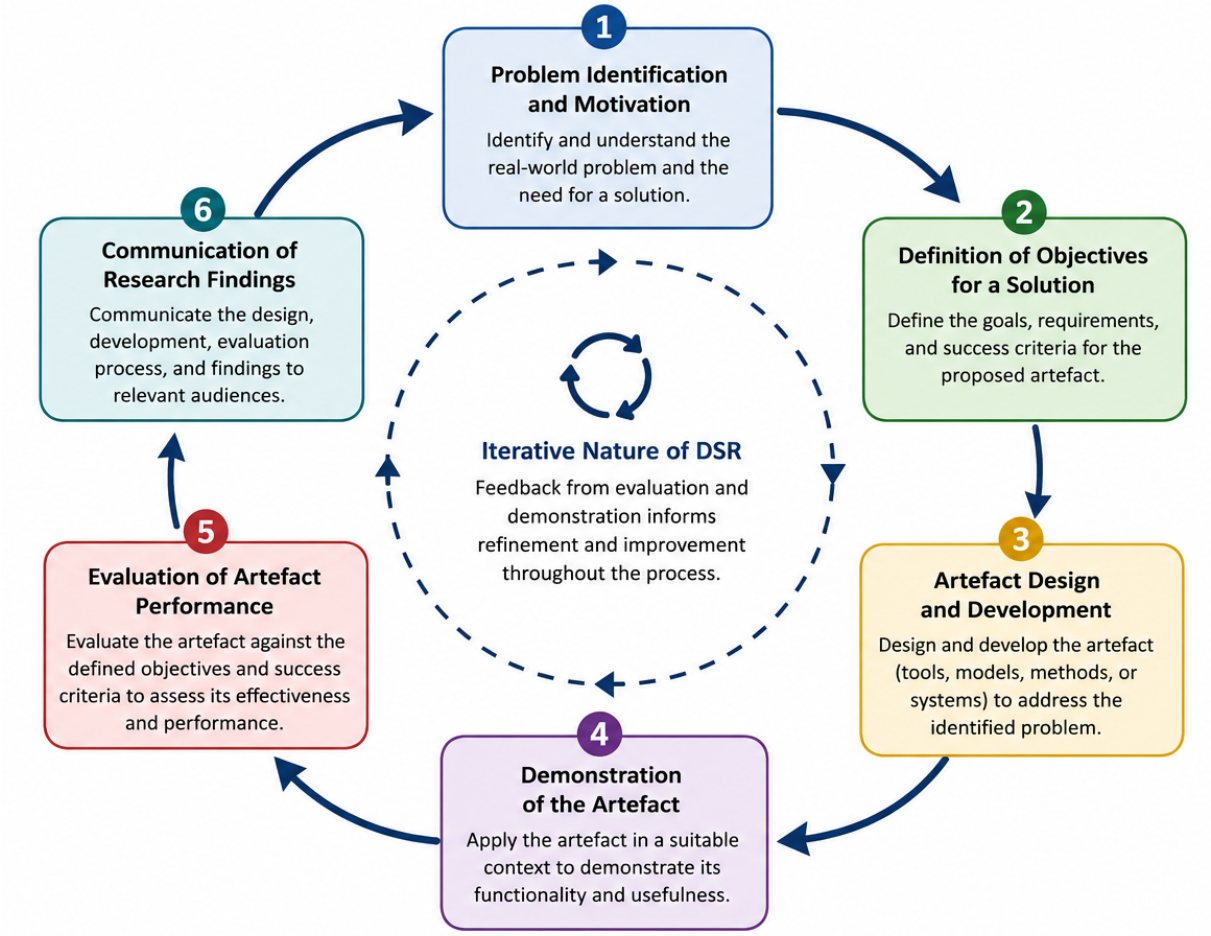


Figure 2.1: Overview of the design science research (DSR) lifecycle

The iterative nature of DSR enables continuous refinement and improvement of the artefact throughout the research process [2, 25].

2.3.2 Applicability of design science research

The DSR methodology is appropriate for this study as it focuses on the development of an artefact intended to solve a practical engineering problem. In this case, the artefact is a communication and tracking system for stratospheric balloon applications, designed to address challenges related to telemetry transmission and position tracking under high-altitude constraints [24, 25, 27, 37].

DSR is further suitable because the artefact will be iteratively designed, implemented, and evaluated through both controlled experiments and real-world high-altitude balloon testing [25, 31, 35]. The outcomes of each evaluation cycle are used to refine and improve system performance, aligning with the iterative nature of the DSR process [25].

2.3.3 Research rigour and validity

Research rigour refers to the systematic and transparent execution of research to ensure that findings are reliable and reproducible [5]. In this study, rigour is achieved through structured testing procedures, repeatable experiments, and consistent data collection methods. Rigour is further reinforced through the iterative development cycles of the DSR methodology, in which continuous evaluation and refinement of the system occur throughout the study [2, 25].

Reliability is supported through repeated experimental and field tests under controlled conditions [5]. Communication range, GPS accuracy, and telemetry performance are measured multiple times using standardised logging procedures to ensure consistency in results.

Validity is also ensured through clearly defined performance metrics aligned with the study objectives, including signal strength, packet success rate, transmission consistency, and GPS accuracy [5]. These metrics ensure that system evaluation reflects actual operational performance requirements [5]. External validity is strengthened through high-altitude balloon testing, which exposes the system to real environmental and atmospheric conditions relevant to its intended application [31, 35, 37].

2.4 Research methods

The study applies a combination of experimental, field-based, and analytical research methods to evaluate the proposed communication and tracking system. These methods support the iterative nature of the DSR methodology by enabling continuous refinement of the artefact based on measurable performance outcomes. The selected methods focus primarily on evaluating communication performance, telemetry reliability, positioning accuracy, and system robustness under realistic operating conditions.

2.4.1 Applied research methods:

- Experimental testing:

  Experimental testing involves evaluating the communication and tracking system under controlled conditions to measure performance metrics such as communication range, signal strength, packet transmission success rate, and GPS accuracy [16, 27]. This allows objective assessment of system functionality under repeatable conditions.

- Field testing:

  Field testing involves high-altitude balloon launches to evaluate system performance under real operational conditions. These tests assess telemetry reliability, GPS tracking continuity, and communication stability in uncontrolled atmospheric environments [31, 35, 37]. Field testing is necessary because environmental conditions such as altitude, atmospheric variation, and distance significantly affect communication performance.

- Iterative prototyping:

  Iterative prototyping requires continuously refining the communication and tracking system based on results obtained from both controlled and field testing. Each iteration improves system performance, reliability, and efficiency in alignment with DSR principles [25].

- Quantitative data analysis:

  Quantitative data analysis involves evaluating telemetry data using measurable performance metrics such as communication range, signal strength, packet transmission success rate, GPS accuracy, and transmission interval consistency [5]. Statistical summaries and graphical analysis are used to assess system performance objectively.

2.4.2 Research procedure

The research procedure follows an iterative DSR process consisting of requirements analysis, literature review, system design, implementation, controlled experimental testing, field testing, quantitative data analysis, and iterative refinement of the artefact [25].

The study begins with identifying communication and tracking requirements for high-altitude balloon systems through literature analysis [27, 35, 37]. Based on these requirements, the communication and tracking system is designed and implemented using selected hardware and software components. Controlled testing is then conducted to evaluate baseline system performance under repeatable conditions before progressing to real-world field testing during high-altitude balloon launches.

Telemetry and tracking data collected during testing are analysed quantitatively using various performance metrics such as communication range, signal strength, packet transmission success rate, and GPS accuracy. The results gathered from testing phases are used to refine and improve the system iteratively until acceptable operational performance is achieved.

2.5 Research design

The research design follows an iterative DSR structure consisting of problem identification, system design, implementation, testing, quantitative evaluation, and artefact refinement. Controlled experimental testing is conducted before real-world field testing through high-altitude balloon launches. Results obtained during evaluation are used to refine the system iteratively until the required performance objectives are achieved.

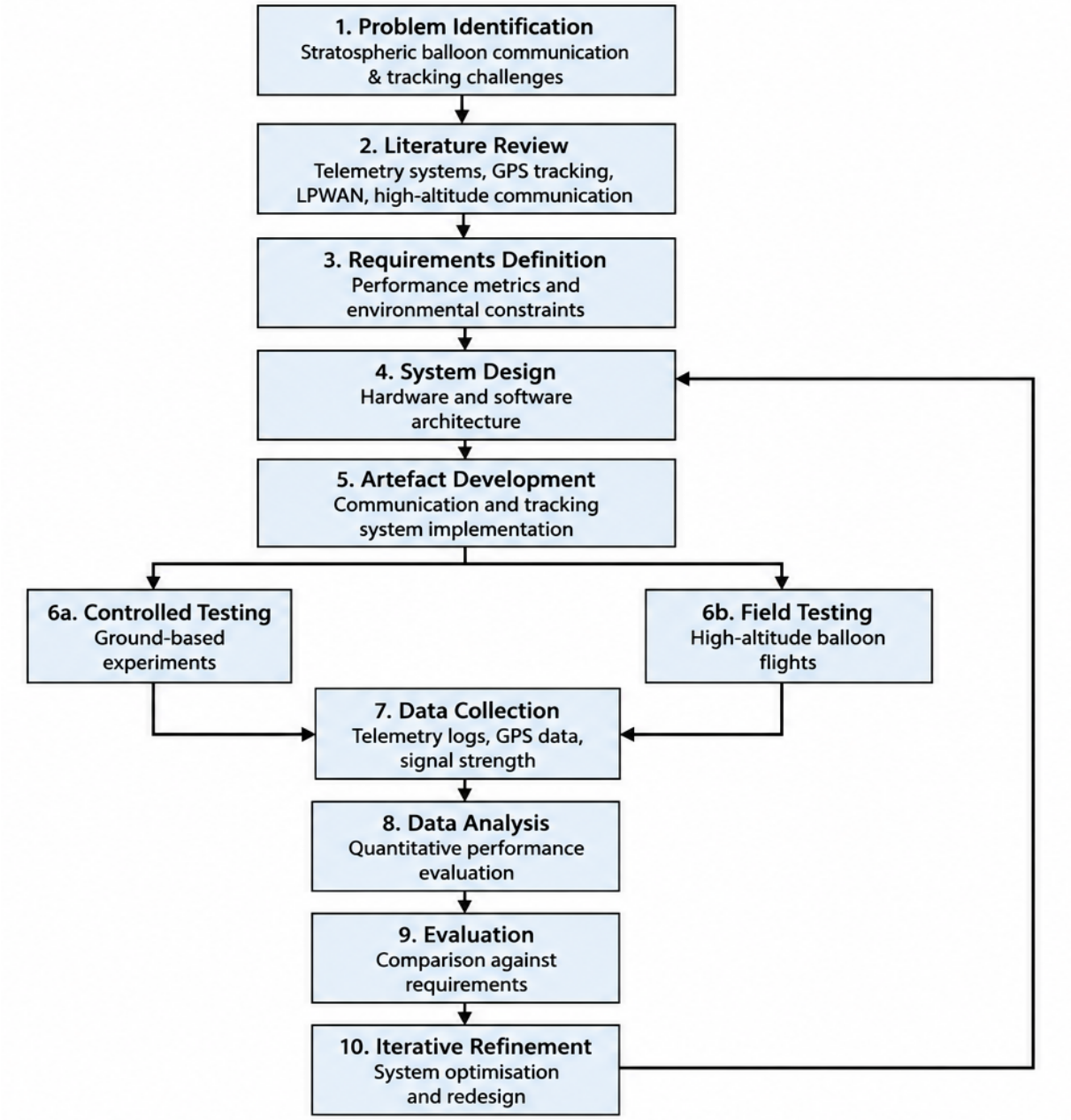


Figure 2.2: Research design followed in the study.

The artefact is evaluated through both controlled and real-world testing environments. Telemetry data collected during testing is analysed quantitatively to determine whether system objectives are achieved, with results used to refine the system iteratively [24, 25].

2.6 Conclusion

This chapter outlined the research methodology adopted for this study, including the pragmatic paradigm and DSR approach. It described the experimental and field testing methods used to evaluate system performance, as well as the quantitative techniques used for data analysis. The research design provides a structured framework for developing and evaluating the communication and tracking system under realistic operating conditions. The next chapter presents the literature review supporting the study.

Chapter 3

Literature review

3.1 Introduction

Reliable communication and tracking capabilities are fundamental requirements for high-altitude balloon systems because they enable telemetry transmission, position monitoring, efficient system operation, and payload recovery [27, 35, 37]. As a result, it is necessary to perform proper investigations on existing literature that will support and guide the development and completion of the proposed system.

This chapter presents a review of the literature used in this study. The reviewed literature provides insight into existing technologies, design approaches, operational limitations, and performance considerations associated with communication and tracking systems.

Literature relating to the key concepts central to this study is reviewed first, followed by additional literature relevant to the broader communication and tracking environment. Finally, the chapter discusses how the literature guides the design, implementation, and evaluation of the proposed communication and tracking system.

3.2 Related literature

The development of a communication and tracking system for a stratospheric balloon requires the integration of several technologies that must operate reliably within a demanding high-altitude environment [35, 37]. These technologies include telemetry systems for transmitting mission data, long-range communication systems for maintaining connectivity, and positioning systems for tracking and payload recovery [27, 37]. Existing research provides valuable insight into the capabilities, limitations, and design considerations associated with these technologies. Reviewing this literature establishes the theoretical foundation of the study, identifies proven approaches, and highlights challenges that must be addressed when developing a communication and tracking solution for stratospheric balloon applications.

This section reviews literature related to the key concepts identified in Chapter 1. The discussion focuses on technologies and design approaches relevant to the development of communication and tracking systems for high-altitude balloon applications.

3.2.1 Telemetry systems in high-altitude platforms

Telemetry systems are fundamental to the operation of high-altitude balloon missions because they provide the mechanism through which flight data, environmental measurements, and payload status information are communicated to ground operators [27, 37]. As balloon missions increasingly support scientific observation, environmental monitoring, and experimental communication systems, reliable telemetry has become essential for mission monitoring, operational decision-making, and payload recovery [27, 37]. Therefore, a considerable amount of research has focused on the design of telemetry systems capable of operating reliably over large distances while remaining energy efficient and lightweight [16, 27].

Telemetry systems generally consist of data acquisition, data processing, and communication subsystems that collectively enable the collection and transmission of information from remote platforms [27, 37]. In high-altitude balloon applications, telemetry data commonly includes GPS coordinates, altitude measurements, environmental sensor readings, battery status information, and system health indicators.

Puričer et al. describe telemetry systems specifically developed for stratospheric balloon applications and emphasise the importance of maintaining communication throughout all mission phases [27]. The authors highlight that telemetry reliability directly influences flight monitoring capabilities and the probability of successful payload recovery.

Wiyagi et al. present a high-altitude balloon payload architecture comprising sensing, processing, communication, positioning, and power management subsystems [37]. Their work illustrates the central role telemetry systems play in integrating data generated by multiple sensors and transmitting the resulting information to ground stations. The architecture further demonstrates the multidisciplinary nature of high-altitude payload design, where telemetry acts as the interface between onboard systems and mission operators.

Brown developed a multi-mode telemetry transmitter for high-altitude balloon applications and demonstrated the importance of communication redundancy in improving mission reliability [3]. The study showed that employing multiple communication mechanisms increases the probability of maintaining connectivity throughout the flight and improves payload recovery success rates.

Recent developments in telemetry communication have explored the use of low-power wide-area networking technologies. Krueger and Bergmaier investigated the use of LoRaWAN as a telemetry solution for high-altitude balloons and reported successful reception of more than 95% of transmitted telemetry data through a public LPWAN infrastructure [16]. The study demonstrated that modern telemetry systems can achieve reliable communication while simultaneously reducing system size, weight, and power consumption.

Experimental results presented by Krueger and Bergmaier further indicate that telemetry communication remained reliable at altitudes exceeding 2.6×10^1 km, with minimal packet loss recorded during flight testing [16]. These findings suggest that LPWAN technologies provide a viable alternative to traditional telemetry solutions for high-altitude applications.

The reviewed literature demonstrates that telemetry systems are critical components of high-altitude balloon missions. Reliable telemetry communication enables flight monitoring, supports mission operations, and improves payload recovery capabilities. Research further highlights the growing adoption of low-power communication technologies that provide long-range telemetry performance while maintaining low energy consumption, which also motivates the selection of LPWAN technology.

3.2.2 UHF-based telemetry communication systems

One of the primary challenges associated with high-altitude telemetry communication is maintaining reliable connectivity over large distances while operating within strict payload constraints. Communication technologies used in balloon systems must therefore provide long communication ranges, low power consumption, and acceptable reliability despite limited spectrum availability [17, 26]. These requirements motivated significant research into UHF and sub-GHz communication technologies, particularly LPWAN systems that are specifically designed for long-range, low-data-rate applications.

The extensive operational areas associated with high-altitude balloon missions require communication systems capable of maintaining connectivity across large geographical regions [22]. Traditional short-range wireless technologies often fail to meet these requirements due to limited transmission ranges and relatively high power consumption [11, 26]. Consequently, sub-GHz communication technologies have emerged as attractive alternatives for telemetry applications.

De Poorter et al. identified sub-GHz communication systems as an important class of LPWAN technologies capable of achieving communication ranges between 1 km and 50 km while maintaining low energy consumption [26]. According to the authors, the favourable propagation characteristics of lower frequencies enable larger coverage areas and improved penetration compared to higher-frequency alternatives.

The migration from the 2.4 GHz spectrum toward sub-GHz frequencies has largely been driven by the need for extended communication range and improved energy efficiency [26]. Although the study primarily focuses on 868 MHz and 900 MHz technologies, the authors indicate that the same communication principles apply to other sub-GHz frequencies, including 433 MHz [26].

A defining characteristic of LPWAN systems is the trade-off between communication range and throughput. De Poorter et al. explain that LPWAN technologies achieve long-range communication by operating at relatively low data rates while optimising power efficiency [26]. Technologies such as LoRaWAN further employ adaptive data rate (ADR) mechanisms to dynamically balance communication range and throughput according to prevailing operating conditions [26].

Bandwidth limitations remain a fundamental characteristic of LPWAN systems. According to De Poorter et al., these technologies are generally designed for small information exchanges involving payloads often smaller than 100 bytes [26]. Consequently, telemetry systems must carefully prioritise the information transmitted during flight operations.

Interference and coexistence challenges also influence LPWAN performance. De Poorter et al. note that limited spectrum availability and increasing adoption of LPWAN technologies can contribute to congestion and communication degradation within unlicensed sub-GHz frequency bands [26]. These considerations necessitate careful communication system design to maintain reliability under congested operating conditions.

The practical advantages of LPWAN communication are validated by Krueger and Bergmaier, who demonstrate that LoRaWAN provides significantly greater communication range than conventional wireless technologies while maintaining low power requirements [16]. Their flight experiments further confirm successful telemetry transmission from altitudes exceeding 2.6×10^1 km using sub-GHz communication technology [16].

The literature demonstrates that UHF and sub-GHz communication technologies provide an effective balance between communication range, energy efficiency, and system complexity. Although bandwidth limitations and interference remain important considerations, LPWAN technologies provide a suitable foundation for long-range telemetry communication in high-altitude balloon applications.

3.2.3 GPS-based tracking systems

Accurate position determination is essential during the flight of a high-altitude balloon. Position information supports flight monitoring, enables trajectory analysis, assists operational decision-making, and facilitates payload recovery following landing [6, 14]. As a result, GPS and other global navigation satellite system (GNSS) technologies have become central components of modern balloon tracking systems [6, 37] and require thorough research for successful tracking and monitoring of a stratospheric balloon system.

Enge describes GPS as a satellite-based positioning technology that determines receiver location through timing measurements obtained from multiple satellites [6]. GPS has become the dominant positioning technology because it offers global coverage, relatively high accuracy, and widespread availability [6].

Wing evaluated the performance of consumer-grade GPS receivers and identified several factors affecting positioning accuracy, including receiver quality, antenna design, satellite visibility, and environmental conditions [36]. These considerations remain relevant when selecting GPS hardware for high-altitude balloon applications.

Wiyagi et al. incorporated a Ublox NEO-M8N GNSS receiver into a high-altitude balloon payload and highlighted its suitability for operation at altitudes approaching 5×10^1 km [37]. The study demonstrates the practical feasibility of integrating GNSS technologies into lightweight atmospheric research platforms.

Recent developments have increasingly integrated GNSS positioning with LPWAN communication technologies. Janssen et al. describe architectures in which GNSS observations are transmitted through LPWAN networks to cloud-based processing platforms for position determination [14]. Such approaches reduce computational requirements on resource-constrained devices while improving energy efficiency.

Janssen et al. further discuss integrated GNSS and LPWAN systems-on-chip that combine positioning and communication capabilities within a single device [14]. These developments reflect a growing convergence between communication and positioning technologies in modern IoT systems.

The survey also highlights positioning architectures in which communication networks serve as transport mechanisms for GNSS observations and assistance data [14]. Consequently, positioning performance increasingly depends not only on GPS receiver capabilities but also on the reliability of supporting communication systems.

The reviewed literature confirms that GPS remains the most suitable positioning technology for high-altitude balloon applications. However, effective tracking increasingly depends on the integration of position tracking and communication technologies, highlighting the importance of reliable telemetry systems for successful tracking performance.

3.2.4 Additional considerations for high-altitude tracking and communication systems

While telemetry communication and GPS-based tracking form the core components of the proposed system, several additional factors influence the overall effectiveness, reliability, and operational success of high-altitude balloon missions. Previous research has identified challenges related to payload recovery, communication coverage, system integration, and technology selection, all of which affect the design of communication and tracking solutions [3, 4, 37]. The following literature therefore examines complementary considerations that provide further context for the development of a robust and practical stratospheric balloon subsystem.

Hybrid tracking and recovery systems

The recovery of high-altitude balloon payloads remains a critical operational objective because payloads often contain valuable scientific instruments, experimental hardware, and recorded mission data [3, 8]. Consequently, researchers have increasingly investigated hybrid tracking approaches that improve reliability through redundancy.

Brown highlights the importance of communication redundancy through the implementation of multi-mode telemetry systems [3]. By employing multiple communication channels, such systems reduce the likelihood of total communication failure and improve payload recovery success rates.

Janssen et al. discuss hybrid architectures that combine GNSS positioning with LPWAN, cellular, and low earth orbit (LEO) satellite communication technologies [14]. These approaches improve coverage and tracking reliability, particularly in remote environments where individual communication technologies may experience limitations.

The authors further describe multi-radio access technology architectures that dynamically select communication mechanisms according to environmental conditions, energy constraints, and coverage availability [14]. Such adaptive systems improve operational resilience while maintaining efficient resource utilisation.

The reviewed studies demonstrate that existing recovery-oriented systems frequently rely on multiple communication technologies to improve tracking reliability. These approaches provide useful benchmarks for evaluating the proposed system, particularly with respect to communication redundancy, tracking continuity, and payload recovery effectiveness.

Limitations of cellular communication for high-altitude platforms

Although cellular communication networks provide extensive terrestrial coverage, their suitability for high-altitude applications remains limited. Cellular infrastructure is primarily designed to support ground-based users and therefore does not adequately account for aerial operating environments.

Colpaert et al. demonstrate that aerial platforms experience significantly different radio propagation conditions compared to terrestrial users [4]. At increasing altitudes, aerial platforms become exposed to signals from multiple base stations simultaneously, resulting in substantial inter-cell interference.

Simulation results presented by Colpaert et al. indicate that signal quality decreases significantly with altitude because increasing numbers of base stations maintain line-of-sight communication paths to the aerial platform [4]. This creates interference conditions that degrade communication reliability.

Coverage analyses further demonstrate that communication performance deteriorates rapidly with increasing altitude [4]. The authors conclude that conventional LTE and mmWave systems cannot reliably support aerial users at elevated operating heights without substantial modifications.

These findings are particularly relevant for stratospheric balloon systems because operational altitudes significantly exceed those considered during conventional cellular network planning [4].

Integrated communication and tracking architectures

Communication and tracking systems are inherently multidisciplinary because they require the integration of sensing, processing, positioning, power management, and communication subsystems [37]. Consequently, system architecture plays an important role in determining overall system performance and reliability [37].

Wiyagi et al. present a comprehensive balloon payload architecture comprising of sensors, microcontrollers, a Raspberry Pi processing platform, communication modules, and GPS positioning systems [37]. Figure 3.1 illustrates how multiple subsystems can be integrated to support mission objectives while operating within strict payload constraints.

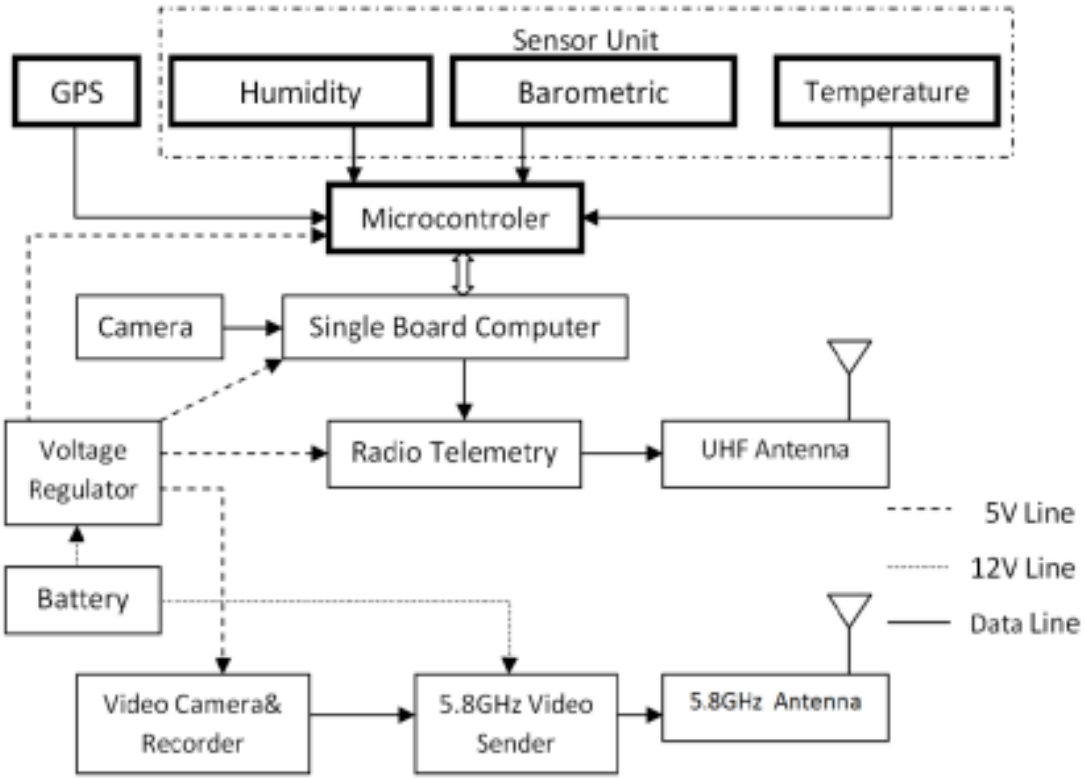


Figure 3.1: High-altitude balloon payload architecture proposed by Wiyagi et al. [37].

The use of Raspberry Pi platforms for data processing and communication management further demonstrates the suitability of modular architectures for high-altitude systems [37]. Modular architectures facilitate subsystem integration, improve maintainability, and support future system expansion.

Comparable architectural principles are evident in IoT-based monitoring systems. Uzair et al. describe an integrated architecture in which Arduino-based sensing platforms communicate with Raspberry Pi processing systems through layered communication structures [34]. The separation of sensing, processing, communication, and application functions provides a useful reference for communication and tracking system design.

Several studies have implemented integrated high-altitude balloon payloads that combine communication, positioning, processing, and sensing functions within a single system. Examples include the architectures proposed by Wiyagi et al. [37] and Wang et al. [35], both of which demonstrate the practical integration of telemetry and tracking subsystems in high-altitude environments. These studies provide reference systems against which the proposed communication and tracking solution can be compared.

3.3 How the literature guides this study

The literature reviewed in this chapter provides the theoretical and technical foundation upon which the proposed communication and tracking system is developed. Beyond establishing the current state of research, the reviewed studies identify key design requirements, technological constraints, and performance considerations that directly influence the development of the proposed subsystem.

The reviewed research supports the selection of a sub-GHz communication approach for telemetry transmission. Studies demonstrate that LPWAN technologies offer a practical balance between communication range, power consumption, and system complexity, making them suitable for high-altitude balloon applications [16, 26]. At the same time, the identified limitations relating to bandwidth availability, interference, and spectrum utilisation highlight important constraints that must be considered during system design and implementation.

The literature also informs the selection of GPS as the primary tracking mechanism. Previous studies demonstrate the suitability of GNSS technologies for high-altitude applications and emphasise the growing integration between communication and positioning systems [6, 14]. These findings support the development of a tracking solution in which positional information forms an integral component of the telemetry data transmitted to ground operators.

Research relating to system architecture further guides the organisation of the proposed subsystem [34, 35, 37]. Existing payload designs demonstrate the value of integrating sensing, processing, positioning, communication, and power-management functions into a modular architecture [34, 35, 37]. These architectural principles influence the selection and interaction of the hardware and software components implemented in this study.

In addition to informing design decisions, the reviewed literature establishes a basis for evaluating the developed artefact. Performance measures reported in previous studies, including communication reliability, packet delivery performance, operational range, and tracking effectiveness, provide reference points against which the proposed system can be assessed [3, 14, 16]. The findings presented in subsequent chapters can therefore be compared with those reported in existing research to determine whether the developed solution achieves its intended objectives.

The literature review therefore serves as both a knowledge base and a design guide for the study. It provides insight into the selection of technologies, identifies relevant design considerations, and establishes evaluation criteria that support the systematic development and assessment of the proposed communication and tracking system.

3.4 Conclusion

This chapter reviewed literature relevant to telemetry systems, UHF-based communication technologies, GPS-based tracking systems, and additional considerations affecting communication and tracking performance in high-altitude balloon applications. The reviewed literature established the theoretical and technological foundation necessary for the development of a communication and tracking system capable of operating under stratospheric conditions.

The literature demonstrates that successful high-altitude balloon missions require reliable long-range communication, accurate positioning, efficient energy utilisation, and robust payload recovery mechanisms [3, 27, 37]. Existing studies support the use of sub-GHz communication technologies and GPS-based tracking systems while highlighting the limitations of cellular communication at high altitudes [4, 6, 16, 26]. The reviewed literature further emphasises the value of integrated system architectures and communication redundancy in improving overall mission reliability.

The next chapter presents the empirical design and development process used to design and develop the proposed communication and tracking system. It outlines the system artefact, describes the iterative development cycles, and explains the evaluation procedures used to assess performance and reliability.

Chapter 4

Empirical study

4.1 Introduction

In the mini-dissertation, the development of a reliable and power-efficient communication and tracking system for a stratospheric balloon operating at high altitudes is investigated. The context and objectives of this study were introduced in Chapter 1, Chapter 2 established the pragmatic paradigm, Design Science Research (DSR) methodology, and Chapter 3 reviewed the literature relevant to telemetry systems, UHF-based communication, and GPS-based tracking. This chapter addresses the design-and-development, empirical objective of the study: to design, implement, and evaluate the proposed communication and tracking artefact through the DSR process.

The empirical study chapter discusses the empirical design and development of the proposed communication and tracking system. It describes the artefact that was produced, explains how the DSR process phases (problem identification, objective definition, design and development, demonstration, and evaluation) [25] were applied and iterated in practice, and reports the findings obtained from testing the proposed system.

The remainder of this chapter is structured as follows. The artefact description describes the communication and tracking artefact, including its purpose, scope, and key requirements, for both the airborne subsystem and the ground station. The description of the development process describes the development process followed, structured according to the DSR phases, and discusses the iterative cycles undertaken. The findings section presents the findings obtained from GPS-based testing and communication-link testing. Finally, this chapter concludes the empirical study of this mini-dissertation and prepares the reader for the conlusion chapter.

4.2 Artefact description

The artefact produced in this study is a communication and tracking system for a stratospheric balloon, developed as one of three interdependent subsystems within a broader group project. The purpose of the artefact is to provide reliable, low-power position tracking and telemetry transmission between a stratospheric balloon and a ground station, operating within the power, bandwidth, and environmental constraints of stratospheric flight [11, 12, 27]. Importantly, the artefact is not limited to the airborne payload; it also includes the design, implementation, and testing of the ground station that receives, logs, and displays the transmitted telemetry, since reliable end-to-end communication depends equally on both ends of the link.

4.2.1 Purpose and scope

The development of the artefact is motivated by the problem identified in Chapter 1: conventional terrestrial infrastructure, such as cellular networks, is unsuitable for high-altitude communication because it is designed for ground-based coverage and suffers from severe inter-cell interference and signal degradation at altitude [4]. UHF communication in the 433 MHz ISM band [12, 13], by contrast, offers long-range, low-power communication that tolerates the line-of-sight and propagation conditions typical of stratospheric flight [17, 26]. The artefact therefore combines a UHF telemetry link with GPS-based positioning to meet the aims defined in Section 1.3.3, and pairs the airborne payload with a dedicated ground station capable of reliably receiving and recording this data throughout the flight.

The scope of the artefact covers both segments of the communication and tracking link. On the airborne side, it is responsible for acquiring GPS position data, receiving processed data from the control and sensor subsystems developed by the other two collaborators in the group project, packaging this data into a telemetry format, and transmitting it over a UHF link. On the ground side, the scope includes the design, assembly, and testing of a ground station capable of receiving the UHF telemetry signal, decoding the transmitted packets, and logging the resulting position data for later analysis against the evaluation metrics defined in Section 1.4.4. Onboard control logic, payload release, geofencing, and atmospheric sensing fall outside of the scope for this artefact and are addressed by the other subsystems (Section 1.4.6).

4.2.2 System architecture

The artefact consists of an airborne subsystem and a ground station subsystem. The following components were selected for their low power consumption, reliability, and suitability for long-range telemetry, consistent with prior stratospheric balloon and LPWAN research [26, 34, 37, 35]:

- A **GPS module** (Uputronics) for acquiring real-time positional data (latitude, longitude, altitude) [6].
- A **Raspberry Pi Zero** single-board computer, responsible for reading GPS data, formatting telemetry packets, and interfacing with the UHF transmitter.

- A **Radiometrix NTX2-434.650-10-HP UHF transmitter**, mounted on the balloon payload and operating at 434.650 MHz within the 433 MHz ISM band, providing the radio-frequency communication link to the ground station [12, 28].
- A battery-based power supply for the airborne subsystem [19].
- A **Radiometrix NRX2 UHF receiver**, mounted at the ground station and interfaced to an **Arduino Uno R3** microcontroller, which decodes the receiver's raw baseband output into telemetry packets and measures the received signal strength (RSSI) [1, 28].
- A **ground-station computer**, connected to the Arduino Uno R3 over USB serial, which logs the decoded telemetry to a file for later analysis.

Together, these components form the two ends of the communication and tracking link: the airborne payload (Uputronics GPS, Raspberry Pi Zero and NTX2 transmitter), which acquires and transmits telemetry, and the ground station (NRX2 receiver, Arduino Uno R3 and logging computer), which receives, decodes, and logs it for monitoring and recovery purposes [27].

4.2.3 System requirements

Based on the aims and objectives defined in Section 1.3.3, the following key requirements guided the design of the artefact:

- The system must be able to acquire and transmit GPS location data to a ground station at regular intervals,
- Operate within the power constraints of a battery-powered, high-altitude platform [19],
- Provide a communication range sufficient for stratospheric operating altitudes [22, 31],
- Be capable of reliably receiving, decoding, and logging telemetry data transmitted from the balloon throughout the flight,
- Support redundancy mechanisms to improve the likelihood of payload recovery in the event of communication failure (Section 1.5.3) [3] and,
- Minimise transmitted information to reduce the risk of unintended information leakage [18], as discussed in Section 1.5.1.

These requirements collectively guided the design and testing decisions described in the remainder of this chapter, for both the airborne payload and the ground station.

4.3 Description of the development process

This section describes how the DSR process phases were applied in practice during the development of the communication and tracking artefact, covering both the airborne payload and the ground station [2, 25]. Development was iterative: each cycle below concludes with a reflection describing what was learned and how it informed the next cycle, consistent with the rigour and validity procedures described in Section 2.3.3.

4.3.1 Problem identification and objective definition

The communication and tracking subsystem was developed to address the need for reliable position tracking and telemetry communication for the stratospheric balloon. The system must be capable of obtaining the balloon's geographic position and making this information available for transmission to the ground station while operating within the constraints of a high-altitude payload. The development therefore focused on establishing a functional interface between the GPS module and the onboard Raspberry Pi Zero, integrating the acquired positional information into the telemetry system, and ultimately transmitting the resulting data to the ground station through the UHF communication link.

The development objectives were translated into a series of practical implementation requirements. The first requirement was to establish reliable communication between the GPS module and Raspberry Pi Zero and obtain valid positional information. Once this interface was established, the GPS data needed to be made available to other components of the system, including the geofencing functionality and the telemetry transmission process. The UHF transmitter and ground station then needed to be integrated to provide an end-to-end communication link between the airborne and ground-based components.

4.3.2 Design and development cycle 1

The first development cycle focused on establishing communication between the Uputronics GPS module and the Raspberry Pi Zero. This was selected as the initial development activity because GPS position information forms the basis of the tracking functionality [6, 14] and is required for both the geofencing functionality and telemetry transmission. The objective of this cycle was therefore to establish a functional interface through which the Raspberry Pi Zero could obtain positional information from the GPS module.

A challenge during the initial implementation was determining the required wiring and interface configuration for the GPS module. Documentation for the specific GPS module was not readily available, making it necessary to investigate the available connections and determine which pins were required for communication with the Raspberry Pi Zero. Via trial and error and moderate experience with Raspberry Pi devices, the required connections were identified and configured to establish communication between the GPS module and the Raspberry Pi Zero as shown in Figure-4.1.

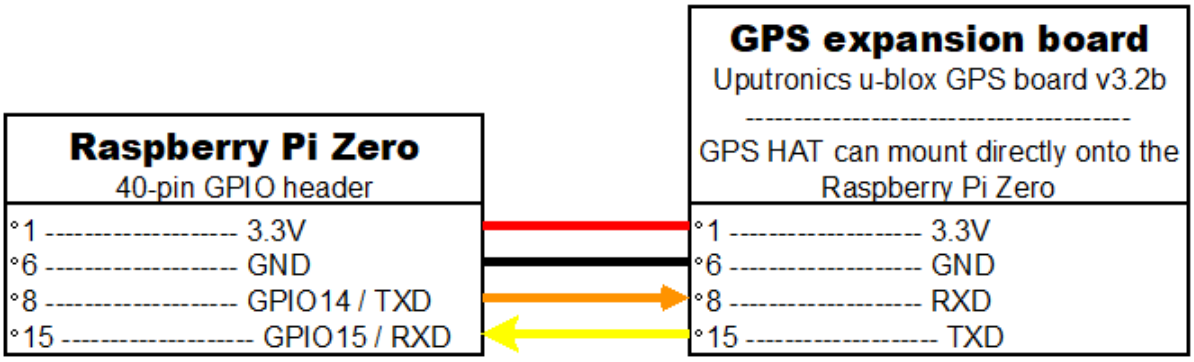


Figure 4.1: Verified UART wiring configuration between the Raspberry Pi Zero and Uputronics GPS expansion board.

After the wiring configuration was determined, a program was developed to communicate with the GPS module and retrieve its positional information. The program implemented for the GPS module to be able to interface with the Raspberry Pi Zero is provided in Appendix **??**; Listing 4.1 below shows the validation function used to determine whether a reported GPS fix is complete enough to be logged or used by the rest of the system.

```
31 def is_complete(report):
32     mode = getattr(report, "mode", 0)
33 
34     if (mode < MIN_MODE):
35         return False, f"mode={mode} (need >= {MIN_MODE})"
36 
37     missing = [
38         field for field in REQUIRED_FIELDS
39         if getattr(report, field, None) is None
40     ]
41 
42     if (missing):
43         return False, f"missing: {', '.join(missing)}"
44     return True, None
```

Listing 4.1: Validity check applied to each GPS fix before it is logged or made available to other components (`gps_execute.py`).

The program executed successfully, processed the GPS data and displayed the results in the Raspberry Pi Zero terminal. As shown in Figure 4.2, the program continuously reported valid GPS fixes, including latitude, longitude, altitude, speed, track, and the number of satellites used. This demonstrates that the Raspberry Pi Zero can successfully communicate with the GPS module and acquire positional data for further processing.

```
strato@StratoBalloon:~/GPS/GroundTest $ sudo python3 gps_execute.py
GPS monitoring started.
Logging to: /home/strato/GPS/GroundTest/logs/gps_log_2026-09-08_22-05-02.csv
Latest GPS data: /home/strato/GPS/GroundTest/latest_gps.json
Valid GPS fixes are logged every 5 seconds.
Press Ctrl+C to stop.

SYSTEM TIME: 2026-09-08T22:05:03.158387+00:00  |  SATELLITE TIME: 2026-09-08T22:05:03.000Z
VALID FIX LOGGED  lat=-26.688736  lon=27.095180  alt=1322.971m  speed=0.061m/s  track=166.6631deg  sats_used=0
SYSTEM TIME: 2026-09-08T22:05:09.159207+00:00  |  SATELLITE TIME: 2026-09-08T22:05:09.000Z
VALID FIX LOGGED  lat=-26.688705  lon=27.095179  alt=1322.144m  speed=0.149m/s  track=166.6631deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:05:14.157966+00:00  |  SATELLITE TIME: 2026-09-08T22:05:14.000Z
VALID FIX LOGGED  lat=-26.688699  lon=27.095177  alt=1319.697m  speed=0.028m/s  track=166.6631deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:05:19.623612+00:00  |  SATELLITE TIME: 2026-09-08T22:05:19.000Z
VALID FIX LOGGED  lat=-26.688678  lon=27.095158  alt=1316.164m  speed=0.217m/s  track=166.6631deg  sats_used=7
SYSTEM TIME: 2026-09-08T22:05:25.159361+00:00  |  SATELLITE TIME: 2026-09-08T22:05:25.000Z
VALID FIX LOGGED  lat=-26.688675  lon=27.095138  alt=1310.473m  speed=0.207m/s  track=166.6631deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:05:30.165357+00:00  |  SATELLITE TIME: 2026-09-08T22:05:30.000Z
VALID FIX LOGGED  lat=-26.688646  lon=27.095130  alt=1310.008m  speed=0.399m/s  track=166.6631deg  sats_used=5
SYSTEM TIME: 2026-09-08T22:05:35.622294+00:00  |  SATELLITE TIME: 2026-09-08T22:05:35.000Z
VALID FIX LOGGED  lat=-26.688657  lon=27.095135  alt=1309.486m  speed=0.306m/s  track=166.6631deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:05:41.160143+00:00  |  SATELLITE TIME: 2026-09-08T22:05:41.000Z
VALID FIX LOGGED  lat=-26.688679  lon=27.095149  alt=1310.31m  speed=0.078m/s  track=166.6631deg  sats_used=7
SYSTEM TIME: 2026-09-08T22:05:47.157988+00:00  |  SATELLITE TIME: 2026-09-08T22:05:47.000Z
VALID FIX LOGGED  lat=-26.688759  lon=27.095155  alt=1307.48m  speed=0.579m/s  track=166.6631deg  sats_used=5
SYSTEM TIME: 2026-09-08T22:05:52.161992+00:00  |  SATELLITE TIME: 2026-09-08T22:05:52.000Z
VALID FIX LOGGED  lat=-26.688832  lon=27.095170  alt=1306.375m  speed=0.876m/s  track=166.7016deg  sats_used=7
SYSTEM TIME: 2026-09-08T22:05:57.620368+00:00  |  SATELLITE TIME: 2026-09-08T22:05:57.000Z
VALID FIX LOGGED  lat=-26.688826  lon=27.095161  alt=1305.13m  speed=0.481m/s  track=171.4333deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:06:03.159622+00:00  |  SATELLITE TIME: 2026-09-08T22:06:03.000Z
VALID FIX LOGGED  lat=-26.688854  lon=27.095163  alt=1303.98m  speed=0.507m/s  track=171.4333deg  sats_used=6
SYSTEM TIME: 2026-09-08T22:06:09.162545+00:00  |  SATELLITE TIME: 2026-09-08T22:06:09.000Z
VALID FIX LOGGED  lat=-26.688844  lon=27.095164  alt=1306.038m  speed=0.347m/s  track=171.4333deg  sats_used=7
SYSTEM TIME: 2026-09-08T22:06:15.158881+00:00  |  SATELLITE TIME: 2026-09-08T22:06:15.000Z
VALID FIX LOGGED  lat=-26.688849  lon=27.095185  alt=1310.426m  speed=0.396m/s  track=171.4333deg  sats_used=8
SYSTEM TIME: 2026-09-08T22:06:20.159128+00:00  |  SATELLITE TIME: 2026-09-08T22:06:20.000Z
VALID FIX LOGGED  lat=-26.688839  lon=27.095197  alt=1313.702m  speed=0.319m/s  track=171.4333deg  sats_used=8
SYSTEM TIME: 2026-09-08T22:06:25.621640+00:00  |  SATELLITE TIME: 2026-09-08T22:06:25.000Z
VALID FIX LOGGED  lat=-26.688851  lon=27.095214  alt=1316.588m  speed=0.439m/s  track=171.4333deg  sats_used=6
```

Figure 4.2: Example of successful GPS data acquisition, showing consecutive valid GPS fixes containing latitude, longitude, altitude, speed, track, and the number of satellites used.

In addition to recording the GPS observations, the latest valid GPS fix is made available to the entire system by storing the latest GPS data in a JSON file within the same directory as the Python program that interfaces with the GPS module. This allows other components to access the most recent positional information without requiring a separate direct connection to the GPS module. Thus, the developed Python program establishes the positional data source required for subsequent integration with the geofencing and telemetry transmission components. Additionally, the Python GPS program logs each positional fix at regular intervals to allow for quantitative analysis in Section 4.4.1.

Reflection on this cycle indicates that establishing and verifying the GPS interface before integrating additional components reduces the complexity of further development. The verified GPS connection provides a known working baseline for the next development cycle, where the payload transmitter will transmit GPS data to the ground station receiver. The wiring configuration and successful GPS output were also documented to provide a reproducible configuration for development and testing.

4.3.3 Design and development cycle 2

Building on the verified GPS interface in Cycle 1, this cycle focused on integrating the UHF transmission component of the communication link. Starting with the physical wiring of the Radiometrix NTX2 UHF transmitter to the Raspberry Pi Zero. Establishing the transmitter interface independently of the GPS subsystem allows for the transmission subsystem to be configured and verified before integrating it with the GPS data source.

The NTX2 transmitter is wired following a UKHAS community wiring diagram for interfacing the NTX2 with a microcontroller [32], together with the manufacturer's datasheet [28], as shown in Figure 4.3.

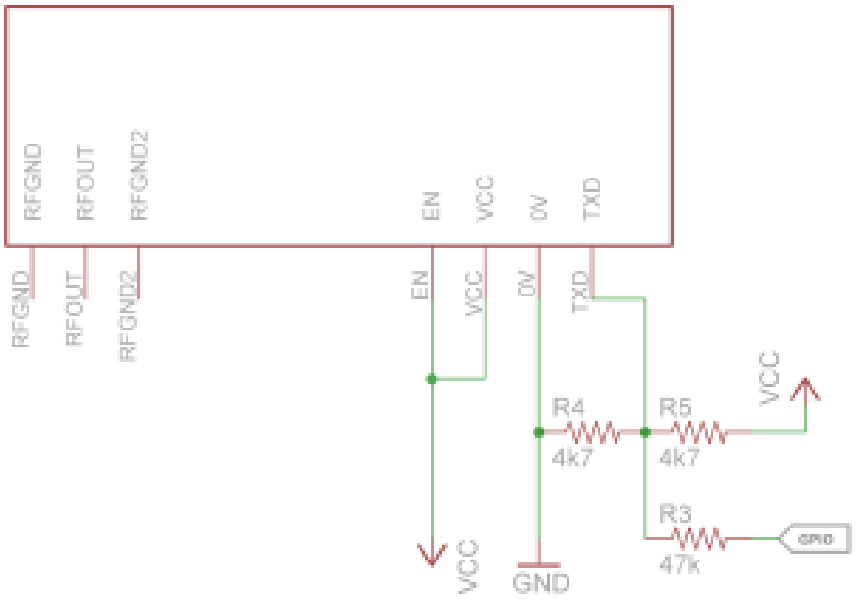


Figure 4.3: Wiring configuration between the Raspberry Pi Zero and the Radiometrix NTX2 UHF transmitter, based on [32].

The series resistor, R3, used on the TXD input was evaluated as part of the development of the NTX2 transmitter interface. The initial resistor configuration was based on the UKHAS guide for interfacing the NTX2 with a microcontroller, which specifies a 47 kΩ resistor in its example circuit [32]. The manufacturer's datasheet states that the TXD input is designed for a 3 V logic level and that a series resistor should be added when the drive level is higher, giving typical values of 10 kΩ for a 3.3 V source and 68 kΩ for a 5 V source [28]. The 47 kΩ value from the UKHAS guide therefore corresponds to a 5 V microcontroller, whereas the Raspberry Pi Zero GPIO output operates at 3.3 V.

During development, the R3 value was adjusted experimentally to determine a suitable interface configuration between the Raspberry Pi Zero and the NTX2 transmitter. Using the 100 kΩ input resistance of the TXD pin [28], the voltage at TXD for a 3.3 V output is approximately 3.0 V for the datasheet value of 10 kΩ, 3.27 V for 1 kΩ, and 2.2 V for 47 kΩ. The 1 kΩ resistor was selected for the final implementation based on the observed performance of the complete transmitter and receiver system. A 470 Ω configuration, which gives almost the same TXD voltage, produced poorer communication performance. The 1 kΩ value was therefore kept as an empirically selected design value, which is slightly above the 3 V level specified by the datasheet,

rather than a value specified by the manufacturer. This iterative adjustment formed part of the development process used to establish a configuration that provided suitable signal behaviour for subsequent testing of the NTX2-to-NRX2 communication link. This produced the completed transmitter artefact, as shown in Figure 4.4.

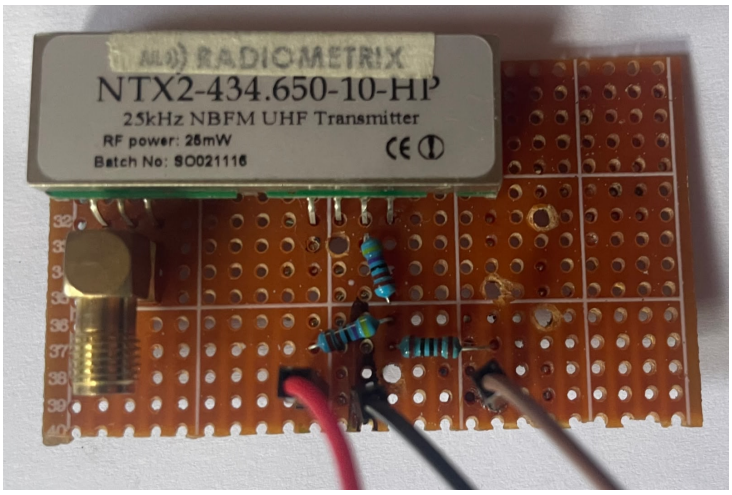


Figure 4.4: Completed Radiometrix NTX2 UHF transmitter artefact.

After establishing the wiring for the NTX2 transmitter, the 434.650 MHz antenna required for transmission was also constructed. A quarter-wave ground-plane antenna was built, consisting of a central radiating element connected through a 50 Ω coaxial cable [28], together with four ground-plane radial wires arranged radially and cut to 16.4 cm according to the practical antenna dimensions specified by the UKHAS community guide [20, 33].The constructed antenna is shown in Figure 4.5.

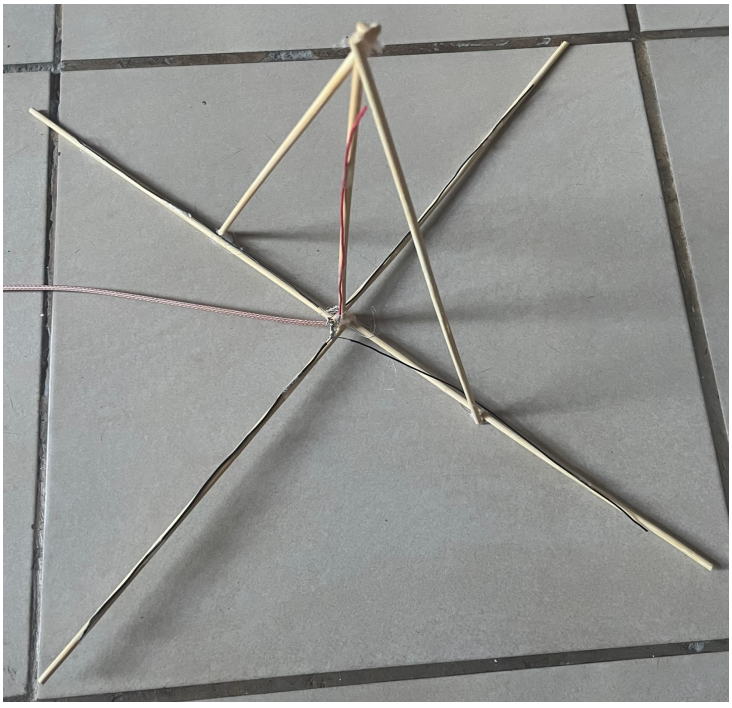


Figure 4.5: Quarter-wave ground-plane antenna constructed for the airborne transmitter, showing the central radiating element and four 16.4 cm ground-plane radials.

With the transmitter wired and the antenna constructed, a Python program (Appendix E.3) was developed to transmit the GPS position over the Radiometrix transmitter. For every transmission interval (60 s by default), the program reads the latest valid fix from the JSON file produced by the GPS program in Cycle 1, converts it to a compact payload, and transmits it by directly toggling a Raspberry Pi GPIO pin connected to the transmitter's TXD input. This design keeps the transmission program independent of the GPS hardware: it only depends on the most recent valid fix being available in `latest_gps.json`.

To minimise the amount of transmitted information [18] (Section 4.2.3), the payload contains only a transmission counter, latitude and longitude (five decimal places), altitude (whole metres) and the time of the fix (HHMMSS), as comma-separated values without labels. An example payload is shown below:

```
1,-26.68863,27.09528,1344,022306
```

The length of the payload is not fixed, because the transmission counter is sent as text and therefore grows by one byte each time it reaches a new power of ten. For the operating area, the remaining fields (latitude, longitude, altitude, time of fix and the separators) occupy approximately 31 to 32 bytes, so the example above is 32 bytes with a one-digit counter, and the payload grows to 33 bytes at the tenth reading, 34 bytes at the hundredth and 35 bytes at the thousandth. At the default interval of 60 s, the counter reaches three digits after approximately 100 minutes and four digits after approximately 17 hours, so a flight of a few hours results in payloads of up to approximately 35 bytes. Each additional byte increases the on-air time of every copy by 32 ms.

The payload is limited to 40 bytes by the frame length field and the receiver's payload buffer. The transmission program enforces this limit in two steps. If a payload exceeds 40 bytes, the latitude and longitude are reduced to four decimal places, and if the payload is still too long, the frame is rejected and that reading is skipped with a warning. With the fields used here, this would only occur once the counter reaches nine digits, which at one reading per minute would take more than a century, so the limit is not a practical constraint for the intended flight duration. The transmission counter acts as an identifier for each reading and is later used by the ground station to recognise that several received frames are copies of the same transmission.

Because the NTX2 is a narrow-band FM transmitter without any built-in framing or error correction, the program implements its own frame structure, shown in Listing 4.2. Each frame consists of a preamble of six `0xAA` bytes, a two-byte synchronisation word (`0x2DD4`), a length byte, the payload, and an 8-bit CRC. The frame is Manchester encoded at a bit period of 4 ms. Manchester coding has a transition in every bit period and no DC component, which suits the NTX2/NRX2 pair because the datasheet advises against data containing a DC component, since frequency drift between the transmitter and receiver causes DC offset errors on the receiver output, and notes that the RXD output is suitable for bi-phase codes [28]. The regular transitions also allow the receiver to recover the bit timing from the preamble.

```
82  def build_frame(payload):
83      if len(payload) == 0 or len(payload) > MAX_PAYLOAD_BYTES:
84          raise ValueError(f"payload must be 1-{MAX_PAYLOAD_BYTES} bytes, got {len
                (payload)}")
85      body = bytes([len(payload)]) + payload
86      return PREAMBLE + SYNC + body + bytes([crc8(body)])
87
88  def to_half_bits(frame):
89      # Manchester (IEEE 802.3): bit 1 = LOW then HIGH, bit 0 = HIGH then LOW.
90      halves = []
91      for byte in frame:
92          for i in range(7, -1, -1):
93              halves.extend([0, 1] if ((byte >> i) & 1) else [1, 0])
94      return halves
95
96  def send_burst(pin, payload, repeats, gap_seconds, bit_us):
97      # Send `repeats` copies of the same frame, each separated by gap_seconds of
            idle.
98      half_seconds = bit_us / 2 / 1_000_000
99      one_frame_halves = to_half_bits(build_frame(payload))
100     per_copy_seconds = len(one_frame_halves) * half_seconds
101
102     for i in range(repeats):
103         play(pin, one_frame_halves, half_seconds, GPIO.LOW)
104         if i < repeats - 1 and gap_seconds > 0:
105             time.sleep(gap_seconds)
106
107     total_seconds = repeats * per_copy_seconds + max(0, repeats - 1) *
            gap_seconds
108     return total_seconds, per_copy_seconds
```

Listing 4.2: Frame construction, Manchester encoding and burst transmission of a GPS payload (`gps_transmit.py`).

The transmitter does not send each reading only once. Instead, as shown in the `send_burst` function in Listing 4.2, six identical copies of the frame are sent back-to-back for every reading. This was done because a single frame is only successfully decoded if the entire frame (preamble, sync word, length, payload and CRC) is received without a single bit error. A corrupted frame fails its CRC check and is discarded, so one interference event, fade or timing glitch during a single transmission would result in the loss of that position update. Repeating the frame increases the probability that at least one copy of every reading is received intact [3, 16], which supports the redundancy requirement for payload recovery (Section 4.2.3) at the cost of a longer on-air time per reading.

Figure 4.6 shows the terminal output of the transmission program while running. For every cycle, the program prints the transmitted payload text, its length in bytes, the number of copies, and the total and per-copy on-air time.

```
strato@StratoBalloon:~/Honours_Telemetry_System/programs/payload/stationary $ sudo python3 gps_transmit.py
2026-10-08 15:18:41  INFO    NTX2 GPS beacon on GPIO17. Ctrl+C to stop.
2026-10-08 15:18:41  INFO    Reading: latest_gps.json
2026-10-08 15:18:41  INFO    Logging transmissions to: logs/gps_transmit.log
2026-10-08 15:18:41  INFO    Sending 6 copies per reading, 0 ms gap between copies, every 60s.
2026-10-08 15:18:49  INFO    sent: 0,-26.68875,27.09527,1336,131840  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:19:49  INFO    sent: 1,-26.68874,27.09529,1334,131940  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:20:49  INFO    sent: 2,-26.68875,27.09530,1325,132040  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:21:49  INFO    sent: 3,-26.68874,27.09529,1328,132140  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:22:49  INFO    sent: 4,-26.68874,27.09527,1332,132240  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:23:49  INFO    sent: 5,-26.68874,27.09526,1337,132340  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:24:49  INFO    sent: 6,-26.68871,27.09526,1338,132440  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:25:49  INFO    sent: 7,-26.68872,27.09526,1336,132540  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:26:49  INFO    sent: 8,-26.68870,27.09524,1329,132640  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:27:49  INFO    sent: 9,-26.68871,27.09526,1327,132740  (32 bytes)  x6 copies  (8.06 s total, 1.34 s/copy)
2026-10-08 15:28:49  INFO    sent: 10,-26.68871,27.09526,1328,132840  (33 bytes)  x6 copies  (8.26 s total, 1.38 s/copy)
```

Figure 4.6: Terminal output of the transmission program on the Raspberry Pi Zero, showing the payload transmitted for each reading and the number of copies sent.

Cycle 2 produced a working transmitter, in which the interface resistor and framing had to be determined empirically (Section 4.3.3). The main lesson was that a raw FM link requires explicit framing, error detection and redundancy to be useful as a telemetry channel. The remaining unknown at the end of this cycle was how well these frames can be received and decoded, which is built further on in the ground station development described below.

4.3.4 Ground station design and testing

An Arduino Uno R3 was selected as the microcontroller used to interface with the Radiometrix NRX2 receiver because its available digital I/O, serial communication capability, USB connectivity, and suitability for rapid prototyping were sufficient for the requirements of the ground station [1]. Like the NTX2 transmitter, the receiver produces a raw baseband output that requires a microcontroller to decode the received signal into usable telemetry data [28, 32]. The required connections between the Radiometrix receiver and the Arduino Uno R3 were identified, as shown in Figure 4.7.

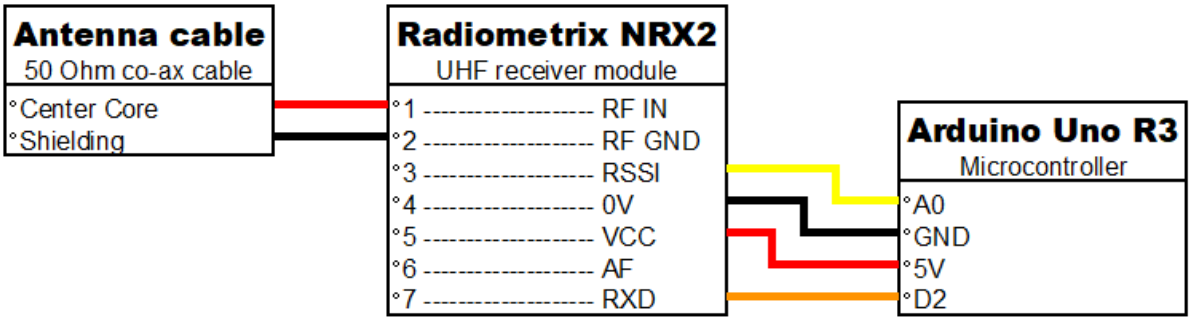


Figure 4.7: Wiring configuration between the Radiometrix NRX2 receiver and the Arduino Uno R3 at the ground station.

The antenna used at the ground station was constructed using the same quarter-wave ground-plane design described in Design and development cycle 2, consisting of a central radiating element connected via a 50 Ω coaxial cable and four 16.4 cm ground-plane radials [28, 33]. The constructed antenna is shown in Figure 4.8.

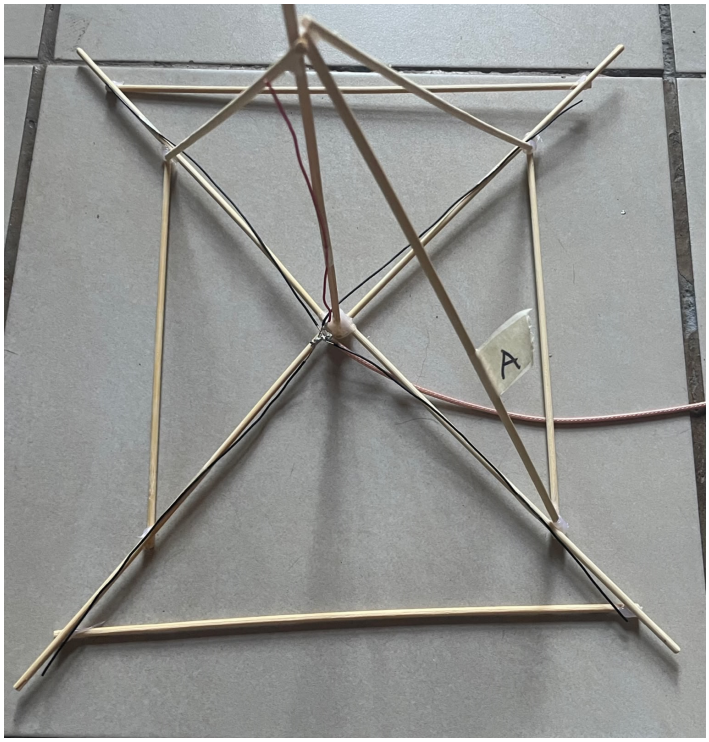


Figure 4.8: Quarter-wave ground-plane antenna used at the ground station.

The NRX2 provides both an analogue audio output (AF, pin 6) and a digital data output (RXD, pin 7). According to the datasheet, the AF pin is DC-coupled with a bias of approximately 0.8 V and carries an audio signal of about 500 mV peak-to-peak, while RXD is the squared version of this signal, provided by an internal data slicer [28]. The AF pin was initially considered as the signal source for the Arduino. However, when measured, it reported a voltage of approximately 4.4 V, which does not correspond to the documented bias level. Since this behaviour could not be explained or relied upon, the AF pin was not used.

Instead, the digital RXD output was connected directly to digital pin D2 of the Arduino Uno R3, and the RSSI output was connected to analogue pin A0, as shown in Figure 4.7. The RXD output is open-collector with an internal 10 kΩ pull-up to Vcc, so with the receiver powered from the 5 V supply of the Arduino, its logic levels are directly compatible with the Arduino input [1, 28], and no level shifting is required. The datasheet specifies the RSSI output as a DC level between 0.5 V and 2 V, which is within the range of the Arduino analogue input [1, 28].

The Arduino Uno R3 runs a sketch (Appendix E.4) that decodes the Manchester-encoded frames produced by the transmission program. Pin D2 triggers an interrupt on every signal edge, and the interrupt service routine only stores the time and level of each edge in a queue. All decoding is done in the main loop, so that no edges are missed while a frame is being processed.

The decoder (Listing 4.3) classifies the time between consecutive edges as either short (half a bit period) or long (a full bit period). Six consecutive long intervals, as produced by the preamble, are required before the decoder locks onto the bit timing. It then searches for the synchronisation word, reads the length byte and payload, and compares the received CRC with the CRC computed over the received bytes. A frame is only accepted as valid when both values match. The synchronisation word is also matched in inverted form, so the decoder works regardless of the polarity of the receiver output.

```
178 void handleEdge(unsigned long t, byte level) {
179   if (!haveLast) { lastEdgeUs = t; haveLast = true; return; }
180   unsigned long dt = t - lastEdgeUs;
181   lastEdgeUs = t;
182
183   byte cls = 0;                                    // 0 = invalid
184   if (dt >= BIT_US / 4 && dt < BIT_US * 3 / 4) cls = 1;          // short
185   else if (dt >= BIT_US * 3 / 4 && dt < BIT_US * 3 / 2) cls = 2; // long
186
187   if (cls == 0) { resetDecoder(); return; }
188
189   if (!locked) {
190     if (cls == 2) {
191       if (++longCount >= 6) {
192         locked = true;
193         atMid = true;
194         locks++;
195         onBit(level);
196       }
197     } else {
198       longCount = 0;
199     }
200     return;
201   }
202
203   if (atMid) {
204     if (cls == 1) {
205       atMid = false;
206     } else {
207       onBit(level);
208     }
209   } else {
210     if (cls == 1) {
211       atMid = true;
212       onBit(level);
213     } else {
214       resetDecoder();
215     }
216   }
217 }
```

Listing 4.3: Edge timing classification and bit-timing lock in the receiver sketch (`receiver_logger.ino`).

For every completed frame, valid or not, the sketch prints a human-readable line for the Serial Monitor, as shown in Figure 4.9, and a machine-readable `LOG` line containing the frame status (OK or BAD), the time, the RSSI in millivolts and the payload. The `LOG` lines are used by the logging program described in Section 4.4.2.

```
15:35:18.849 -> NRX2 MANCHESTER RECEIVER (with LOG, output)
15:35:22.393 -> [status] RSSI=1871 mV  edges=730  preamble locks=1  frames ok/bad=0/0
15:35:22.671 -> 2231 ms  OK  RSSI=1876 mV  ok/bad=1/0  | 0,-26.68867,27.09525,1346,133516
15:35:22.671 -> LOG,OK,2231,1876,0,-26.68867,27.09525,1346,133516
15:35:23.972 -> 3575 ms  OK  RSSI=1871 mV  ok/bad=2/0  | 0,-26.68867,27.09525,1346,133516
15:35:24.008 -> LOG,OK,3575,1871,0,-26.68867,27.09525,1346,133516
15:35:24.430 -> [status] RSSI=1871 mV  edges=1435  preamble locks=3  frames ok/bad=2/0
15:35:25.337 -> 4920 ms  OK  RSSI=1867 mV  ok/bad=3/0  | 0,-26.68867,27.09525,1346,133516
15:35:25.337 -> LOG,OK,4920,1867,0,-26.68867,27.09525,1346,133516
15:35:26.396 -> [status] RSSI=1871 mV  edges=2164  preamble locks=4  frames ok/bad=3/0
15:35:26.663 -> 6263 ms  OK  RSSI=1871 mV  ok/bad=4/0  | 0,-26.68867,27.09525,1346,133516
15:35:26.705 -> LOG,OK,6263,1871,0,-26.68867,27.09525,1346,133516
15:35:28.402 -> [status] RSSI=1867 mV  edges=2263  preamble locks=4  frames ok/bad=4/0
15:35:30.424 -> [status] RSSI=1862 mV  edges=2263  preamble locks=4  frames ok/bad=4/0
15:35:32.418 -> [status] RSSI=1867 mV  edges=2263  preamble locks=4  frames ok/bad=4/0
15:35:34.394 -> [status] RSSI=1867 mV  edges=2263  preamble locks=4  frames ok/bad=4/0
```

Figure 4.9: Arduino Serial Monitor output showing decoded frames, their RSSI, and the running count of valid and invalid frames.

The ground station demonstrated that frames transmitted by the Raspberry Pi Zero can be received and CRC-validated by the Arduino Uno R3. The unexpected AF voltage showed that the receiver's documented behaviour could not be assumed, and that outputs need to be measured before being used. Using the digital RXD output simplified the interface and kept the decoding in software, where the timing could be adjusted.

4.4 Findings

This section evaluates the performance of the developed communication and tracking artefact against the objectives and evaluation metrics defined for the study. The results are presented for the GPS tracking component and the end-to-end communication link between the airborne payload and ground station. GPS-based testing established the GPS receiver's positional consistency under stationary conditions, while the field results will evaluate the GPS tracking component's behaviour under flight conditions. The findings are used to determine whether the developed artefact provides the required functionality and performance for its intended stratospheric balloon application.

4.4.1 GPS-based testing results

GPS-based testing was conducted to evaluate the performance and consistency of the GPS component. The tests were performed at stationary locations to determine the extent to which the GPS module maintained a consistent reported position while connected to the Raspberry Pi Zero. The recorded GPS observations were analysed using the developed program included in Appendix A, which calculates positional deviations and changes between consecutive GPS observations from the recorded data.

The results obtained from the analysis are presented in Table 4.1 and calculated using the Python program in Appendix E.2. The table summarises the positional variation observed across 22 GPS-based test sessions. Mean and maximum positional deviation provide an indication of the overall variation in the reported position, while the mean consecutive position changes indicate the magnitude of movement between successive GPS observations. The number of position changes greater than 5m is included to identify larger deviations that occurred while the GPS receiver remained stationary.

Table 4.1: Quantitative summary of GPS-based GPS positional consistency.

Metric	**Result**
Number of test sessions	22
Total GPS readings	13356
Mean positional deviation (m)	5.623
Maximum positional deviation (m)	51.475
Mean consecutive position change (m)	1.618
Position changes >5 m	93

The values shown in Table 4.1 were obtained from the terminal output produced when the analysis program was executed. The program processed the recorded GPS observations from the 22 GPS-based test sessions and calculated the positional consistency metrics presented in the table. The terminal output provided a direct record of the calculated results and was used to verify that the values reported in Table 4.1 corresponded to the processed test data. Figure 4.10 shows an example of the terminal output generated during the analysis.

Log file	Readings	Mean Dev.	Max Dev.	Mean Change	Max Change	Jumps
gps_log_2026-09-08_22-10-59.csv	61	7.651 m	16.017 m	1.512 m	5.204 m	1
gps_log_2026-09-08_22-16-32.csv	59	6.071 m	25.039 m	1.937 m	11.331 m	5
gps_log_2026-09-08_22-21-59.csv	55	5.761 m	15.069 m	1.691 m	9.289 m	2
gps_log_2026-09-08_22-39-08.csv	65	7.859 m	23.341 m	2.162 m	10.446 m	6
gps_log_2026-09-09_00-44-00.csv	411	4.217 m	21.898 m	0.759 m	6.678 m	2
gps_log_2026-09-09_01-24-39.csv	5	0.112 m	0.233 m	0.126 m	0.244 m	0
gps_log_2026-09-09_01-25-23.csv	4036	4.856 m	15.985 m	0.468 m	2.985 m	0
gps_log_2026-09-13_16-42-21.csv	2949	8.688 m	51.475 m	1.385 m	15.692 m	55
gps_log_2026-09-14_07-01-11.csv	2995	4.857 m	27.727 m	0.795 m	7.090 m	5
gps_log_2026-09-14_12-44-34.csv	1046	4.272 m	13.934 m	0.603 m	2.651 m	0
gps_log_2026-09-14_16-48-37.csv	732	6.488 m	22.487 m	1.104 m	7.357 m	4
gps_log_2026-09-14_17-58-01.csv	572	4.587 m	13.139 m	0.856 m	4.173 m	0
gps_log_2026-09-16_20-57-11.csv	23	7.562 m	16.097 m	3.586 m	15.071 m	5
gps_log_2026-09-16_21-02-34.csv	16	15.061 m	36.847 m	4.807 m	11.139 m	5
gps_log_2026-09-16_21-04-07.csv	12	7.879 m	27.418 m	6.707 m	30.663 m	3
gps_log_2026-09-16_21-23-49.csv	6	0.641 m	1.164 m	0.535 m	0.855 m	0
gps_log_2026-09-16_21-26-07.csv	77	4.583 m	11.808 m	1.007 m	3.202 m	0
gps_log_2026-09-16_21-45-50.csv	17	4.455 m	8.985 m	1.504 m	3.256 m	0
gps_log_2026-09-16_22-46-27.csv	90	3.890 m	9.023 m	1.262 m	4.240 m	0
gps_log_2026-09-17_22-23-59.csv	60	8.034 m	14.000 m	0.900 m	3.089 m	0
gps_log_2026-09-17_22-34-54.csv	15	2.435 m	3.191 m	0.774 m	2.220 m	0
gps_log_2026-09-19_02-18-10.csv	54	3.741 m	9.940 m	1.109 m	2.744 m	0

OVERALL SUMMARY

Measurement	Value
Total CSV files read	22
Files containing GPS values	22
Empty CSV files	0
Total GPS readings	13356

GPS Metric	Result
Average positional deviation	5.623 m
Largest positional deviation	51.475 m
Average consecutive position change	1.618 m
Total jumps > 5 m	93

Figure 4.10: Terminal output produced by the GPS log analysis program.

The calculated results indicate that the GPS receiver exhibited measurable positional variation while remaining stationary. Across the 22 test sessions, the mean positional deviation was 5.623 m, with a maximum deviation of 51.475 m. The mean change between consecutive observations was 1.618 m. A total of 93 consecutive position changes exceeded 5m. These variations represent changes in the reported GPS position rather than physical movement of the receiver, and therefore indicate the positional consistency that can be expected from the GPS component under stationary conditions [6, 36].

The metrics in Table 4.1 were generated by the core analysis function shown in Listing 4.4. The function processes successive GPS observations, calculates the distance between reported positions, determines the positional deviations, and aggregates the resulting values into the summary statistics reported above.

```
def analyse_file(path):
    #Analyse one GPS CSV file.
    with open(path, newline="", encoding="utf-8") as f:
        f.readline()  # Skip metadata header line
        reader = csv.DictReader(f)
        if (not {"latitude", "longitude", "altitude_m", "speed_mps", "
            satellites_used"}.issubset(reader.fieldnames or [])):
            return None
        rows = list(reader)

    valid_rows = []
    for row in rows:
        try:
            valid_rows.append((row, float(row["latitude"]), float(row["longitude
                "])))
        except (KeyError, TypeError, ValueError):
            continue

    if (len(valid_rows) < 2): return None

    latitudes, longitudes = [r[1] for r in valid_rows], [r[2] for r in
        valid_rows]
    altitudes, speeds, satellites = [], [], []

    for r, _, _ in valid_rows:
        for target, key in [(altitudes, "altitude_m"), (speeds, "speed_mps"), (
            satellites, "satellites_used")]:
            try: target.append(float(r[key]))
            except (KeyError, TypeError, ValueError): pass

    ref_lat, ref_lon = statistics.mean(latitudes), statistics.mean(longitudes)
    deviations = [distance(ref_lat, ref_lon, lat, lon) for lat, lon in zip(
        latitudes, longitudes)]
    changes = [distance(latitudes[i - 1], longitudes[i - 1], latitudes[i],
        longitudes[i]) for i in range(1, len(latitudes))]

    return {
        "file": os.path.basename(path), "readings": len(valid_rows),
        "mean_deviation": statistics.mean(deviations), "std_deviation":
            statistics.stdev(deviations), "max_deviation": max(deviations),
        "lat_std": statistics.stdev(latitudes), "lon_std": statistics.stdev(
            longitudes),
        "alt_mean": statistics.mean(altitudes) if altitudes else None,
        "alt_std": statistics.stdev(altitudes) if len(altitudes) > 1 else None,
        "alt_range": max(altitudes) - min(altitudes) if altitudes else None,
        "speed_mean": statistics.mean(speeds) if speeds else None, "speed_max":
            max(speeds) if speeds else None,
        "sat_mean": statistics.mean(satellites) if satellites else None,
        "sat_min": min(satellites) if satellites else None, "sat_max": max(
            satellites) if satellites else None,
        "change_mean": statistics.mean(changes), "change_max": max(changes),
        "jumps": sum(c > JUMP_THRESHOLD for c in changes)
    }
```

Listing 4.4: Core function that produces the metrics in Table 4.1(`analysis.py`)

The GPS-based testing therefore established a baseline for the positional consistency of the GPS component under stationary conditions. Although the receiver produced measurable variation in its reported position, the majority of consecutive observations remained relatively close to one another, demonstrating that the GPS module was able to provide usable positional data through the Raspberry Pi Zero. These results provide a reference against which the GPS observations obtained during the subsequent field test can be compared under flight conditions.

4.4.2 Communication link testing results

Communication link testing was conducted using the transmitter and ground station described in Section 4.3. The transmission program sent each GPS reading as a group of six identical copies, and the ground station recorded the copies that it received. Six tests were done at distances of 5, 10 and 20 m, with two tests at each distance. In each test, 16 groups (96 copies) were transmitted at 60 s intervals. The UHF link was tested outside and under line of sight with the antennas placed on the ground facing upwards.

Two metrics were calculated from the logs. The *group success rate* is the percentage of transmitted groups for which at least one copy was received. The *copy reception rate* is the percentage of all transmitted copies that were received. The number of groups transmitted was determined from the range of transmission counters in the log, and counters missing from the sequence were counted as lost groups. The results are presented in Table 4.2.

Table 4.2: Performance of the UHF link.

Test	**Dist. (m)**	**Sent**	**Recv.**	**Groups (%)**	**Copies (%)**	**RSSI (mV)**
1	5	16	96	100	100	1900.125
2	5	16	95	100	98.958	1910.813
3	10	16	96	100	100	1917.25
4	10	16	96	100	100	1914.75
5	20	16	91	100	94.79	1814.563
6	20	16	95	100	98.958	1808.75

The results show the proportion of groups successfully received and the overall proportion of transmitted copies received. The number of copies received for each group indicates the effectiveness of the six-copy redundancy, particularly where groups were recovered despite only a subset of their copies being received. These results are compared with the requirement for reliable reception throughout the flight in Section 4.4.3.

The values in Table 4.2 were from the receiver logging program (Appendix E.5), which runs on the ground-station computer and reads the `LOG` lines sent by the Arduino over USB serial. Only valid frames whose payload matches the expected GPS format are processed. The program therefore groups the received copies by their transmission counter and counts how many copies of each group arrive. Once 10 s pass without a new copy of a group, the group is regarded as complete, and a single row is written to the log containing the GPS coordinates, the RSSI, and the number of copies received (between 1 and 6). A group with no received copies is never logged, but is identified from the gap in the sequence of transmission counters. Listing 4.5 shows the parsing of a received payload.

```python
def parse_gps_payload(payload_text):
    # Parse '<group_id>,<lat>,<lon>,<alt>,<hhmmss>'. Returns a dict or None if
        the payload doesn't match the expected GPS format (e.g. a non-GPS test
        beacon got decoded) - caller should just skip those.
    fields = payload_text.split(",")
    if len(fields) != 5:
        return None
    try:
        return {
            "group_id": int(fields[0]),
            "latitude": float(fields[1]),
            "longitude": float(fields[2]),
            "altitude_m": float(fields[3]),
            "fix_time_hhmmss": fields[4],
        }
    except ValueError:
        return None
```

Listing 4.5: Parsing of a received GPS payload in the logging program (`gps_receive_logger.py`).

Figure 4.11 shows the terminal output of the logging program during a test, in which the copies of each group are counted as they arrive and a single row is logged when the group is complete.

```
D:\Desktop\receiver_logger>py gps_receive_logger.py --port COM5
Listening on COM5 @ 115200 baud. Logging unique fixes to logs\gps_received.csv
Ctrl+C to stop.

[new]  group 0: first copy received
       group 0: copy #1 (RSSI=1871 mV)
       group 0: copy #2 (RSSI=1876 mV)
       group 0: copy #3 (RSSI=1876 mV)
       group 0: copy #4 (RSSI=1871 mV)
       group 0: copy #5 (RSSI=1871 mV)
       group 0: copy #6 (RSSI=1867 mV)
[logged] group 0: lat=-26.68875 lon=27.09527 alt=1336m fix_time=131840 copies_received=6
[new]  group 1: first copy received
       group 1: copy #1 (RSSI=1871 mV)
       group 1: copy #2 (RSSI=1876 mV)
       group 1: copy #3 (RSSI=1871 mV)
       group 1: copy #4 (RSSI=1876 mV)
       group 1: copy #5 (RSSI=1871 mV)
       group 1: copy #6 (RSSI=1862 mV)
[logged] group 1: lat=-26.68874 lon=27.09529 alt=1334m fix_time=131940 copies_received=6
[new]  group 2: first copy received
       group 2: copy #1 (RSSI=1871 mV)
       group 2: copy #2 (RSSI=1867 mV)
       group 2: copy #3 (RSSI=1871 mV)
       group 2: copy #4 (RSSI=1871 mV)
       group 2: copy #5 (RSSI=1871 mV)
       group 2: copy #6 (RSSI=1871 mV)
[logged] group 2: lat=-26.68875 lon=27.09530 alt=1325m fix_time=132040 copies_received=6
```

Figure 4.11: Terminal output of the receiver logging program, showing copies of each group being received and the final logged row, including the number of copies received.

The results demonstrate that the UHF link successfully received at least one copy of every transmitted group across the six tests conducted at distances of 5, 10 and 20 m. Overall, 569 of the 576 transmitted copies were received, giving a copy reception rate of approximately 98.8%. This indicates that transmitting six copies provided redundancy, allowing a group to be recovered even when some copies were not received. However, these tests were conducted over limited ground distances and do not establish the reliability of the communication link under flight conditions [11, 17]. The flight data discussed in the following section are therefore necessary to further assess the performance of the communication and tracking system during field testing.

4.4.3 Flight results

The complete artefact was flown on a balloon on 9 October 2026. This test is a demonstration of the artefact under real flight conditions and provides a comparison with the stationary baseline. All times below are given in UTC unless stated otherwise; local time (SAST) is UTC+2.

The airborne subsystem was powered by a modified power bank and transmitted on 434.650 MHz, one packet every 60 s with six copies of each packet. Three logs were analysed: the GPS log kept on the Raspberry Pi Zero, the transmission log of the telemetry subsystem, and the reception log of the ground station. The analysis was done with the script in Appendix E.6.

The GPS and the transmitter were switched on before the balloon was released, so the logs contain a stationary period at the launch site. Liftoff was identified from the GPS altitude. The balloon was stationary until the fix at 09:00:51 and was already climbing at the next fix at 09:01:21, so liftoff took place at about 09:01 (11:01 local time). Transmitted packets 0 to 30 were therefore sent on the ground, and packets 31 to 48 were sent during the ascent. The very last packet received at the ground station was packet 48.

Table 4.3 summarises the GPS data recorded during the flight and Figure 4.12 shows the altitude and the number of satellites used against time. Altitude increased steadily from the launch altitude of about 1 450 m to 5 360 m. The mean ascent rate over the logged period was 3.7 m/s. No dropouts or jumps in altitude were visible in the GPS log.

Table 4.3: Summary of the GPS data recorded during the flight test.

Metric	**Result**
Total GPS fixes logged (including ground period)	105
GPS fixes after liftoff	35
Logged time after liftoff (min)	17.7
Launch altitude (m)	1 448
Maximum altitude recorded (m)	5 360
Mean ascent rate (m/s)	3.7
Maximum horizontal speed (m/s)	15.3
Horizontal distance from launch at last fix (km)	6.0
Satellites used after liftoff (min / mean / max)	16 / 17.4 / 19

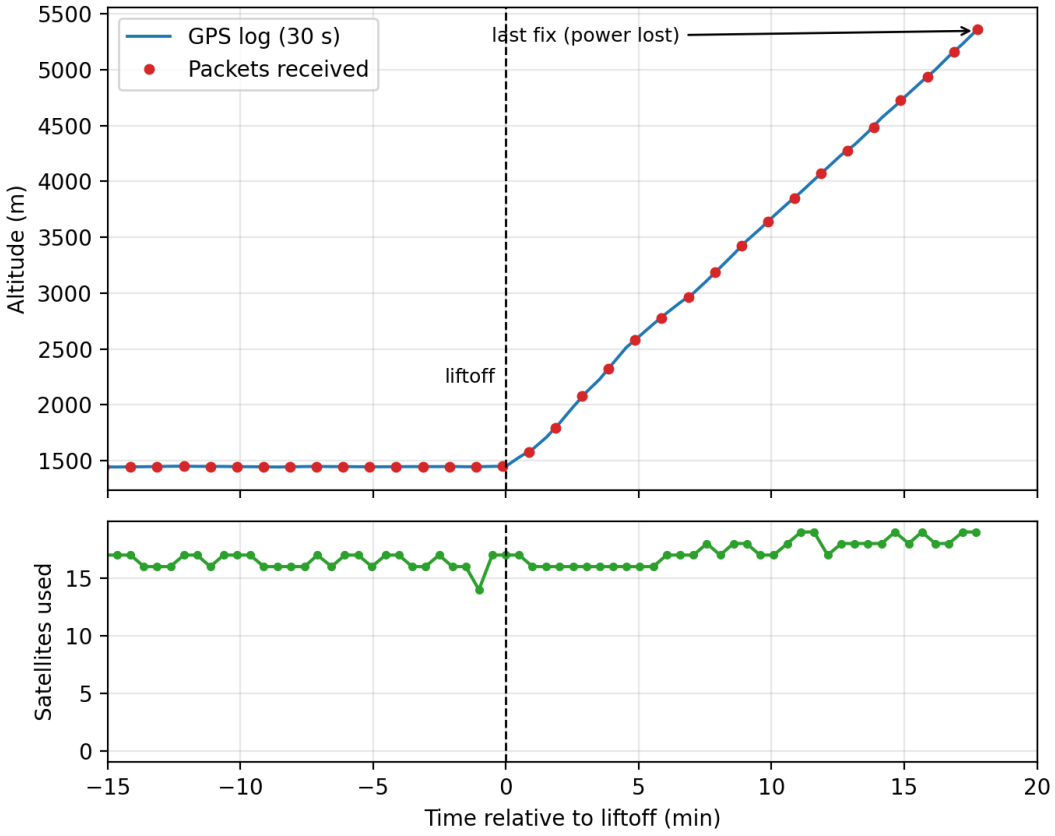


Figure 4.12: Altitude and number of satellites used against time relative to liftoff. The red markers are the positions received at the ground station. The record ends at the last fix before the power loss.

The GPS module kept a valid fix with at least 16 satellites for the whole ascent. The satellite count was slightly higher during the flight (mean 17.4) than during the last 15 minutes on the ground (mean 16.4), a small difference that may be due to a clearer view of the sky. The stationary ground tests in the previous section measured the repeatability of the reported position at a fixed point. A flight has no fixed reference position, so the two cannot be compared in metres. The comparison made here is of fix continuity and satellite count only. The recorded flight reached 5.4 km, so these results do not show how the module behaves above 12 km, where many GPS modules stop reporting unless an airborne dynamic mode is set.

Table 4.4 gives the results of the transmission performance. The ground station started logging at packet 12, meaning packets 0 to 11 were not recorded. One transmission (packet 23) was split over two rows of the reception log, one with a single copy and one with five copies. These were merged into one packet with six copies. Of the 37 packets sent since the receiver was started, all 37 were received, and the contents of every received packet matched what the transmission log shows as sent. All 18 packets sent after liftoff were also received.

Table 4.4: Telemetry link performance during the flight test. The ground column covers packets 12 to 30, and the flight column covers packets 31 to 48.

Metric	Ground	Flight
Packets received / sent	19 / 19	18 / 18
Packet success rate (%)	100	100
Copies decoded (of six per packet)	111 / 114	92 / 108
Copy success rate (%)	97.4	85.2
Packets with fewer than six copies	3 of 19	7 of 18
Mean RSSI (mV)	1 762	1 255
RSSI range (mV)	1 036 – 1 813	938 – 1 549
Transmission interval: mean 60.02 s, standard deviation 0.14 s (48 intervals)		
Maximum distance with successful reception: 7.2 km (6.0 km horizontal)		

The transmission interval was 60 s for 47 of the 48 intervals and 61 s for one, which is within the 1 s resolution of the log. The 61 s interval occurred when the packet grew from 32 to 33 bytes and the transmission time per packet increased from 8.06 s to 8.26 s.

Figure 4.13 shows RSSI and the number of copies decoded against the distance between the balloon and the ground station. The RSSI value is the analogue output of the NRX2, which was not calibrated to dBm, so only relative changes are interpreted. RSSI fell from about 1 800 mV at the launch site to mostly between 1 150 and 1 260 mV beyond 2 km. The correlation between RSSI and distance over the flight was $r = -0.74$. The decrease is not smooth, and the values at 5.2 km and 6.6 km are lower than the one at 7.2 km. One ground reading (packet 16, 1 036 mV) was also far below the others taken at the same position, so single RSSI values vary by several hundred millivolts without any change in distance.

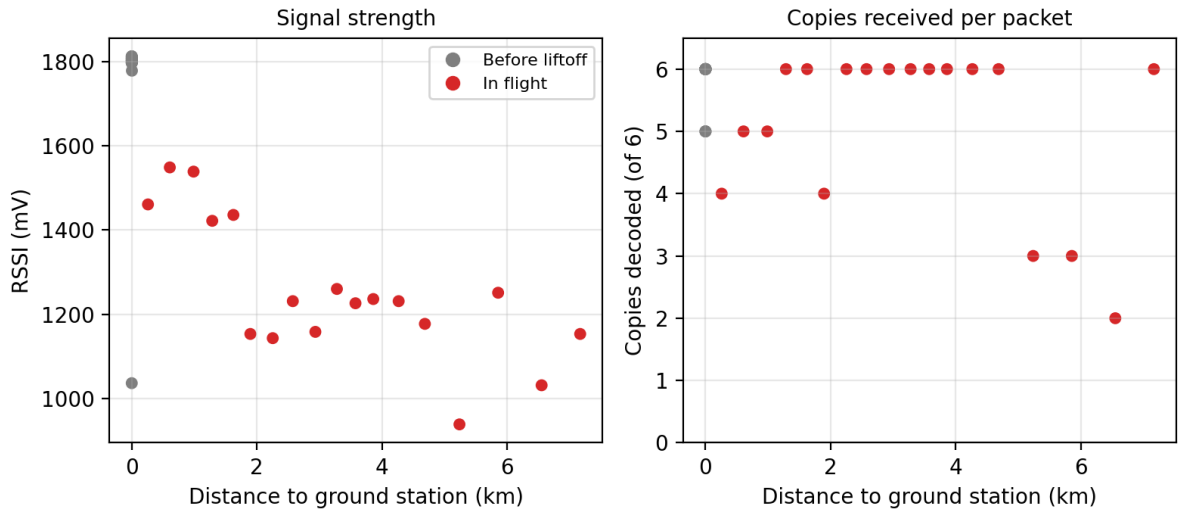


Figure 4.13: RSSI (left) and number of copies decoded per packet (right) against distance to the ground station. Grey points were recorded before liftoff and red points during the ascent.

The copies decoded per packet did not fall steadily with distance. Most packets sent within 5 km arrived with all six copies. Packets 45, 46 and 47, sent at 5.2 to 6.6 km, arrived with three, three and two copies. Packet 48, sent at 7.2 km, arrived with all six. Some of the packets sent in the first 2 km after liftoff also lost one or two copies. The data cannot show why. Possible causes are the radiation pattern of the antennas at the changing elevation angle, and local obstructions while the balloon was low, but neither was tested.

Every packet was decoded at least once, even when only two of six copies were received. This shows that the six-copy redundancy mechanism was needed to keep the packet success rate at 100 % during the ascent.

The GPS log ends at 09:18:33 (11:18 local), 17.7 minutes after liftoff and at an altitude of about 5.35 km. Packet 48 was sent at 09:18:52 and was received with all six copies, after which no further packets were received, and the transmission log has no entry after packet 48. The power bank failed to supply continuous power to the Raspberry Pi Zero while its battery still had more than enough charge. The balloon landed at about 15:30 local time. Because the last packet was received with all six copies from a distance of 7.2 km, the end of the data does not look like a loss of radio signal. It matches an abrupt loss of power. The cause of the sudden power bank failure was not confirmed but was possibly caused by loose wiring or some type of auto-shutoff behaviour.

The failure matters for the evaluation of the subsystems. The data shows how the GPS and telemetry performed during the first 17.7 minutes of the ascent, but they do not show performance above 5.4 km, at burst, during descent, or at greater range. The power supply is the weakest part of the airborne subsystem.

4.4.4 Summary of findings

The findings show that the artefact acquired GPS positions, transmitted them over the UHF link and logged them at the ground station, first in ground-based tests and then on a balloon flight. In the ground tests, the reported GPS position varied while the receiver was stationary. The mean positional deviation over 22 sessions was 5.623 m, but the maximum was 51.475 m, so the mean does not give a guaranteed search radius around the last reported coordinate [6, 36]. The link tests at 5, 10 and 20 m received at least one copy of every group and 569 of 576 copies (98.8%) [3, 16].

The flight test showed the same behaviour in the air during the initial ascent. Over about 17.5 minutes, the balloon climbed from about 1 450 m to about 5.4 km. The GPS module kept reporting positions with at least 16 satellites; all 18 packets sent after liftoff were received, and the last packet was received from 7.2 km away. Only 85.2% of the individual copies were received in flight, but every packet was decoded at least once, which is consistent with the six-copy scheme helping to protect each position update. The transmission interval was 60 s for 47 of the 48 intervals.

The flight also showed the main weakness of the airborne subsystem. The power bank stopped supplying the Raspberry Pi Zero after about 17.5 minutes while its battery was not depleted, and telemetry ended there. The flight did not reach stratospheric altitude, and it did not include burst, descent or the limit of the link's range. Against the requirements in Section 4.2.3, GPS acquisition and regular transmission, and reception and logging at the ground station, were met for the logged part of the flight. Sufficient range for stratospheric altitudes was not established. Operation within the power constraints was not achieved for a full flight. Redundancy was shown only in the repeated copies, since the power failure was not covered by any backup.

4.5 Conclusion

This chapter presented the design, development and evaluation of the communication and tracking artefact for a stratospheric balloon. Following the Design Science Research methodology, the GPS interface, the UHF transmitter and the ground station were built and tested in successive cycles, and the artefact was then evaluated with ground-based tests and a balloon flight. The ground tests showed usable but variable GPS positions and a reliable link over short distances. During the flight, the ground station artefact tracked the balloon and delivered every packet sent after liftoff, up to an altitude of about 5.4 km and a distance of 7.2 km, until the power bank stopped supplying power after about 17.5 minutes. These results support the design of the GPS and UHF link, but they do not establish its performance at stratospheric altitude, over a complete flight, or at the limit of its range. Chapter 5 uses these findings to answer the research question and to discuss the contributions and limitations of the study.

Chapter 5

Conclusion

5.1 Introduction

The communication and tracking system has to deliver position and telemetry data to a ground station over a long-range link while staying within the power, weight and bandwidth limits of a battery-powered payload. Section 1.3.2 proposes the following research question: how can a reliable and power-efficient communication and tracking system be built for a stratospheric balloon that operates at high altitudes? Chapter 4 reports the empirical part of the study, namely the artefact and its test results. The conclusion chapter addresses the empirical objective by judging those results against the aims in Section 1.3.3, and the theoretical objective by relating them to the literature reviewed in Chapter 3.

This chapter summarises the research, states what it contributed, describes what limited the communication and tracking system, and evaluates how trustworthy the findings are against the criteria set in Chapters 1 and 2. It then reflects on the decisions and the processes behind the work and ends with a short concluding statement.

The chapter begins by summarising the study and addressing the research question and objectives in Section 5.2. The contributions are outlined in Section 5.3, followed by a discussion of the study's limitations and possible future research in Section 5.4. Section 5.5 assesses the reliability and validity of the research, while Section 5.6 reflects on the research process. Finally, Section 5.7 brings the chapter to a close.

5.2 Overview of the research

The study followed the DSR [25] process within the pragmatic paradigm [7]. Cellular networks perform poorly for aerial users [4], whereas sub-GHz links offer long-range transmission at low power [26], so a 433 MHz UHF link combined with GPS positioning was chosen, with a ground station treated as part of the artefact. Three development cycles produced the GPS interface on the Raspberry Pi Zero, the NTX2 transmitter with its own frame structure that sends six copies of every frame, and a ground station in which an NRX2 receiver is decoded by an Arduino Uno R3 and logged on a computer.

Ground testing gave a mean positional deviation of 5.623 m (maximum 51.475 m) over 22 stationary sessions. In six link tests at 5 to 20 m, at least one copy of every group was received, and 98.8% of copies (569 of 576) arrived. These values are of the same order as the reception above 95% reported for LoRaWAN telemetry on balloons [16], although the conditions are too different for a direct comparison. Table 5.1 sets the outcome of each objective from the listed objectives in Chapter 1 against the evidence in Chapter 4.

Table 5.1: Outcome of each study objective.

Objective	**Outcome**	**Evidence and remaining gap**
Collect and transmit essential flight data	Achieved for GPS data	Counter, position, altitude and fix time were sent. Control and sensor data were not included.
GPS-based real-time tracking	Achieved up to 5.4 km	16 to 19 satellites and no fix loss during the ascent.
Long-range, low-power link at high altitude	Partly achieved	Reception at 7.2 km slant range. Stratospheric altitude and current draw not tested.
Optimise telemetry for bandwidth and power	Partly achieved	Payload of 32 to 35 bytes; no alternatives compared.
Redundancy for communication and recovery	Partly achieved	Six copies kept packet success at 100% in flight. No second tracker.
Reliable tracking and recovery of the payload	Partly achieved	The payload was successfully recovered using a separate GPS tracker provided by a supervisor; recovery was therefore not demonstrated using the developed communication and tracking system itself.
Evaluate through GPS and link testing	Achieved up to 5.4 km	Sections 4.4.1 to 4.4.3.

The research question is therefore answered in part. The tests show that the system can deliver position data reliably when each reading is transmitted repeatedly, both on the ground and during a 17.7-minute ascent to 5.4 km. However, reliability at the intended stratospheric altitude was not demonstrated. The balloon was recovered after the flight using a separate GPS tracker provided by the supervisor, rather than through the developed telemetry system alone. This confirms that the payload was recovered, but does not establish that the developed system could independently support recovery. Power efficiency was addressed through component selection and a compact payload design, but power consumption was not measured, and the flight exposed the power supply as a critical weakness.

5.3 Contributions of the research

The main result is a documented prototype of both ends of a UHF telemetry link: an airborne unit that acquires, frames and transmits GPS fixes, and a ground station that receives, validates and logs them. The wiring, frame format and source code are in Chapter 4 and Appendix A, so the system can be rebuilt. NTX2 and NRX2 interfaces are already described in community material [32] and multi-mode balloon transmitters have been published [3], so the contribution lies in a tested path from GPS fix to logged position that includes the receiving end, and in a record of where the hardware departed from its documentation. A 1 kΩ series resistor on the NTX2 input performed better than 470 Ω although both give nearly the same pin voltage [28], and the NRX2 audio output measured 4.4 V against a documented 0.8 V bias, so the digital output was used [28].

Two results come from the tests themselves. Repeating each frame kept packet success at 100% in flight although individual copies were lost (Section 4.4.3), and a stationary GPS receiver reported positions up to 51 m from its mean [36], so a search area should not be sized from the mean deviation alone. The flight also showed that the power supply was a critical point of failure, as its failure interrupted the telemetry system's operation. Although the balloon was subsequently recovered using an independent GPS tracker, this recovery depended on equipment outside the developed artefact. This reinforces the need for independent tracking and a more thoroughly tested power supply in future designs, consistent with the treatment of the power subsystem of a stratospheric experiment as a design problem of its own [19].

5.4 Limitations and future research

This section discusses the aspects that limited this study and the evaluation done on the artefact, followed by a discussion of future research for the communication and tracking system.

5.4.1 Limitations

The main limitation is the flight record. One flight gave 17.7 minutes of data and a maximum altitude of 5.4 km, so the stratosphere, burst, descent and landing were not observed. Chapter 1 planned multiple launches. With one flight, the drop in copy success cannot be separated from antenna orientation or terrain. The ground link tests covered at most 20 m, with two tests per distance and 16 groups per test. With only 18 packets sent during the ascent and none lost, the upper 95% bound on the packet failure rate is about 17% by the rule of three [10], which is too high to claim high reliability.

Current drawn by the payload was never logged, although the research question asks for power efficiency. The artefact uses one radio link, so repeated frames protect against lost frames but not against the loss of the transmitter, supply or Raspberry Pi, which is what happened. The GPS deviation is measured against each session's own mean and describes repeatability, not accuracy. Only GPS data were transmitted, so integration with the control and sensor subsystems is not demonstrated, and operation of the GPS module above 12 km was not verified [37]. Although the balloon was recovered using an external GPS tracker, the developed communication and tracking system was not demonstrated as an independent means of locating and recovering the payload.

5.4.2 Future research

A repeat flight with a power supply that has run for several hours on the bench with the complete payload and the current drawn recorded, and with a local log on the payload so that the flight record does not depend on reception. Range should be measured on the ground over several kilometres beforehand [11, 17]. Redundancy should go beyond repeated frames: Brown showed the value of more than one communication mechanism on one payload [3], and Janssen et al. describe hybrid architectures [14]. A second tracker on its own supply, possibly LoRaWAN [16], would remove the single point of failure, and a cellular tracker could help locate the payload after landing even though coverage is poor at high altitude [4]. The GPS module should be set to an airborne mode [37], a binary payload could shorten on-air time [26], and compliance of transmission power and duty cycle with the National Radio Frequency Plan [12] should be confirmed before further launches.

5.5 Evaluation of the study

The study is quantitative, so its trustworthiness is evaluated through the reliability and validity of its measurements. Chapters 1 and 2 committed to structured testing, repeated experiments, consistent logging, metrics tied to the objectives, and testing in the intended environment [5, 23, 30].

5.5.1 Reliability

All GPS data were produced and analysed by the same programs (Appendix A), so the 22 sessions and 13 356 readings were handled identically. The link tests were repeated at each distance. The pairs agree at 5 and 10 m but differ by four percentage points at 20 m (94.79% and 98.96% of copies), so two repetitions are few. In flight, the transmission interval had a mean of 60.02 s and a standard deviation of 0.14 s, and the content of every received packet matched the transmission log, which cross-checks the Pi, transmitter and ground station logs against each other. RSSI is an uncalibrated voltage, and single readings varied by several hundred millivolts at the same position, so it is used for trends only [28]. Frames are accepted only when the CRC matches and the payload has the expected five fields. An 8-bit CRC cannot exclude every corrupted frame, but an unnoticed wrong position in the logs is unlikely.

5.5.2 Validity

The GPS accuracy measurement has limited validity because the tests did not use a known reference position [6, 36]. The tests therefore show how consistent the GPS measurements were rather than how accurate they were. In contrast, packet and copy success directly measure whether the transmitted position data successfully reached the ground station.

For internal validity, hardware, programs and frame settings were the same throughout; frames were built for the antennas, and the flight distances assume that the ground station stood at the launch position. No numerical thresholds for acceptable performance were set before testing, so results were compared with the literature [16] and not with a pass mark. External validity is limited. Chapter 2 argued that balloon testing would strengthen it, but the flight reached 5.4 km and not the 20 to 30 km of the problem description [22], so the findings hold for the conditions tested [31, 35]. Measured against the DSR activities of Peffers et al. [25], problem definition, design, demonstration and communication were carried out, and evaluation is the incomplete activity. The criteria for DSR work [2, 24] would call for the repeat flights proposed above.

5.6 Reflection on the study

Establishing the GPS interface before the transmitter, and the transmitter before the receiver, kept each fault tied to one component. Writing the latest fix to a JSON file separated the GPS and transmission programs, so the transmitter could change without touching the GPS code, and the geofencing work in the group project could read the same file. Where the hardware did not behave as documented, measuring settled the matter faster than further reading. The main lesson was that a raw FM link needs framing [28], error detection and repetition added by the designer [3, 16], and that a system is only as reliable as its least tested part. The link and GPS were tested extensively, but the power supply was not, and the flight exposed it.

The strengths are the end-to-end scope including the ground station, the documented cycles, and the reporting of results that fall short of the objectives. The weaknesses are the shortage of flight data, the missing power measurements, the short link tests and the single radio link. Further testing is recommended on the complete payload on its flight power supply for longer than the planned flight with the current recorded, carry a second tracker, do range testing on the ground first, set performance thresholds before testing, and plan for more than one flight date.

The objectives were achieved in part, as Table 5.1 shows. The telemetry, tracking and ground-station objectives were demonstrated up to 5.4 km. Payload recovery was successful, but it relied on a separate GPS tracker provided by the supervisor rather than the developed telemetry system alone. Operation at the intended stratospheric altitude and power efficiency therefore remain unverified. The study delivered a tested prototype and an honest evaluation of its performance, but it does not establish that the developed system can independently provide reliable tracking and recovery under the intended operating conditions.

The project was managed through the DSR cycles and the milestones in Section 1.4.6. The planned documentation deadlines were generally met, although the empirical study chapter was completed slightly later than planned. The practical development was also affected by delays in obtaining opportunities for additional launches, as launch availability depended on supervisor availability. Within the artefact development, additional time was required to resolve the GPS wiring and determine the appropriate resistor configuration for the transmitter. The communication subsystem was deliberately developed around GPS data as its primary information source, allowing the subsystem to be developed and tested independently of the control and sensor subsystems. This helped maintain progress on the communication and tracking component, although complete integration of the other subsystem data was not demonstrated within the final artefact.

5.7 Conclusion of the study

This study set out to determine how a reliable and power-efficient communication and tracking system could be built for a stratospheric balloon. Following the Design Science Research methodology, a GPS and UHF telemetry system with a matching ground station was designed, built and tested in three iterative cycles. Ground tests and a flight to 5.4 km showed that position data can be acquired, framed, transmitted and logged reliably when each reading is repeated, with every packet received in the flight test. A power supply failure ended logging early. The system is therefore a validated prototype with limited flight evidence. Although the balloon was recovered using an independent GPS tracker, the flight did not demonstrate that the developed telemetry system could independently support payload recovery. Link reliability in the stratosphere and payload power consumption remain to be shown, which requires a repeat flight on a tested power supply, measured current draw and an independent backup tracker.

Bibliography

[1] Arduino, "Arduino uno rev3," Arduino Documentation, 2026. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/

[2] J. v. Brocke, A. Hevner, and A. Maedche, "Introduction to Design Science Research," *Progress in IS*, p. 1, 2020.

[3] B. Brown, "Multi-mode transmitter for high-altitude balloon telemetry," in *Academic High Altitude Conference*, vol. 2011, no. 1. Iowa State University Digital Press, 2011, pp. 1–6.

[4] A. Colpaert, E. Vinogradov, and S. Pollin, "Aerial Coverage Analysis of Cellular Systems at LTE and mmWave Frequencies Using 3D City Models," *Sensors*, pp. 1–13, 2018.

[5] J. W. Creswell and J. D. Creswell, *Research design: Qualitative, quantitative, and mixed methods approaches*. Sage publications, 2017.

[6] P. Enge, "The Global Positioning System: Signals, measurements, and performance," *Int. J. Wirel. Inf. Networks*, 2010.

[7] M. Y. Feilzer, "Doing mixed methods research pragmatically: Implications for the rediscovery of pragmatism as a research paradigm," *Journal of Mixed Methods Research*, vol. 4, no. 1, pp. 6–16, 2010.

[8] T. G. Guzik, "The High Altitude Student Platform (HASP) as a model multi-payload balloon platform," *2015 IEEE Aerospace Conference*, 2015.

[9] J. Habermas, *The Theory of Communicative Action: Volume 1*. Beacon Press, 1984.

[10] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, Apr. 1983.

[11] Z. Hasirci and I. H. Cavdar, "Propagation modeling dependent on frequency and distance for mobile communications via high altitude platforms (HAPs)," *2012 35th International Conference on Telecommunications and Signal Processing (TSP)*, 2012.

[12] Independent Communications Authority of South Africa, "National Radio Frequency Plan (NRFP) 2021," Tech. Rep., 2022, government Gazette No. 46088, 25 March 2022.

[13] Independent Communications Authority of South Africa (ICASA), "Radio Frequency Spectrum Regulations," 2011, government Gazette No. 34172, 31 March 2011.

[14] T. Janssen, A. Koppert, R. Berkvens, and M. Weyn, "A Survey on IoT Positioning Leveraging LPWAN, GNSS, and LEO-PNT," *IEEE Internet of Things Journal*, pp. 1–34, 2023.

[15] C. Kivunja and A. B. Kuyini, "Understanding and Applying Research Paradigms in Educational Contexts," 2017.

[16] J. Krueger and P. Bergmaier, "Testing Next-Generation Telemetry Using LoRaWAN On High-Altitude Balloons," *2024 Academic High Altitude Conference*, pp. 1–13, 2024.

[17] A. Larsson, A. Piotrowski, T. D. Giles, and D. Smart, "Near-earth RF propagation - Path loss and variation with weather," *2013 International Conference on Radar*, 2013.

[18] P. Leu, I. Puddu, A. Ranganathan, and S. Čapkun, "I Send, Therefore I Leak: Information Leakage in Low-Power Wide Area Networks," *arXiv: Cryptography and Security*, pp. 1–13, 2019.

[19] S. Marín-Coca, D. González-Bárcena, E. Roibás-Millán, and S. Pindado, "On the modeling and simulation of a stratospheric experiment power subsystem," *Acta Astronautica*, vol. 198, pp. 421–430, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0094576522003046

[20] A. Maxworth, "Worksheet 01 - monopole," Antenna Design With HFSS, 2022. [Online]. Available: https://digitalcommons.usm.maine.edu/oer$_m axworth_a dwafss$/10/

[21] R. Miura, T. Matsuda, F. Ono, T. Matsumura, J. Suzuki, and T. Takamori, "A Study on Spectrum Sharing for the User and Feeder Links of Command and Telemetry Communications for UAVs Using a High Altitude Relay System," *International Symposium on Wireless Personal Multimedia Communications*, 2021.

[22] A. Pankine, Z. Li, D. Parsons, M. Purucker, E. Weinstock, W. Wiscombe, and K. Nock, "Stratospheric Satellites for Earth Observations," 2009.

[23] Y. S. Park, L. Konge, and A. R. Artino, "The Positivism Paradigm of Research." *Academic medicine : journal of the Association of American Medical Colleges*, 2020.

[24] K. Peffers, T. Tuunanen, and B. Niehaves, "Design science research genres: introduction to the special issue on exemplars and criteria for applicable design science research," *European Journal of Information Systems*, pp. 129–137, 2018.

[25] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," vol. 24, no. 3, pp. 45–77, _eprint: https://doi.org/10.2753/MIS0742-1222240302. [Online]. Available: https://doi.org/10.2753/MIS0742-1222240302

[26] E. D. Poorter, J. Hoebeke, M. Strobbe, I. Moerman, S. Latré, M. Weyn, B. Lannoo, and J. Famaey, "Sub-GHz LPWAN Network Coexistence, Management and Virtualization: An Overview and Open Research Challenges," *Wireless personal communications*, pp. 1–16, 2017.

[27] P. Puričer, P. Kovář, J. Mikeš, and J. Kákona, "Telemetry system for research stratospheric balloon," *2017 International Conference on Applied Electronics (AE)*, 2017.

[28] Radiometrix Ltd, "Ntx2/nrx2 – uhf narrow band fm tx & rx datasheet," Radiometrix, 2012, issue 1, 30 July 2012. Accessed September 2026. [Online]. Available: https://www.radiometrix.com/datasheets/ntx2nrx2.pdf

[29] J. Saldaña, "The coding manual for qualitative researchers," 2021.

[30] G. Shanks, "Guidelines for Conducting Positivist Case Study Research in Information Systems," *Australas. J. Inf. Syst.*, 2002.

[31] P. Techavijit, S. Chivapreecha, P. Sukchalerm, and N. Wongphuangfuthaworn, "Suitable altitude for long-operated communication high altitude balloon with experimental flights," *2016 8th International Conference on Knowledge and Smart Technology (KST)*, 2016.

[32] UK High Altitude Society (UKHAS), "Linking an arduino to a radiometrix ntx2b transmitter," UKHAS Wiki, 2016, accessed September 2026. [Online]. Available: https://ukhas.org.uk/doku.php?id=guides:linkingarduinotontx2

[33] ——, "Payload antenna," UKHAS Wiki, 2016, last modified 7 April 2016. Accessed September 2026. [Online]. Available: https://ukhas.org.uk/doku.php?id=guides:payload$_a$*ntenna*

[34] M. Uzair, S. Y. Al-Kafrawi, K. M. Al-Janadi, and I. A. Al-Bulushi, "A Low-Cost IoT Based Buildings Management System (BMS) Using Arduino Mega 2560 And Raspberry Pi 4 For Smart Monitoring and Automation," *International journal of electrical and computer engineering systems*, pp. 1–48, 2022.

[35] Wang, Zhanchao and Huang, Min and Qian, Lulu and Zhao, Baowei and Wang, Guangming, "High-altitude balloon-based sensor system design and implementation," *Sensors*, vol. 20, no. 7, pp. 2–3, 2020. [Online]. Available: https://www.mdpi.com/1424-8220/20/7/2080

[36] M. Wing, "Consumer-Grade Global Positioning Systems (GPS) Receiver Performance," 2008.

[37] R. O. Wiyagi, T. A. Agus *et al.*, "High Altitude Balloon Payload Design for Atmospheric Observations," *Journal of Electrical Technology UMY*, vol. 1, no. 1, pp. 50–57, 2017.

Appendix A

Code of Conduct

NORTH-WEST UNIVERSITY
YUNIBESITI YA BOKONE-BOPHIRIMA
NOORDWES-UNIVERSITEIT

Research and Innovation

CODE OF CONDUCT FOR RESEARCHERS

This code of conduct is applicable to all NWU researchers.

As a researcher of the North-West University (NWU), I subscribe to the rules of the NWU Institutional Research Ethics Regulatory Committee (IRERC), all applicable policies of the NWU as well as all national and international laws and regulations applicable to my field of study. Furthermore, I commit myself to abide by the ethical principles and responsibilities as set out in the Singapore statement on Research Integrity (22 September 2010), in any and all research endeavours that I undertake as a researcher of the NWU.

The four major principles of research integrity to which I will adhere and that will guide my research are:

- Honesty in all aspects of research;
- Accountability in the conduct of research;
- Professional courtesy and fairness in working with others;
- Good stewardship of research on behalf of others.

Consequently I will also adhere to the following ethical responsibilities:

1. I will take responsibility for the originality and trustworthiness of my research.

2. I will stay abreast of and adhere to all institutional, national, and international laws, regulations, and policies applicable and related to my research.

3. I will at all times employ appropriate research methods, base my conclusions on critical analysis of the evidence and report my findings and interpretations fully and objectively.

4. I will keep clear and accurate records of all research that I have conducted in a manner that will allow verification and replication of my work by others, if applicable.

5. I will, where applicable, share my data and findings openly and promptly, in line with external funding rules. This will be done as soon as possible after I have had an opportunity to establish priority and ownership claims.

6. I will take responsibility for my own contributions to publications, funding applications, reports and other representations of my research. I will also and only include authors who meet valid authorship criteria.

7. I will acknowledge the names and roles of those who made significant contributions to my research in publications, including writers, funders, sponsors, and others, but do not meet authorship criteria.

8. In my peer reviews, I will provide fair, prompt and rigorous evaluations and I will respect confidentiality when I review others' work.

9. I will disclose all conflicts of interest (financial and other) that could compromise the trustworthiness of my work in research proposals, publications, public communications, and in review activities.

10. When I publically address a community in the spirit of academic freedom, I will in all stages base my professional comments on research findings (if applicable) and my expertise. I will distinguish between professional comments and opinions based on personal views.

11. Should any irresponsible research practices and/or research misconduct become known to me or brought under my attention, I will report such irresponsible research activities to the appropriate authorities.

12. I will respond to irresponsible research practices or conduct, by taking prompt actions as set out in the procedures of the university. I will also protect those who report misconduct in good faith, to the best of my abilities.

13. I will endeavour to create and sustain an environment that encourage research integrity through education of students, research teams and peers, as well as abide by policies, and reasonable standards for advancement.

14. I will at all times weigh societal benefits against the risks inherent in my work.

Name: Denzel Verster Signature:  Date: 07/04/2026

Original details: (11664754) P:\9. Research and Post-graduate Education\9.1 Implementation of the research strategy\9.1.5 Ethics\9.1.5.1.3_Code_Conduct_2017.docm
18 July 2017

File reference: 9.1.5.1.3

Appendix B

Student Study Leader Agreement

ITRI671 Research Project Agreement

This document will serve as a contract between the student and the supervisor.

Name of student: **Denzel Verster**

Student number: **45036349**

Name of supervisor: **Professor Gunther Drevin**

To ensure the success of your project, the following agreement is made between the supervisor and the student:

1.	Identifying the keywords and possible sources is the responsibility of the student and not of the supervisor.
2.	For any literature study, the student must make use of books, articles, conference papers and internet sources. (NO Wikipedia or questionable websites)
3.	It is the responsibility of the student to identify a primary objective and secondary objectives for the study. The supervisor will give guidance as to whether the objectives are achievable.
4.	Improvements on feedback received from the supervisor, e.g. formulation of statements, language use, referencing and technical finishing of the document are the responsibility of the student.
5.	Plagiarism is reviewed per chapter. A warning will be issued if plagiarism is detected. A second warning will result in disciplinary steps being taken. Adhere to the NWU guidelines regarding AI use.
6.	Keep to the schedule agreed upon and submit work on time. Late/No submission will result in a warning.
7.	Feedback from the supervisor will be given within an agreed upon timeframe after submission.
8.	Attend all meetings scheduled, be on time and be prepared. Mutual respect will be given to the time of the supervisor and the student.
9.	If any uncertainty or problem arises, the student will seek clarity from the supervisor immediately.
10.	The content of all the documentation and the empirical analysis must be the student's own work. The supervisor is not a source but should rather be seen as a guide (sounding board).
11.	The supervisor may at any time request the student to make use of the services of a language editor. The supervisor will not be responsible for language editing. Please ensure that your work is of a high standard in this regard.

Signature of student:  Date: 07/04/2026

Signature of supervisor: _____ Date: _____

If the student or supervisor is not satisfied with the effort of the other party –
please contact the module facilitator immediately to resolve the issue.

Appendix C

Declaration of Integrity

Plagiarism, Artificial Intelligence and Academic Integrity

- When you submit an assignment, it should always be your own individual work unless clearly indicated as a group activity.
- Copying text from other learners or sources without giving credit by citing the source and adding the source to your reference list is considered plagiarism.
- When you use information from a source, you should paraphrase the text. This means reformulating the information in your own words to explain what you have read. The assignment aims not to reproduce existing material but to ascertain whether you can integrate texts, add your own interpretation and/or critique the texts, and offer a creative solution to existing problems.
- It is your responsibility to familiarise yourself with the *NWU Academic Integrity Policy* and not make yourself guilty of *plagiarism*.
- It is generally assumed that using Artificial Intelligence (AI) tools such as ChatGPT is not allowed unless clearly stated. You should note that the NWU uses Turnitin to detect plagiarism and/or the use of Artificial Intelligence.
- You are responsible for confirming your lecturers' stance on using AI and familiarising yourself with the *NWU Guidelines on the Responsible and Ethical use of Artificial Intelligence*.
- Remember that language editing tools and/or paraphrasing tools are considered a form of AI and might be flagged by Turnitin. Always save a copy of your original, unedited work before you submit it to any of these tools.
- Plagiarism or the unethical use of AI is a serious offence. Disciplinary steps may include a significant deduction of marks and/or being reported on the AIITSA system. For more information, visit the NWU page on *Academic Integrity*.
- It is also unethical and unacceptable to do someone else's work, lend it to someone else, or make it available to be copied.

DECLARATION

Name: Denzel Verster

Student number: 45036349

Module: ITRI671

Supervisor: Professor Gunther Drevin

Title of the Research Project:

Communication and Tracking System for a Stratospheric Balloon

I hereby declare that:

1. The above research report is my own work and that all the sources that I have used or quoted have been indicated and acknowledged by means of complete references.
2. I submitted the research report to originality checking software and that it falls within the accepted requirements for originality (under supervisor of the supervisor).
3. No data presented in this work is fabricated or falsified.
4. I adhere to all the Plagiarism, Artificial Intelligence and Academic Integrity guidelines above.



Signature of Student Date 07/04/2026

Appendix D

Ethics Form

ITRI 671 Research ethics form: Honours project

Name: Denzel Verster

Title of project: Communication and Tracking System for a Stratospheric Balloon

Supervisor: Prof. Gunther Drevin

Co-supervisor: Mr. J. Heystek Grobler

Co-supervisor: Dr. Rudi Serfontein

Starting and end dates of project: 11 February 2026– 19 November 2026

Please discuss the ethical clearance with your supervisor by using the **General principles and guidelines document on eFundi.**.

Indicate the ethics level.

Level for this project	**1**	**2** X	**3**

1. Have you read the information available related to research ethics (Chapter 5 of Researching Information Systems and Computing; BJ Oates and Chapter 13 of Writing for computer science, J Zobel; Manual for post graduate studies, available on efundi)?	Yes X	No

2. Do you make use of people as source of data in your project (for example the completion of questionnaires or evaluation of products)? In what way? This study does not involve human participants as a source of data. All data will be generated through system testing, including ground-based experiments and high-altitude balloon launches.	Yes	No X

Appendix E

Programs

E.1 Ground-based GPS acquisition and logging program

```
#!/usr/bin/env python3
import gps, csv, json, os, time
from datetime import datetime, timezone

# Only latitude and longitude are required for a valid GPS position
REQUIRED_FIELDS = ["lat", "lon"]
MIN_MODE = 3  # 2 = 2D fix, 3 = 3D fix
LOG_INTERVAL = 30  # seconds

# Directory containing this Python script
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

# Log directory
LOG_DIR = os.path.join(SCRIPT_DIR, "logs")

# Create logs directory if it does not exist
os.makedirs(LOG_DIR, exist_ok=True)

# Time the program was started
START_TIME = datetime.now(timezone.utc)

# Unique log filename for this execution
LOG_FILENAME = (
    f"gps_log_{START_TIME.strftime('%Y-%m-%d_%H-%M-%S')}.csv"
)

LOG_FILE = os.path.join(LOG_DIR, LOG_FILENAME)
# File containing the latest valid GPS information
LATEST_FILE = os.path.join(SCRIPT_DIR, "latest_gps.json")

def is_complete(report):
    mode = getattr(report, "mode", 0)

    if (mode < MIN_MODE):
        return False, f"mode={mode} (need >= {MIN_MODE})"

    missing = [
```

```
        field for field in REQUIRED_FIELDS
        if getattr(report, field, None) is None
    ]

    if (missing):
        return False, f"missing: {', '.join(missing)}"
    return True, None

def create_log_file():
    # Create a new CSV file for this program execution.
    with open(LOG_FILE, "w", newline="") as file:
        writer = csv.writer(file)

        # Record when this logging session started
        writer.writerow([
            f"Logging session started: "
            f"{START_TIME.isoformat()}"
        ])

        writer.writerow([
            "system_time",
            "satellite_time",
            "latitude",
            "longitude",
            "altitude_m",
            "speed_mps",
            "track_deg",
            "satellites_used"
        ])

def log_gps_data(data):
    # Append a successful GPS fix to the current session log.
    with open(LOG_FILE, "a", newline="") as file:
        writer = csv.writer(file)

        writer.writerow([
            data["system_time"],
            data["satellite_time"],
            data["latitude"],
            data["longitude"],
            data["altitude_m"],
            data["speed_mps"],
            data["track_deg"],
            data["satellites_used"]
        ])

def update_latest_gps(data):
    # Write the latest valid GPS fix to a JSON file. The temporary file prevents
        another program from reading a partially-written JSON file.
    temporary_file = LATEST_FILE + ".tmp"
    with open(temporary_file, "w") as file:
        json.dump(data, file, indent=4)
    os.replace(temporary_file, LATEST_FILE)

def main():
    create_log_file()
```

```
    session = gps.gps(mode=gps.WATCH_ENABLE)
    print("GPS monitoring started.")
    print(f"Logging to: {LOG_FILE}")
    print(f"Latest GPS data: {LATEST_FILE}")
    print(f"Valid GPS fixes are logged every {LOG_INTERVAL} seconds.")
    print("Press Ctrl+C to stop.\n")
    last_log_time = 0

    while (True):
        try:
            report = session.next()
        except StopIteration:
            continue
        except KeyError:
            continue

        if (report.get("class") != "TPV"):
            continue

        ok, reason = is_complete(report)

        if (not ok):
            continue

        sats = getattr(session, "satellites_used", 0)

        # Optional GPS values are allowed to be unavailable
        altitude = getattr(report, "alt", None)
        speed = getattr(report, "speed", None)
        track = getattr(report, "track", None)

        # System time from the Raspberry Pi
        system_time = datetime.now(timezone.utc).isoformat()

        # GPS time received from the satellites through the GPS receiver
        satellite_time = getattr(report, "time", None)

        latest_valid_fix = {
            "system_time": system_time,
            "satellite_time": satellite_time,
            "latitude": report.lat,
            "longitude": report.lon,
            "altitude_m": altitude,
            "speed_mps": speed,
            "track_deg": track,
            "satellites_used": sats
        }

        # ALWAYS update the latest GPS information.
        # This means geofencing/transmission can access
        # the newest valid position independently of
        # the 5-second research logging interval.
        update_latest_gps(latest_valid_fix)

        current_time = time.monotonic()

```

```
        # Log one valid GPS measurement every 5 seconds
        if (current_time - last_log_time >= LOG_INTERVAL):
            log_gps_data(latest_valid_fix)
            print(
                f"SYSTEM TIME: {system_time}  |  "
                f"SATELLITE TIME: "
                f"{satellite_time if satellite_time is not None else 'N/A'}"
            )
            print(
                f"VALID FIX LOGGED  "
                f"lat={report.lat:.6f}  "
                f"lon={report.lon:.6f}  "
                f"alt={altitude if altitude is not None else 'N/A'}m  "
                f"speed={speed if speed is not None else 'N/A'}m/s  "
                f"track={track if track is not None else 'N/A'}deg  "
                f"sats_used={sats}"
            )
            last_log_time = current_time

if (__name__ == "__main__"):
    try:
        main()
    except KeyboardInterrupt:
        print("\nGPS monitoring stopped.")
```

E.2 GPS data analysis program

```
#!/usr/bin/env python3

import csv, math, os, statistics
from datetime import datetime

# CONFIGURATION
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
LOG_DIR = os.path.join(BASE_DIR, "logs")
OUTPUT_FILE = os.path.join(BASE_DIR, "log_analysis.txt")
JUMP_THRESHOLD = 5.0  # metres

# HELPER FUNCTIONS
def distance(lat1, lon1, lat2, lon2):
    #Calculate distance between two GPS coordinates in metres.
    R = 6371000
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
    a = min(1.0, max(0.0, math.sin(dp / 2) ** 2 + math.cos(p1) * math.cos(p2) *
        math.sin(dl / 2) ** 2))
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

def fmt(value, unit=""):
    #Format numerical values for the report.
    return "N/A" if value is None else f"{value:.3f}{unit}" if isinstance(value,
         float) else f"{value}{unit}"

def make_table(headers, rows):
```

```
    #Create a simple text table.
    if (not rows): return []
    widths = [max(len(str(headers[i])), max(len(str(row[i])) for row in rows))
        for i in range(len(headers))]
    sep = "+-" + "-+-".join("-" * w for w in widths) + "-+"
    header = "| " + " | ".join(str(headers[i]).ljust(widths[i]) for i in range(
        len(headers))) + " |"
    return [sep, header, sep] + ["| " + " | ".join(str(row[i]).ljust(widths[i])
        for i in range(len(headers))) + " |" for row in rows] + [sep]

# GPS LOG ANALYSIS
def analyse_file(path):
    #Analyse one GPS CSV file.
    with open(path, newline="", encoding="utf-8") as f:
        f.readline()  # Skip metadata header line
        reader = csv.DictReader(f)
        if (not {"latitude", "longitude", "altitude_m", "speed_mps", "
            satellites_used"}.issubset(reader.fieldnames or [])):
            return None
        rows = list(reader)

    valid_rows = []
    for row in rows:
        try:
            valid_rows.append((row, float(row["latitude"]), float(row["longitude
                "])))
        except (KeyError, TypeError, ValueError):
            continue

    if (len(valid_rows) < 2): return None

    latitudes, longitudes = [r[1] for r in valid_rows], [r[2] for r in
        valid_rows]
    altitudes, speeds, satellites = [], [], []

    for r, _, _ in valid_rows:
        for target, key in [(altitudes, "altitude_m"), (speeds, "speed_mps"), (
            satellites, "satellites_used")]:
            try: target.append(float(r[key]))
            except (KeyError, TypeError, ValueError): pass

    ref_lat, ref_lon = statistics.mean(latitudes), statistics.mean(longitudes)
    deviations = [distance(ref_lat, ref_lon, lat, lon) for lat, lon in zip(
        latitudes, longitudes)]
    changes = [distance(latitudes[i - 1], longitudes[i - 1], latitudes[i],
        longitudes[i]) for i in range(1, len(latitudes))]

    return {
        "file": os.path.basename(path), "readings": len(valid_rows),
        "mean_deviation": statistics.mean(deviations), "std_deviation":
            statistics.stdev(deviations), "max_deviation": max(deviations),
        "lat_std": statistics.stdev(latitudes), "lon_std": statistics.stdev(
            longitudes),
        "alt_mean": statistics.mean(altitudes) if altitudes else None,
        "alt_std": statistics.stdev(altitudes) if len(altitudes) > 1 else None,
        "alt_range": max(altitudes) - min(altitudes) if altitudes else None,
```

```
        "speed_mean": statistics.mean(speeds) if speeds else None, "speed_max":
            max(speeds) if speeds else None,
        "sat_mean": statistics.mean(satellites) if satellites else None,
        "sat_min": min(satellites) if satellites else None, "sat_max": max(
            satellites) if satellites else None,
        "change_mean": statistics.mean(changes), "change_max": max(changes),
        "jumps": sum(c > JUMP_THRESHOLD for c in changes)
    }

# MAIN
def main():
    print(f"\n{'=' * 78}\n                              GPS LOG ANALYSIS\n{'=' * 78}"
        )
    if (not os.path.exists(LOG_DIR)):
        os.makedirs(LOG_DIR)
        print(f"\nLog directory created at: {LOG_DIR}\nPlease place .csv log
            files in this directory and re-run.")
        return

    files = sorted(fn for fn in os.listdir(LOG_DIR) if fn.lower().endswith(".csv
        "))
    total_files, results, empty_files = len(files), [], []

    for filename in files:
        path = os.path.join(LOG_DIR, filename)
        try:
            res = analyse_file(path)
            if (res is None or res["readings"] < 2):
                empty_files.append(filename)
                print(f"  [EMPTY]    {filename}")
            else:
                results.append(res)
                print(f"  [ANALYSED] {filename:<42}{res['readings']:>4} readings
                    ")
        except Exception as error:
            empty_files.append(filename)
            print(f"  [ERROR]    {filename} ({error})")

    total_readings = sum(r["readings"] for r in results)

    # BUILD REPORT
    report = [
        f"{'=' * 78}\n                              GPS LOG ANALYSIS\n{'=' * 78}\n"
        f"Generated: {datetime.now().astimezone().isoformat()}\n\n"
        f"DETAILED MEASUREMENTS\n{'-' * 78}"
    ]

    if (results):
        for r in results:
            report.append(
                f"\nFILE: {r['file']}\nReadings: {r['readings']}\n\n"
                f"  Positional deviation: mean={fmt(r['mean_deviation'], ' m')},
                     std={fmt(r['std_deviation'], ' m')}, max={fmt(r['
                    max_deviation'], ' m')}\n"
                f"  Position variation: latitude std={fmt(r['lat_std'])},
                    longitude std={fmt(r['lon_std'])}\n"
```

```
                f"  Altitude: mean={fmt(r['alt_mean'], ' m')}, std={fmt(r['alt_std'], ' m')}, range={fmt(r['alt_range'], ' m')}\n"
                f"  Speed: mean={fmt(r['speed_mean'], ' m/s')}, max={fmt(r['speed_max'], ' m/s')}\n"
                f"  Satellites: mean={fmt(r['sat_mean'])}, min={fmt(r['sat_min'])}, max={fmt(r['sat_max'])}\n"
                f"  Consecutive position changes: mean={fmt(r['change_mean'], ' m')}, max={fmt(r['change_max'], ' m')}, jumps > {JUMP_THRESHOLD:.0f} m={r['jumps']}"
            )
    else:
        report.append("\nNo usable GPS logs were found.")

    # INDIVIDUAL LOG RESULTS
    report.append(f"\n\nINDIVIDUAL LOG RESULTS\n{'-' * 78}\n")
    if (results):
        headers = ["Log file", "Readings", "Mean Dev.", "Max Dev.", "Mean Change", "Max Change", "Jumps"]
        rows = [[r["file"], r["readings"], fmt(r["mean_deviation"], " m"), fmt(r["max_deviation"], " m"), fmt(r["change_mean"], " m"), fmt(r["change_max"], " m"), r["jumps"]] for r in results]
        report.extend(make_table(headers, rows))
    else:
        report.append("No usable GPS logs were found.")

    # OVERALL SUMMARY
    report.append(f"\n\n{'=' * 78}\n                                OVERALL SUMMARY\n{'=' * 78}\n")
    report.extend(make_table(["Measurement", "Value"], [
        ["Total CSV files read", total_files],
        ["Files containing GPS values", len(results)],
        ["Empty CSV files", len(empty_files)],
        ["Total GPS readings", total_readings]
    ]))

    if (results):
        m_devs = [r["mean_deviation"] for r in results]
        chgs = [r["change_mean"] for r in results if r["change_mean"] is not None]
        metrics = [
            ["Average positional deviation", f"{statistics.mean(m_devs):.3f} m"],
            ["Largest positional deviation", f"{max(r['max_deviation'] for r in results):.3f} m"],
            ["Average consecutive position change", f"{statistics.mean(chgs):.3f} m" if chgs else "N/A"],
            [f"Total jumps > {JUMP_THRESHOLD:.0f} m", sum(r["jumps"] for r in results)]
        ]
        report.append("\n")
        report.extend(make_table(["GPS Metric", "Result"], metrics))

    # INTERPRETATION
    report.append(
        f"\n\nINTERPRETATION\n{'-' * 78}\n\n"
```

```
        "Each CSV is analysed independently using its own mean latitude/
            longitude as the reference position.\n\n"
        "Therefore, positional deviation represents GPS consistency/
            repeatability rather than absolute accuracy against a surveyed
            reference coordinate.\n\n"
        "For a stationary receiver, speed should ideally remain close to 0 m/s.
            Consecutive position changes therefore indicate apparent GPS
            movement or measurement variation.\n\n"
        f"A position change greater than {JUMP_THRESHOLD:.0f} m is classified as
             a jump.\n\n"
        "Track is not included because it is not particularly meaningful when
            the GPS receiver is stationary."
    )

    # SAVE REPORT
    report_text = "\n".join(report)
    print(f"\n{report_text}")
    with open(OUTPUT_FILE, "w", encoding="utf-8") as f: f.write(report_text)
    print(f"\n{'=' * 78}\nReport saved to: {OUTPUT_FILE}\n{'=' * 78}")

    # EMPTY FILE CLEANUP
    if (empty_files):
        print(f"\n{'-' * 78}\nEMPTY CSV CLEANUP\n{'-' * 78}\n\nTotal files read:
                {total_files}\nFiles with GPS values:  {len(results)}\nEmpty
            files:            {len(empty_files)}\n\nA CSV with fewer than 2
            valid GPS readings is considered empty.\n")
        if (input("Delete the empty CSV files? [y/N]: ").strip().lower() in ("y"
            , "yes")):
            deleted = 0
            for fn in empty_files:
                try:
                    os.remove(os.path.join(LOG_DIR, fn))
                    deleted += 1
                    print(f"  Deleted: {fn}")
                except OSError as err:
                    print(f"  Could not delete {fn}: {err}")
            print(f"\nDeleted {deleted} of {len(empty_files)} empty CSV file(s).
                ")
        else:
            print("\nNo files were deleted.")

if (__name__ == "__main__"):
    main()
```

E.3 GPS-NTX2 transmission program

```
#!/usr/bin/env python3
"""
NTX2 GPS beacon transmitter (Raspberry Pi, RPi.GPIO, GPIO17).

Same as your gps_transmit.py (with the transmission_counter group-id
field you added), plus file logging: every line this script would have
printed to the console (what it transmitted, or a warning/error) is
also written to logs/gps_transmit.log in the current directory. It does
```

```
NOT log raw GPS JSON reads in between transmissions - only transmit
events and problems, exactly as you asked.

Expected JSON shape (extra fields are ignored):
    {
      "system_time": "...",
      "satellite_time": "2026-09-19T02:23:06.000Z",
      "latitude": -26.6886298,
      "longitude": 27.0952813,
      "altitude_m": 1344.117,
      "speed_mps": 0.07,
      "track_deg": 354.5929,
      "satellites_used": 9
    }

Payload format sent over the air:
    <transmission_counter>,<lat 5dp>,<lon 5dp>,<alt m, whole>,<HHMMSS of fix
        time>

Example: "1,-26.68863,27.09528,1344,022306"

The receiver sketch needs the LOG, line added (see rx_manchester_logging.ino)
for the companion gps_receive_logger.py to parse frames automatically.

Usage:
    python3 gps_transmit_v2.py
    python3 gps_transmit_v2.py --json-path /home/strato/latest_gps.json
    python3 gps_transmit_v2.py --interval 120 --repeats 6

Run with sudo for GPIO access / better timing.
"""

import argparse, gc, json, logging, os, time
from datetime import datetime, timezone
import RPi.GPIO as GPIO

# ---------- timing ----------

def wait_until(deadline):
    # Sleep most of the way, then spin for the last 2 ms for accuracy.
    while True:
        remaining = deadline - time.perf_counter()
        if (remaining <= 0):
            return
        if (remaining > 0.0025):
            time.sleep(remaining - 0.002)

def play(pin, levels, tick_seconds, idle_level):
    # Output one level per tick, then return the line to its idle level.
    gc.disable()
    try:
        deadline = time.perf_counter()
        for level in levels:
            GPIO.output(pin, level)
            deadline += tick_seconds
            wait_until(deadline)
```

```
    finally:
        GPIO.output(pin, idle_level)
        gc.enable()

# ---------- Manchester framing (matches the Arduino decoder exactly) ----------

PREAMBLE = b"\xAA" * 6
SYNC = b"\x2D\xD4"
MAX_PAYLOAD_BYTES = 40  # must match MAX_PAYLOAD in the Arduino sketch

def crc8(data):
    crc = 0
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = ((crc << 1) ^ 0x07) & 0xFF if (crc & 0x80) else (crc << 1) & 0
                xFF
    return crc

def build_frame(payload):
    if len(payload) == 0 or len(payload) > MAX_PAYLOAD_BYTES:
        raise ValueError(f"payload must be 1-{MAX_PAYLOAD_BYTES} bytes, got {len
            (payload)}")
    body = bytes([len(payload)]) + payload
    return PREAMBLE + SYNC + body + bytes([crc8(body)])

def to_half_bits(frame):
    # Manchester (IEEE 802.3): bit 1 = LOW then HIGH, bit 0 = HIGH then LOW.
    halves = []
    for byte in frame:
        for i in range(7, -1, -1):
            halves.extend([0, 1] if ((byte >> i) & 1) else [1, 0])
    return halves

def send_burst(pin, payload, repeats, gap_seconds, bit_us):
    # Send `repeats` copies of the same frame, each separated by gap_seconds of
        idle.
    half_seconds = bit_us / 2 / 1_000_000
    one_frame_halves = to_half_bits(build_frame(payload))
    per_copy_seconds = len(one_frame_halves) * half_seconds

    for i in range(repeats):
        play(pin, one_frame_halves, half_seconds, GPIO.LOW)
        if i < repeats - 1 and gap_seconds > 0:
            time.sleep(gap_seconds)

    total_seconds = repeats * per_copy_seconds + max(0, repeats - 1) *
        gap_seconds
    return total_seconds, per_copy_seconds

# ---------- GPS JSON -> compact payload ----------
def fix_time_hhmmss(satellite_time_str):
    # Extract HHMMSS from an ISO-ish timestamp. Falls back to current UTC time
        if parsing fails.
    try:
        s = satellite_time_str.replace("Z", "+00:00")
```

```
        dt = datetime.fromisoformat(s)
    except (ValueError, AttributeError, TypeError):
        dt = datetime.now(timezone.utc)
    return dt.strftime("%H%M%S")

def build_gps_payload(fix, transmission_counter):
    lat = float(fix["latitude"])
    lon = float(fix["longitude"])
    alt = float(fix["altitude_m"])
    hhmmss = fix_time_hhmmss(fix.get("satellite_time"))

    text = f"{transmission_counter},{lat:.5f},{lon:.5f},{alt:.0f},{hhmmss}"
    payload = text.encode("ascii")

    if len(payload) > MAX_PAYLOAD_BYTES:
        text = f"{transmission_counter},{lat:.4f},{lon:.4f},{alt:.0f},{hhmmss}"
        payload = text.encode("ascii")

    return text, payload

def read_latest_fix(json_path):
    with open(json_path, "r") as f:
        return json.load(f)

# ---------- logging setup ----------

def setup_logger(log_dir):
    os.makedirs(log_dir, exist_ok=True)
    log_path = os.path.join(log_dir, "gps_transmit.log")

    logger = logging.getLogger("gps_transmit")
    logger.setLevel(logging.INFO)
    logger.propagate = False

    fmt = logging.Formatter("%(asctime)s  %(levelname)-7s %(message)s", datefmt=
        "%Y-%m-%d %H:%M:%S")

    file_handler = logging.FileHandler(log_path)
    file_handler.setFormatter(fmt)
    logger.addHandler(file_handler)

    console_handler = logging.StreamHandler()
    console_handler.setFormatter(fmt)
    logger.addHandler(console_handler)

    return logger, log_path

# ---------- main ----------
def parse_args():
    p = argparse.ArgumentParser(description="NTX2 GPS beacon transmitter")
    p.add_argument("--gpio", type=int, default=17)
    p.add_argument("--bit-us", type=int, default=4000,
                   help="Manchester bit period in microseconds (default 4000,
                       matches tested setup)")
    p.add_argument("--repeats", type=int, default=6,
```

```
                    help="copies of the frame per reading, back-to-back (default 6)")
    p.add_argument("--gap-ms", type=float, default=0.0,
                    help="idle gap in milliseconds between copies within one burst (default 0)")
    p.add_argument("--interval", type=float, default=60.0,
                    help="seconds between readings/bursts (default 60)")
    p.add_argument("--json-path", default="latest_gps.json",
                    help="path to the GPS JSON file (default: latest_gps.json in the current directory)")
    p.add_argument("--log-dir", default="logs",
                    help="directory for gps_transmit.log (default: ./logs)")
    p.add_argument("--seconds", type=float, default=0,
                    help="stop after N seconds total (0 = run until Ctrl+C)")
    return p.parse_args()

def main():
    args = parse_args()
    logger, log_path = setup_logger(args.log_dir)

    try:
        os.sched_setscheduler(0, os.SCHED_FIFO, os.sched_param(50))
    except (AttributeError, PermissionError, OSError):
        pass

    GPIO.setmode(GPIO.BCM)
    GPIO.setup(args.gpio, GPIO.OUT, initial=GPIO.LOW)

    end_time = None if (args.seconds <= 0) else time.monotonic() + args.seconds
    logger.info(f"NTX2 GPS beacon on GPIO{args.gpio}. Ctrl+C to stop.")
    logger.info(f"Reading: {args.json_path}")
    logger.info(f"Logging transmissions to: {log_path}")
    logger.info(f"Sending {args.repeats} copies per reading, {args.gap_ms:g} ms gap between copies, "
                f"every {args.interval:g}s.")

    transmission_counter = 0  # group identifier for this run's transmissions
    try:
        while (end_time is None or time.monotonic() < end_time):
            cycle_start = time.monotonic()
            try:
                fix = read_latest_fix(args.json_path)
                text, payload = build_gps_payload(fix, transmission_counter)

                total_s, per_copy_s = send_burst(
                    args.gpio, payload, args.repeats, args.gap_ms / 1000.0, args.bit_us
                )
                logger.info(f"sent: {text}  ({len(payload)} bytes)  x{args.repeats} copies  "
                            f"({total_s:.2f} s total, {per_copy_s:.2f} s/copy)")
                transmission_counter += 1

            except FileNotFoundError:
                logger.warning(f"{args.json_path} not found - skipping this cycle")
```

```
            except (json.JSONDecodeError, KeyError, TypeError, ValueError) as e:
                logger.warning(f"bad/incomplete GPS data ({e}) - skipping this
                    cycle")

            elapsed = time.monotonic() - cycle_start
            sleep_for = max(0.0, args.interval - elapsed)
            time.sleep(sleep_for)
    except KeyboardInterrupt:
        pass
    finally:
        GPIO.output(args.gpio, GPIO.LOW)
        GPIO.cleanup()
        logger.info("Stopped.")

if (__name__ == "__main__"):
    main()
```

E.4 Arduino-NRX2 receiver program

```
/*
  NRX2 Manchester receiver (Arduino Uno) - with machine-parseable logging

  Same decoder as rx_manchester.ino. The only addition: every completed
  frame (good or bad CRC) also prints a single-line, comma-separated
  "LOG," record meant for a PC-side script to parse automatically:

      LOG,<OK|BAD>,<millis>,<RSSI_mV>,<payload text>

  The payload text itself is whatever ASCII was sent - for the GPS
  beacon that's "<group_id>,<lat>,<lon>,<alt>,<hhmmss>", so a LOG line
  looks like:

      LOG,OK,45231,1862,5,-26.68863,27.09528,1344,022306

  A parser should split on ',' with maxsplit=4: the first four fields
  are fixed (LOG, status, millis, RSSI), and everything after the 4th
  comma is the raw payload text (which may itself contain commas).

  The original human-readable line is kept too, for anyone watching the
  Serial Monitor live.

  Wiring (NRX2 powered from the Uno's 5V):
    NRX2 pin 7  RXD   -> D2   (direct, no resistor)
    NRX2 pin 3  RSSI  -> A0
    NRX2 pin 5  Vcc   -> 5V,  pins 4 and 2 -> GND,  pin 1 -> antenna

  Pair with gps_transmit_v2.py / ntx2_tx*.py --mode manchester.
  BIT_US must match the transmitter's --bit-us (default 4000).
*/

const byte RXD_PIN = 2;                 // INT0
const byte RSSI_PIN = A0;
const unsigned long BIT_US = 4000;      // Manchester bit period (must match the
    Pi)
```

```
const uint16_t SYNC_WORD = 0x2DD4;
const byte MAX_PAYLOAD = 40;

// ---------- edge queue (filled by the interrupt) ----------
const byte QUEUE_SIZE = 64;              // must be a power of two
volatile unsigned long edgeTime[QUEUE_SIZE];
volatile byte edgeLevel[QUEUE_SIZE];
volatile byte qHead = 0;
volatile byte qTail = 0;
volatile unsigned int overflowCount = 0;

void onEdge() {
  byte next = (qHead + 1) & (QUEUE_SIZE - 1);
  if (next == qTail) {
    overflowCount++;
    return;
  }
  edgeTime[qHead] = micros();
  edgeLevel[qHead] = digitalRead(RXD_PIN);   // level AFTER this edge
  qHead = next;
}

// ---------- decoder state ----------
const byte ST_SYNC = 0, ST_LEN = 1, ST_PAYLOAD = 2, ST_CRC = 3;

unsigned long lastEdgeUs = 0;
bool haveLast = false;
bool locked = false;        // true once the preamble has fixed the bit timing
bool atMid = false;         // was the previous edge a mid-bit edge?
byte longCount = 0;
byte state = ST_SYNC;
uint16_t shiftReg = 0;
bool inverted = false;
byte bitCount = 0;
byte curByte = 0;
byte payloadLen = 0;
byte payloadPos = 0;
byte payload[MAX_PAYLOAD + 1];
byte crcCalc = 0;

unsigned long edgesSeen = 0;
unsigned long locks = 0;
unsigned long framesOk = 0;
unsigned long framesBad = 0;

byte crc8Update(byte crc, byte data) {
  crc ^= data;
  for (byte i = 0; i < 8; i++) {
    crc = (crc & 0x80) ? (byte)((crc << 1) ^ 0x07) : (byte)(crc << 1);
  }
  return crc;
}

void resetDecoder() {
  locked = false;
  atMid = false;
```

```
  longCount = 0;
  state = ST_SYNC;
  shiftReg = 0;
  bitCount = 0;
  curByte = 0;
}

void printFrame(bool ok) {
  unsigned long nowMs = millis();
  long rssiMv = analogRead(RSSI_PIN) * 5000L / 1023L;

  // Human-readable line (for live viewing in the Serial Monitor)
  Serial.print(nowMs);
  Serial.print(F(" ms  "));
  Serial.print(ok ? F("OK  ") : F("BAD CRC  "));
  Serial.print(F("RSSI="));
  Serial.print(rssiMv);
  Serial.print(F(" mV  ok/bad="));
  Serial.print(framesOk);
  Serial.print('/');
  Serial.print(framesBad);
  Serial.print(F("  | "));
  for (byte i = 0; i < payloadLen; i++) {
    char c = payload[i];
    Serial.print((c >= 32 && c < 127) ? c : '.');
  }
  Serial.println();

  // Machine-parseable line for the companion PC script
  Serial.print(F("LOG,"));
  Serial.print(ok ? F("OK") : F("BAD"));
  Serial.print(',');
  Serial.print(nowMs);
  Serial.print(',');
  Serial.print(rssiMv);
  Serial.print(',');
  for (byte i = 0; i < payloadLen; i++) {
    char c = payload[i];
    Serial.print((c >= 32 && c < 127) ? c : '.');
  }
  Serial.println();
}

void onByte(byte value) {
  if (state == ST_LEN) {
    if (value == 0 || value > MAX_PAYLOAD) { resetDecoder(); return; }
    payloadLen = value;
    payloadPos = 0;
    crcCalc = crc8Update(0, value);
    state = ST_PAYLOAD;
  } else if (state == ST_PAYLOAD) {
    payload[payloadPos++] = value;
    crcCalc = crc8Update(crcCalc, value);
    if (payloadPos >= payloadLen) state = ST_CRC;
  } else if (state == ST_CRC) {
    bool ok = (value == crcCalc);
```

```
    if (ok) framesOk++; else framesBad++;
    printFrame(ok);
    resetDecoder();
  }
}

void onBit(byte bit) {
  if (state == ST_SYNC) {
    shiftReg = (shiftReg << 1) | bit;
    bool found = false;
    if (shiftReg == SYNC_WORD) { inverted = false; found = true; }
    else if (shiftReg == (uint16_t)~SYNC_WORD) { inverted = true; found = true; }
    if (found) {
      state = ST_LEN;
      bitCount = 0;
      curByte = 0;
    }
    return;
  }

  if (inverted) bit ^= 1;
  curByte = (curByte << 1) | bit;
  bitCount++;
  if (bitCount == 8) {
    byte value = curByte;
    bitCount = 0;
    curByte = 0;
    onByte(value);
  }
}

void handleEdge(unsigned long t, byte level) {
  if (!haveLast) { lastEdgeUs = t; haveLast = true; return; }
  unsigned long dt = t - lastEdgeUs;
  lastEdgeUs = t;

  byte cls = 0;                                    // 0 = invalid
  if (dt >= BIT_US / 4 && dt < BIT_US * 3 / 4) cls = 1;          // short
  else if (dt >= BIT_US * 3 / 4 && dt < BIT_US * 3 / 2) cls = 2; // long

  if (cls == 0) { resetDecoder(); return; }

  if (!locked) {
    if (cls == 2) {
      if (++longCount >= 6) {
        locked = true;
        atMid = true;
        locks++;
        onBit(level);
      }
    } else {
      longCount = 0;
    }
    return;
  }
```

```
202
203   if (atMid) {
204     if (cls == 1) {
205       atMid = false;
206     } else {
207       onBit(level);
208     }
209   } else {
210     if (cls == 1) {
211       atMid = true;
212       onBit(level);
213     } else {
214       resetDecoder();
215     }
216   }
217 }
218
219 void setup() {
220   pinMode(RXD_PIN, INPUT);
221   Serial.begin(115200);
222   Serial.println(F("NRX2 MANCHESTER RECEIVER (with LOG, output)"));
223   attachInterrupt(digitalPinToInterrupt(RXD_PIN), onEdge, CHANGE);
224 }
225
226 void loop() {
227   static unsigned int lastOverflow = 0;
228   static unsigned long lastStatusMs = 0;
229
230   while (qTail != qHead) {
231     unsigned long t = edgeTime[qTail];
232     byte level = edgeLevel[qTail];
233     qTail = (qTail + 1) & (QUEUE_SIZE - 1);
234     edgesSeen++;
235     handleEdge(t, level);
236   }
237
238   if (overflowCount != lastOverflow) {
239     lastOverflow = overflowCount;
240     resetDecoder();
241   }
242
243   if ((locked || longCount > 0) && (micros() - lastEdgeUs > BIT_US * 4)) {
244     resetDecoder();
245   }
246
247   if (millis() - lastStatusMs >= 2000) {
248     lastStatusMs = millis();
249     Serial.print(F("[status] RSSI="));
250     Serial.print(analogRead(RSSI_PIN) * 5000L / 1023L);
251     Serial.print(F(" mV  edges="));
252     Serial.print(edgesSeen);
253     Serial.print(F("  preamble locks="));
254     Serial.print(locks);
255     Serial.print(F("  frames ok/bad="));
256     Serial.print(framesOk);
257     Serial.print('/');
```

```
    Serial.print(framesBad);
    Serial.println();
  }
}
```

E.5 Receiver logging program

```
#!/usr/bin/env python3
"""
GPS receive logger (PC/laptop side, connected to the Arduino over USB serial).

Reads the Arduino's serial output, looks for lines of the form:

    LOG,<OK|BAD>,<millis>,<RSSI_mV>,<payload text>

and, for OK frames with payload "<group_id>,<lat>,<lon>,<alt>,<hhmmss>",
tracks how many copies of each group_id have arrived. It does NOT write a
duplicate row to the log for every copy - instead, once FINALIZE_SECONDS
has passed with no new copy of a given group, it writes exactly ONE row
to logs/gps_received.csv recording the fix data and how many copies were
received for that transmission.

Requires pyserial:
    pip install pyserial

Usage:
    python3 gps_receive_logger.py --port /dev/ttyACM0
    python3 gps_receive_logger.py --port COM3 --baud 115200

Find your port:
    Linux/Mac: ls /dev/tty.*  or  ls /dev/ttyACM* /dev/ttyUSB*
    Windows:   Device Manager -> Ports (COM & LPT)
"""

import argparse, csv, os, time, serial
from datetime import datetime, timezone

FINALIZE_SECONDS = 10.0   # how long to wait after the last copy before
                          # treating a transmission group as "done" and
                          # writing its final row with the copy count

def parse_log_line(line):
    # Parse a 'LOG,...' line. Returns (status, millis, rssi_mv, payload_text) or
         None if the line isn't a valid LOG line.
    if not line.startswith("LOG,"):
        return None
    parts = line.split(",", 4)
    if len(parts) != 5:
        return None
    _, status, millis_str, rssi_str, payload_text = parts
    try:
        millis = int(millis_str)
        rssi_mv = int(rssi_str)
    except ValueError:
```

```
47          return None
48      return status, millis, rssi_mv, payload_text
49
50  def parse_gps_payload(payload_text):
51      # Parse '<group_id>,<lat>,<lon>,<alt>,<hhmmss>'. Returns a dict or None if
            the payload doesn't match the expected GPS format (e.g. a non-GPS test
            beacon got decoded) - caller should just skip those.
52      fields = payload_text.split(",")
53      if len(fields) != 5:
54          return None
55      try:
56          return {
57              "group_id": int(fields[0]),
58              "latitude": float(fields[1]),
59              "longitude": float(fields[2]),
60              "altitude_m": float(fields[3]),
61              "fix_time_hhmmss": fields[4],
62          }
63      except ValueError:
64          return None
65
66  def ensure_csv_header(csv_path):
67      is_new = not os.path.exists(csv_path)
68      if is_new:
69          os.makedirs(os.path.dirname(csv_path), exist_ok=True)
70      f = open(csv_path, "a", newline="")
71      writer = csv.writer(f)
72      if is_new:
73          writer.writerow([
74              "logged_at_utc", "group_id", "latitude", "longitude",
75              "altitude_m", "fix_time_hhmmss", "rssi_mv", "copies_received"
76          ])
77          f.flush()
78      return f, writer
79
80  def main():
81      p = argparse.ArgumentParser(description="GPS receive logger (Arduino serial
            -> CSV)")
82      p.add_argument("--port", required=True, help="serial port, e.g. /dev/ttyACM0
            or COM3")
83      p.add_argument("--baud", type=int, default=115200)
84      p.add_argument("--log-dir", default="logs", help="directory for gps_received
            .csv (default: ./logs)")
85      p.add_argument("--finalize-seconds", type=float, default=FINALIZE_SECONDS,
86                     help=f"seconds of silence before finalizing a group (default
                           {FINALIZE_SECONDS})")
87      args = p.parse_args()
88
89      csv_path = os.path.join(args.log_dir, "gps_received.csv")
90      csv_file, csv_writer = ensure_csv_header(csv_path)
91
92      print(f"Listening on {args.port} @ {args.baud} baud. Logging unique fixes to
            {csv_path}")
93      print("Ctrl+C to stop.\n")
94
```

```
    # group_id -> {"data": {...}, "rssi_mv": int, "copies": int, "last_seen": float}
    groups = {}

    def finalize_group(group_id):
        g = groups.pop(group_id)
        data = g["data"]
        csv_writer.writerow([
            datetime.now(timezone.utc).isoformat(),
            data["group_id"],
            f'{data["latitude"]:.5f}',
            f'{data["longitude"]:.5f}',
            f'{data["altitude_m"]:.0f}',
            data["fix_time_hhmmss"],
            g["rssi_mv"],
            g["copies"],
        ])
        csv_file.flush()
        print(f"[logged] group {group_id}: "
              f"lat={data['latitude']:.5f} lon={data['longitude']:.5f} "
              f"alt={data['altitude_m']:.0f}m fix_time={data['fix_time_hhmmss']} "
              f"copies_received={g['copies']}")

    try:
        with serial.Serial(args.port, args.baud, timeout=1) as ser:
            while True:
                raw = ser.readline()
                if raw:
                    try:
                        line = raw.decode("ascii", errors="replace").strip()
                    except Exception:
                        line = None

                    if line:
                        parsed = parse_log_line(line)
                        if parsed:
                            status, millis, rssi_mv, payload_text = parsed
                            if status == "OK":
                                gps = parse_gps_payload(payload_text)
                                if gps:
                                    gid = gps["group_id"]
                                    now = time.monotonic()
                                    if gid not in groups:
                                        groups[gid] = {
                                            "data": gps,
                                            "rssi_mv": rssi_mv,
                                            "copies": 0,
                                            "last_seen": now,
                                        }
                                        print(f"[new]  group {gid}: first copy received")
                                    groups[gid]["copies"] += 1
                                    groups[gid]["last_seen"] = now
                                    print(f"       group {gid}: copy #{groups[gid]['copies']} "
```

```
                                        f"(RSSI={rssi_mv} mV)")

                # Finalize any groups that have gone quiet for long enough
                now = time.monotonic()
                done = [gid for gid, g in groups.items()
                        if now - g["last_seen"] >= args.finalize_seconds]
                for gid in done:
                    finalize_group(gid)

    except KeyboardInterrupt:
        pass
    finally:
        # Flush out whatever groups were still pending at shutdown
        for gid in list(groups.keys()):
            finalize_group(gid)
        csv_file.close()
        print("\nStopped.")

if __name__ == "__main__":
    main()
```

E.6 Flight Analysis program

```
#!/usr/bin/env python3
"""
Flight-test analysis for the stratospheric balloon communication and
tracking artefact.

Inputs (all from the 2026-10-09 flight):
  gps_log_*.csv      airborne GPS log (UTC timestamps, 30 s interval)
  gps_transmit.log   airborne transmission log (LOCAL time, UTC+2)
  gps_received.csv   ground-station reception log (UTC timestamps)

Outputs (written to OUT_DIR):
  flight_altitude.png    altitude and satellites against time
  flight_link.png        RSSI and copies received against distance
  flight_summary.txt     every number used in Chapter 4

Usage:
  python3 flight_analysis.py
  python3 flight_analysis.py --gs-lat -26.9029 --gs-lon 27.1806 --gs-alt 1447
"""
import argparse, re
from datetime import timedelta
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

LOCAL_OFFSET = timedelta(hours=2)    # SAST = UTC+2
CLIMB_THRESHOLD_M = 30               # rise above pre-launch median = airborne
EARTH_R = 6371000.0
```

```
32
33  def haversine(lat1, lon1, lat2, lon2):
34      p1, p2 = np.radians(lat1), np.radians(lat2)
35      dp, dl = p2 - p1, np.radians(lon2) - np.radians(lon1)
36      a = np.sin(dp / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dl / 2) ** 2
37      return 2 * EARTH_R * np.arcsin(np.sqrt(a))
38
39  def load_gps(path):
40      g = pd.read_csv(path, skiprows=1)
41      g["t"] = pd.to_datetime(g["satellite_time"], utc=True)
42      return g
43
44  def load_tx(path):
45      rows = []
46      pat = re.compile(
47          r"^(\d{4}-\d\d-\d\d \d\d:\d\d:\d\d)\s+INFO\s+sent: "
48          r"(\d+),([-\d.]+),([-\d.]+),(\d+),(\d+)\s+\((\d+) bytes\)")
49      for line in Path(path).read_text().splitlines():
50          m = pat.match(line)
51          if m:
52              ts, gid, lat, lon, alt, ft, nbytes = m.groups()
53              t_local = pd.Timestamp(ts)
54              rows.append(dict(group_id=int(gid),
55                               t_sent=(t_local - LOCAL_OFFSET).tz_localize("UTC"),
56                               lat=float(lat), lon=float(lon), alt=int(alt),
57                               fix_time=ft, nbytes=int(nbytes)))
58      return pd.DataFrame(rows)
59
60  def load_rx(path):
61      r = pd.read_csv(path)
62      # A transmission can be split across two log rows (seen for group 23).
63      # Merge rows with the same group_id: sum copies, mean RSSI.
64      merged = (r.groupby("group_id", as_index=False)
65                  .agg(latitude=("latitude", "first"),
66                       longitude=("longitude", "first"),
67                       altitude_m=("altitude_m", "first"),
68                       fix_time_hhmmss=("fix_time_hhmmss", "first"),
69                       rssi_mv=("rssi_mv", "mean"),
70                       copies_received=("copies_received", "sum"),
71                       logged=("logged_at_utc", "first")))
72      n_split = int(r.group_id.duplicated().sum())
73      return merged, n_split
74
75  def main():
76      ap = argparse.ArgumentParser()
77      ap.add_argument("--gps", default="/mnt/user-data/uploads/gps_log_2026-10-09
            _08-25-55.csv")
78      ap.add_argument("--tx", default="/mnt/user-data/uploads/gps_transmit.log")
79      ap.add_argument("--rx", default="/mnt/user-data/uploads/gps_received.csv")
80      ap.add_argument("--out", default="/mnt/user-data/outputs/flight")
81      ap.add_argument("--gs-lat", type=float, default=None,
82                      help="ground station latitude (default: launch position)")
83      ap.add_argument("--gs-lon", type=float, default=None)
84      ap.add_argument("--gs-alt", type=float, default=None)
85      a = ap.parse_args()
86
```

```
    out = Path(a.out)
    out.mkdir(parents=True, exist_ok=True)
    S = []                                      # summary lines

    def say(s=""):
        print(s)
        S.append(s)

    g, tx = load_gps(a.gps), load_tx(a.tx)
    rx, n_split = load_rx(a.rx)

    # ---------- liftoff detection from GPS altitude ----------
    pre = g[g.t < g.t.iloc[0] + pd.Timedelta(minutes=30)]    # after fix settles
    pre_alt = pre[pre.satellites_used >= 10].altitude_m.median()
    climbing = g[(g.altitude_m > pre_alt + CLIMB_THRESHOLD_M) & (g.
        satellites_used >= 10)]
    first_climb = climbing.t.iloc[0]
    last_still = g[g.t < first_climb].t.iloc[-1]
    t0 = last_still                                          # t = 0 reference
    g["min"] = (g.t - t0).dt.total_seconds() / 60
    fl = g[g.t >= first_climb].copy()                        # flight segment
    pre_launch = g[(g.t <= last_still) & (g.satellites_used >= 10)]

    # ground station position
    gs_lat = a.gs_lat if a.gs_lat is not None else pre_launch.latitude.median()
    gs_lon = a.gs_lon if a.gs_lon is not None else pre_launch.longitude.median()
    gs_alt = a.gs_alt if a.gs_alt is not None else pre_alt
    gs_assumed = a.gs_lat is None

    say("=== LIFTOFF ===")
    say(f"Pre-launch median altitude: {pre_alt:.1f} m")
    say(f"Last stationary fix : {last_still:%H:%M:%S} UTC ({(last_still+
        LOCAL_OFFSET):%H:%M:%S} local)")
    say(f"First climbing fix  : {first_climb:%H:%M:%S} UTC ({(first_climb+
        LOCAL_OFFSET):%H:%M:%S} local)")
    say("NOTE: stated release time was 10:59 local. The GPS shows the balloon "
        "stationary until the fix above.")

    # ---------- flight overview ----------
    last = g.iloc[-1]
    dur_min = (last.t - t0).total_seconds() / 60
    dist_launch = haversine(gs_lat, gs_lon, last.latitude, last.longitude)
    asc = np.polyfit(fl["min"] * 60, fl.altitude_m, 1)[0]  # m/s linear fit
    gain = fl.altitude_m.max() - pre_alt
    say("\n=== FLIGHT OVERVIEW (GPS log) ===")
    say(f"Liftoff lies between two fixes, so time after liftoff is "
        f"{(last.t-first_climb).total_seconds()/60:.1f} to {dur_min:.1f} min")
    say(f"Total GPS fixes logged        : {len(g)}")
    say(f"Fixes in flight segment       : {len(fl)}")
    say(f"Logged time from liftoff (min): {dur_min:.1f}")
    say(f"Last fix                      : {last.t:%H:%M:%S} UTC ({(last.t+
        LOCAL_OFFSET):%H:%M:%S} local)")
    say(f"Maximum altitude (m)          : {g.altitude_m.max():.0f}")
    say(f"Altitude gain above launch (m): {gain:.0f}")
    say(f"Mean ascent rate, linear fit (m/s): {asc:.2f}")
    say(f"Max horizontal speed (m/s)    : {fl.speed_mps.max():.1f}")
```

```
    say(f"Mean horizontal speed (m/s)   : {fl.speed_mps.mean():.1f}")
    say(f"Horizontal distance at last fix (km): {dist_launch/1000:.2f}")
    say(f"Satellites in flight min/mean/max: {fl.satellites_used.min()} / "
        f"{fl.satellites_used.mean():.1f} / {fl.satellites_used.max()}")
    say(f"Satellites pre-launch (>=10 sats) min/mean/max: "
        f"{pre_launch.satellites_used.min()} / {pre_launch.satellites_used.mean
            ():.1f} / "
        f"{pre_launch.satellites_used.max()}")
    last15 = g[(g.t > last_still - pd.Timedelta(minutes=15)) & (g.t <=
        last_still)]
    say(f"Satellites, last 15 min before liftoff mean/min/max: "
        f"{last15.satellites_used.mean():.1f} / {last15.satellites_used.min()} /
             {last15.satellites_used.max()}")
    say(f"Flight fixes with >=4 satellites: {(fl.satellites_used>=4).sum()}/{len
        (fl)}")
    gap = g.t.diff().dt.total_seconds().dropna()
    say(f"GPS log interval mean/max (s)  : {gap.mean():.1f} / {gap.max():.0f}")
    say(f"Cold-start: first fix with >=4 sats at "
        f"{g[g.satellites_used>=4].t.iloc[0]:%H:%M:%S} UTC "
        f"(log started {g.t.iloc[0]:%H:%M:%S})")

    # ---------- transmission performance ----------
    first_air = int(tx[tx.t_sent > first_climb].group_id.min())
    last_ground = first_air - 1
    iv = tx.t_sent.diff().dt.total_seconds().dropna()
    say("\n=== TRANSMISSION (transmit log) ===")
    say(f"Packets transmitted (0..{tx.group_id.max()}): {len(tx)}")
    say(f"Last pre-launch packet: {last_ground}; first airborne packet: {
        first_air}")
    say(f"Airborne packets sent: {first_air}..{tx.group_id.max()} "
        f"({tx.group_id.max()-first_air+1})")
    say(f"Transmission interval mean/std/min/max (s): "
        f"{iv.mean():.2f} / {iv.std():.2f} / {iv.min():.0f} / {iv.max():.0f}")
    say(f"Intervals != 60 s: {(iv != 60).sum()} of {len(iv)} "
        f"(log resolution is 1 s)")
    say(f"Packet length: {tx.nbytes.min()}-{tx.nbytes.max()} bytes; 6 copies per
         packet")

    # ---------- reception ----------
    say("\n=== RECEPTION (ground station) ===")
    say(f"Receiver log starts at packet {rx.group_id.min()} "
        f"(packets 0..{rx.group_id.min()-1} not logged: receiver not yet running
            )")
    say(f"Split log rows merged: {n_split}")
    exp = tx[tx.group_id >= rx.group_id.min()]
    got = set(rx.group_id)
    missing = sorted(set(exp.group_id) - got)
    say(f"Packets expected (since receiver start): {len(exp)}; received: {len(
        got & set(exp.group_id))}; "
        f"missing: {missing if missing else 'none'}")

    # payload integrity: compare each received packet to what was sent
    m = exp.merge(rx, on="group_id")
    bad = m[(abs(m.lat - m.latitude) > 1e-5) | (abs(m.lon - m.longitude) > 1e-5)
            | (m.alt != m.altitude_m) | (m.fix_time != m.fix_time_hhmmss.astype(
                str).str.zfill(6))]
```

```
    say(f"Received payloads differing from transmitted payloads: {len(bad)} of {
        len(m)}")

    m["phase"] = np.where(m.group_id <= last_ground, "ground", "flight")
    m["hdist_km"] = haversine(gs_lat, gs_lon, m.latitude, m.longitude) / 1000
    m["slant_km"] = np.sqrt(m.hdist_km**2 + ((m.altitude_m - gs_alt) / 1000) **
        2)
    for ph, d in m.groupby("phase"):
        say(f"[{ph}] packets {d.group_id.min()}..{d.group_id.max()} (n={len(d)})
            : "
            f"success {len(d)}/{len(d)}; copies/packet mean {d.copies_received.
                mean():.2f} "
            f"(min {d.copies_received.min()}); copy success {d.copies_received.
                sum()}/{6*len(d)} "
            f"= {100*d.copies_received.sum()/(6*len(d)):.1f}%; RSSI mean {d.
                rssi_mv.mean():.0f} mV "
            f"(min {d.rssi_mv.min():.0f}, max {d.rssi_mv.max():.0f})")
    f = m[m.phase == "flight"]
    say(f"Max slant range with successful reception: {f.slant_km.max():.2f} km "
        f"(horizontal {f.hdist_km.max():.2f} km, packet {int(f.loc[f.slant_km.
            idxmax(),'group_id'])})")
    say(f"Flight packets received with fewer than 6 copies: "
        f"{(f.copies_received<6).sum()} of {len(f)}")
    say(f"Lowest flight copy counts: "
        f"{f.sort_values('copies_received')[['group_id','copies_received','
            slant_km']].head(4).to_dict('records')}")
    r_corr = np.corrcoef(f.slant_km, f.rssi_mv)[0, 1]
    say(f"Pearson r (RSSI vs slant range, flight): {r_corr:.2f}")
    # Ground-phase RSSI outliers
    gnd = m[m.phase == "ground"]
    out_g = gnd[gnd.rssi_mv < gnd.rssi_mv.median() - 300]
    if len(out_g):
        say(f"Ground-phase RSSI outliers (>300 mV below median): "
            f"{out_g[['group_id','rssi_mv']].to_dict('records')}")
    # Reception latency
    m["logged_t"] = pd.to_datetime(m.logged, utc=True)
    lat_s = (m.logged_t - m.t_sent).dt.total_seconds()
    say(f"Send-to-log latency mean/max (s): {lat_s.mean():.1f} / {lat_s.max():.1
        f}")

    # ---------- last packet vs final GPS log fix ----------
    say("\n=== FINAL POSITION ===")
    lp = tx.iloc[-1]
    say(f"Last received packet {int(lp.group_id)}: {lp.lat:.5f}, {lp.lon:.5f}, "
        f"{lp.alt} m, fix {lp.fix_time} UTC")
    say(f"Last GPS log fix: {last.latitude:.5f}, {last.longitude:.5f}, "
        f"{last.altitude_m:.0f} m, {last.t:%H%M%S} UTC")
    say(f"Ground station position used for distances: {gs_lat:.5f}, {gs_lon:.5f
        }, {gs_alt:.0f} m "
        f"({'ASSUMED = launch position' if gs_assumed else 'supplied'})")

    # ---------- figures ----------
    plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3})

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 5.6), sharex=True,
                                   gridspec_kw={"height_ratios": [2, 1]})
```

```
    ax1.plot(g["min"], g.altitude_m, "-", color="C0", lw=1.4, label="GPS log (30 s)")
    mm = m.copy()
    mm["min"] = (pd.to_datetime(mm.fix_time_hhmmss.astype(str).str.zfill(6),
                                format="%H%M%S").dt.time.apply(
        lambda t: t.hour * 3600 + t.minute * 60 + t.second) -
        (t0.hour * 3600 + t0.minute * 60 + t0.second)) / 60
    ax1.plot(mm["min"], mm.altitude_m, "o", ms=4, color="C3", label="Packets received")
    ax1.axvline(0, color="k", ls="--", lw=1)
    ax1.text(-0.4, 2200, "liftoff", ha="right", fontsize=9)
    ax1.annotate("last fix (power lost)",
                 (g["min"].iloc[-1], g.altitude_m.iloc[-1]),
                 xytext=(3.0, 5250), textcoords="data", ha="center", fontsize=9,
                 arrowprops=dict(arrowstyle="->"))
    ax1.set_xlim(-15, 20)
    ax1.set_ylabel("Altitude (m)")
    ax1.legend(loc="upper left")
    ax2.plot(g["min"], g.satellites_used, "-o", ms=3, color="C2")
    ax2.axvline(0, color="k", ls="--", lw=1)
    ax2.set_ylabel("Satellites used")
    ax2.set_xlabel("Time relative to liftoff (min)")
    fig.tight_layout()
    fig.savefig(out / "flight_altitude.png", dpi=200)
    plt.close(fig)

    fig, (b1, b2) = plt.subplots(1, 2, figsize=(8, 3.6))
    col = np.where(m.phase == "ground", "C7", "C3")
    b1.scatter(m.slant_km, m.rssi_mv, c=col, s=22)
    b1.set_xlabel("Distance to ground station (km)")
    b1.set_ylabel("RSSI (mV)")
    b1.set_title("Signal strength", fontsize=10)
    b2.scatter(m.slant_km, m.copies_received, c=col, s=22)
    b2.set_xlabel("Distance to ground station (km)")
    b2.set_ylabel("Copies decoded (of 6)")
    b2.set_ylim(0, 6.5)
    b2.set_title("Copies received per packet", fontsize=10)
    from matplotlib.lines import Line2D
    b1.legend(handles=[Line2D([], [], marker="o", ls="", color="C7", label="Before liftoff"),
                       Line2D([], [], marker="o", ls="", color="C3", label="In flight")],
              loc="upper right", fontsize=8)
    fig.tight_layout()
    fig.savefig(out / "flight_link.png", dpi=200)
    plt.close(fig)

    (out / "flight_summary.txt").write_text("\n".join(S) + "\n")
    print(f"\nWrote figures and flight_summary.txt to {out}")

if __name__ == "__main__":
    main()
```